GROUP 3 DOCUTRUST

DocuTrust Defence Handbook

Seventy-eight themed examination questions with model answers, per-student defence packs, six Masterclass deep dives for all six members, and the eight core concepts explained for absolute beginners

- 78 themed questions across six themes: architecture, Java, security, database, hard challenges, mobile
- 6 per-student packs: owned modules, feature claims, pitches, personal Q&A
- 6 Masterclass deep dives: file-by-file ownership, five analogies each, code walkthroughs, glossaries, 90 personal Q&As
- Proof style: greppable finding IDs in code, mvn clean package from a clean clone, live manual API demos (Postman/cURL)
- Appendix: hooks, architecture, folders, database, server, tech stack and the data API - like I'm 5
- Companion to the DocuTrust Project Guide, DEFENCE_GUIDE.md and the repository docs

Table of Contents

How to Use This Handbook.....

Section A — Architecture and Design (Q1–Q12).....

Section B — Java Backend Without Spring (Q13–Q24).....

Section C — Authentication and Security (Q25–Q38).....

Section D — Database and Transactions (Q39–Q50).....

Section E — The Hardest Challenges, Deep Dive (Q51–Q62).....

Section F — React Native, MVP, Testing and Teamwork (Q63–Q78).....

Per-Student Defence Packs.....

- Student 1 — Infrastructure & Database.....
- Student 2 — Auth & OTP Security.....
- Student 3 — Business Logic (Documents, Sharing, Verification).....
- Student 4 — Mobile App (React Native 0.88 / Expo SDK 58).....
- Student 5 — The 8-Pass Security Audit.....
- Student 6 — Deployment & DevOps.....

The Defence Masterclass — Deep Dives for Each Student.....

Student 1 — Infrastructure & Database.....

- My slice in one page.....
- Elevator pitch (≈ 1 minute).....
- What I own, file by file.....
- Five layman's analogies.....
- The critical code walkthrough.....
- My domain, in plain words (glossary).....
- My ultimate Q&A bank.....
- The handshake — how my slice feeds the other five.....

Student 2 — Auth & OTP Security.....

- My slice in one page.....
- Elevator pitch (≈ 1 minute).....

What I own, file by file.....
Five layman's analogies.....
The critical code walkthrough.....
My domain, in plain words (glossary).....
My ultimate Q&A bank.....
The handshake — how my slice feeds the other five.....
Student 3 — Business Logic (Documents, Sharing, Verification).....
My slice in one page.....
Elevator pitch (≈ 1 minute).....
What I own, file by file.....
Five layman's analogies.....
The critical code walkthrough.....
My domain, in plain words (glossary).....
My ultimate Q&A bank.....
The handshake — how my slice feeds the other five.....
Student 4 — Mobile App.....
My slice in one page.....
Elevator pitch (≈ 1 minute).....
What I own, file by file.....
Five layman's analogies.....
The critical code walkthrough.....
My domain, in plain words (glossary).....
My ultimate Q&A bank.....
The handshake — how my slice feeds the other five.....
Student 5 — The 8-Pass Security Audit.....
My slice in one page.....
Elevator pitch (≈ 1 minute).....
What I own, file by file.....
Five layman's analogies.....
The critical code walkthrough.....

My domain, in plain words (glossary).....
My ultimate Q&A bank.....
The handshake — how my slice feeds the other five.....
Student 6 — Deployment & DevOps.....
My slice in one page.....
Elevator pitch (≈ 1 minute).....
What I own, file by file.....
Five layman's analogies.....
The critical code walkthrough.....
My domain, in plain words (glossary).....
My ultimate Q&A bank.....
The handshake — how my slice feeds the other five.....
Appendix — The Eight Core Concepts, Explained for Absolute Beginners.....
1. React Hooks — the sticky notes that remember things (rehearse: Student 4).....
2. The Overall Architecture — the customer, the waiter, the kitchen (rehearse: Student 1).....
3. The Frontend Folders — it's like a smartphone (rehearse: Student 4).....
4. The Backend Folders — a well-organized restaurant (rehearse: Student 1).....
5. The Database — a highly organized filing cabinet (rehearse: Student 1).....
6. The Server — a 24/7 receptionist answering phones (rehearse: Student 6).....
7. The Tech Stack — what we chose and why (rehearse: Student 6).....
8. The Data Consumption API — the standardized order form (rehearse: Student 3).....

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

How to Use This Handbook

This handbook prepares all six members of Group 3 for the individual oral defence. Part one contains **seventy-eight questions** grouped into six themes, from overall architecture down to the trickiest follow-ups about revocation and token reuse. Part two contains **six per-student defence packs**, each mapping one student to the modules they own, the numbered features they can claim, a two-minute pitch script, and five personal questions with model answers. The questions are the ones lecturers most commonly ask for this exact brief: a React Native and Expo client, a pure Java backend without Spring, PostgreSQL on Supabase, and the six hard challenges named in the project statement.

Rehearse in three passes. First read your own pack aloud twice, because your two-minute pitch is the only part of the defence you fully control. Second, work through all seventy-eight answers in pairs, one person asking and one answering without looking; anything you cannot answer in thirty seconds goes on a revision card. Third, run the demo script while answering — a demonstration attached to an answer is worth more than the answer alone. Keep the star habit: state the claim, show the mechanism, then name the evidence in the repository (a file, a test, or an edge-case script) that proves it.

Three universal answers are worth memorising verbatim. When asked why no framework: the brief forbids Spring and JUnit, and writing the router, middleware, JSON layer and test runner ourselves turned hidden framework decisions into examinable code. When asked what you are proudest of: the security chain — encrypted at rest, streamed through guarded endpoints, guarded by one atomic SQL UPDATE, audited by append-only triggers — because every claim is demonstrable live. When asked what you would improve: background notification delivery outside the app (push), object storage with server-side encryption for the vault, and moving integration tests into a containerised pipeline.

Theme	Questions	Typical examiner angle
A. Architecture and design	Q1–Q12	Big-picture choices and their justifications
B. Java backend without Spring	Q13–Q24	How the plumbing actually works
C. Authentication and security	Q25–Q38	JWT, hashing, encryption, MFA, rate limiting
D. Database and transactions	Q39–Q50	Schema, constraints, migrations, concurrency
E. Hardest challenges deep dive	Q51–Q62	The six named challenges, edge cases, attacks
F. React Native, MVP, tests, teamwork	Q63–Q78	Client patterns, testing strategy, roadmap defence

Table 1. Question bank map.

Section A — Architecture and Design (Q1–Q12)

Q1. Walk us through the system in one minute.

DocuTrust is a document vault with three parts. A React Native Expo client with 46 screens talks over HTTPS to a pure-Java REST server; the server keeps metadata in PostgreSQL on Supabase and file bytes encrypted with AES-256-GCM in a server-side vault. Everything a user does — upload, share, verify, review — goes through one router, one middleware chain and one set of services, and every security-relevant action is written to an append-only audit trail. The three roles are member, org_manager and admin, all enforced server-side.

Q2. Why a modular monolith instead of microservices on the backend?

The brief says microservices in the frontend, not the backend, and for a six-person, three-week project a modular monolith is the honest engineering choice: one deployment, real transactions across modules, no network calls between services to fail. We kept the benefits microservices give — independent modules, clear boundaries, small readable files — by enforcing a strict Router, Controller, Service, DAO layering with no skipping. If the system grew, each service package already has a single reason to exist, so extracting one into a process later would be a packaging change, not a redesign.

Q3. Why did you write your own JSON parser — is that not risky?

It was a deliberate trade-off forced by the no-heavy-framework brief, and we treat it as a teaching asset rather than a liability. The parser is a recursive-descent implementation of the JSON grammar — objects, arrays, strings with escapes, numbers, booleans and null — small enough to check by reading: round-trips preserve data and malformed input is rejected. In production we would use a hardened library such as Jackson, and we say exactly that in the defence: the point of this project was to demystify what libraries do, not to claim a student parser belongs in a real deployment.

Q4. What is the request lifecycle from tap to response?

A tap triggers a presenter, which calls the ApiClient, which sends the request with the stored Bearer token. On the server the router matches the path, then middleware runs in order: CORS, logging, JWT authentication, role guard, rate limiting. The controller parses the request into plain data and calls a service; the service enforces business rules inside a transaction using DAOs with PreparedStatements; the response goes back as a JSON envelope with ok, data and optional meta. Errors bubble up as typed exceptions that one global handler converts into the error envelope with the right status code.

Q5. Why is ownership checked in SQL rather than in Java code?

Because a WHERE clause cannot be forgotten the way an if-statement can. Every document query is scoped with AND owner_id = ?, so the database itself refuses to return rows the caller does not own, and a request for someone else's document yields a 404 rather than a 403 — the server does not even confirm the row exists. This is defence in depth: even if a future code path skipped a service-level check, the SQL shape still protects the data. The edge-case script proves it with two accounts.

Q6. Explain the three roles and what each can do.

Member is the default: a personal vault with documents, folders, sharing, verification and notifications. Org_manager additionally manages their organisation's members — invites, role changes, removals — through guarded endpoints. Admin works the platform: the review queue for sensitive documents, the user directory with suspension, a read-only platform-wide document view, and the audit log browser. Role checks are one-line guards per route plus audit rows on every refusal, and the admin surface in the app is hidden by role but never trusted — the server is what refuses.

Q7. What is the admin review process and when does it trigger?

Uploads of type medical or financial cannot become active immediately: a schema CHECK forbids a sensitive document to be active unless its review_status is submitted, so the service parks the upload as pending_review and opens an admin_reviews row. An admin works the queue — joined against its target so the card shows full context — and approves, rejects with a note, or escalates; every decision updates the target, notifies the owner and writes an audit row. The same queue also handles suspicious user accounts and flagged verification requests, which is why it is a process, not a screen.

Q8. How does the frontend know what the backend offers?

Through a written contract, docs/API_CONTRACTS.md, that both sides coded against before either was finished: paths, methods, guards, request and response shapes, the envelope, the error codes and the pagination convention. On the client the contract is re-expressed once in endpoints.js, so no screen hardcodes a URL; on the server the route table mirrors it. The /api/meta endpoint also exposes machine-readable metadata — name, tagline, roles, screen and feature counts — that the About screen renders.

Q9. Why PostgreSQL and not MongoDB?

DocuTrust's data is relational and rule-heavy: shares belong to documents, versions belong to documents, and the rules — one target per share, sensitive documents must be reviewed, version numbers unique per document — are exactly what foreign keys, CHECK constraints

and UNIQUE indexes express best. MongoDB would push those invariants into application code, which can have bugs; Postgres enforces them on every statement, including manual ones. We also rely on Postgres features a document store lacks: real transactions with isolation for the atomic share guard, triggers for append-only audit tables, and partial plus trigram indexes for our query patterns. Supabase simply hosts that same engine.

Q10. What is MVP and why did you choose it over MVVM or MVC?

MVP is Model-View-Presenter: the view is a dumb React component, the model is the data layer, and the presenter holds the logic and a small state machine, with the view never touching the model directly. We chose it because presenters are plain classes, so the logic stays reviewable in one place with no React rendering at all. MVVM's two-way bindings are a poor fit with React's one-way data flow, and MVC tends to let fat controllers accrete view knowledge. MVP also gives the whole app one consistent lifecycle — idle, loading, success, error, empty — so every screen behaves identically.

Q11. What are frontend microservices in this project, concretely?

Nine self-contained modules — auth, documents, folders, sharing, verification, notifications, admin, org, profile — each owning its API calls, models and presenters behind hard walls. Screens are dumb views registered in a central SCREEN_REGISTRY that navigation, deep links and notification taps all resolve through, so no module imports another module's internals. The payoff is ownership and replaceability: the sharing module can be rewritten without touching documents, and where each piece of logic lives has a one-directory answer. It mirrors the backend's modular monolith deliberately — same walls, same story.

Q12. How does the SEO metadata requirement fit a mobile app?

Search engines cannot crawl a native app, so the SEO surface is the one public web page the system serves: the verification link. When a browser asks for a verification token, the Java service renders a lightweight HTML page with title and meta description, Open Graph tags for social previews and JSON-LD structured data for the verification event — content-negotiated from the same endpoint that returns JSON to the app. The Expo config additionally carries the app's name, description and deep-link scheme, and /api/meta exposes structured metadata for integrations. So the requirement is met where SEO is actually meaningful.

Section B — Java Backend Without Spring (Q13–Q24)

Q13. How does your server work without Spring?

It is a plain Java program using com.sun.net.httpserver.HttpServer, the JDK's built-in HTTP engine — the same one application servers use underneath. Main.java creates the server on port 8080, registers 74 route paths on a hand-written router, applies the middleware chain, runs migrations, seeds demo data and starts a scheduler. Spring's job — dependency wiring, routing, filters, JSON — is done by about two thousand lines of our own small, explicit code. Every mechanism the framework would have hidden is now a file we can open and defend.

Q14. Explain your router. How does /api/documents/:id match?

Routes are registered as method plus pattern plus handler. At startup the router splits each pattern into segments and stores them; at request time it splits the incoming path the same way and compares segment by segment — literal segments must be equal, colon-prefixed segments match anything and capture their value into a map. Exact static segments win over parameters when both match, and method mismatch yields 404/405 handling. Query strings are parsed into a map on the same context object, so a handler reads ctx.pathParam(id) and ctx.query(search) without touching raw exchange internals.

Q15. What is middleware in your design, and what does each piece do?

Middleware is a wrapper applied around a handler before any business code runs. Our chain is CORS (answer preflights, stamp allowed origins), logging (one line per request with status and duration), authentication (verify the JWT, load claims, produce precise 401s), role guard (compare the role claim, refuse with 403 and audit), and rate limiting (sliding window per IP for auth and public endpoints). Handlers are composed functionally — requireRole(admin, authenticate(h)) — so the chain reads like the security policy it enforces. Because it is our own code, we can show exactly where each check happens and in what order.

Q16. How do you handle multipart file uploads without libraries?

The context object reads the raw body bytes, extracts the boundary from the Content-Type header and splits the parts, parsing each part's headers for name and filename and returning a map of fields plus the file part's bytes and metadata. Upload size is capped before buffering to prevent memory abuse, and the allowed mime types are checked immediately. The file then goes straight into streaming SHA-256 computation and AES-GCM encryption, so plaintext never lingers. The hand-built multipart body in the defence demo exercises every branch of it, which doubles as proof the parser is correct.

Q17. How does your connection pool work?

It is a small thread-safe pool backed by a blocking queue: on start it opens a configurable number of PostgreSQL connections and validates them; getConnection() borrows one or blocks briefly, and close() returns it rather than closing the socket. Stale connections are detected by a lightweight validity check and replaced on borrow. DAOs always use try-with-

resources so a connection is returned even when a statement throws, and transactions borrow one connection and keep it for the whole unit of work. In production this would be HikariCP; the point here was to show we understand what a pool does and why N connections bound throughput.

Q18. Why PreparedStatements, and what do they prevent?

PreparedStatements send SQL structure and data separately: parameters travel as typed values, never spliced into the SQL text, which makes SQL injection structurally impossible in our DAO layer — there is no string concatenation of user input anywhere. They also let the driver cache execution plans, and they document intent: every query's shape is visible at the top of the DAO method. The defence demonstration is simple: search for a title containing a quote and an OR 1=1 attempt; the trigram search returns rows whose title literally contains that text, and nothing else happens.

Q19. What does your migration runner do?

On boot, before the server accepts traffic, the runner reads the numbered V01–V11 SQL files, compares them against a schema_migrations table recording what has already been applied, and executes the new ones in order inside a transaction each. Applied files are never re-run, so restarts are idempotent, and a failure mid-file rolls the whole file back rather than leaving half a schema. The team applies the same files by hand in Supabase's SQL editor when demonstrating, which shows the runner is a convenience, not a lock-in. This is exactly how Flyway works, minus the dependency.

Q20. How are errors handled across the whole server?

Business code throws a single typed AppException carrying an HTTP status, a machine-readable code and an optional field. One global boundary catches exceptions, converts known types into the error envelope { ok:false, error:{code,message,field} } and unknowns into a 500 INTERNAL_ERROR with the details logged server-side and an audit row — never leaked to the client. Middleware-generated failures such as 401, 403 and 429 use the same envelope, so the client's error handling has exactly one shape to parse. The client maps codes to behaviour: TOKEN_EXPIRED refreshes and retries, TOKEN_REUSE logs out, VALIDATION_ERROR shows field errors.

Q21. Is your server multi-threaded? What does that imply?

Yes — HttpServer dispatches each exchange on a worker thread pool, so handler code must be thread-safe. We keep handlers stateless: anything mutable lives either in the database, in the per-request context object, or in deliberately concurrent structures such as the rate limiter's window and the download-ticket map, which use concurrent collections and atomic operations. JDBC connections are never shared between threads — each request borrows its

own from the pool — and transactions never escape the thread that opened them. The scheduler runs on its own single-threaded executor so its database work cannot interleave badly with request work.

Q22. How would you scale this server?

Three honest stages. Vertically first: the server is stateless apart from the vault and the in-memory ticket map, so bigger machines and a tuned pool go a long way. Second, move the two pieces of in-memory state out — tickets into a short-TTL table or Redis, the rate limiter into a shared store — and then multiple instances can sit behind a load balancer with the vault on object storage. Third, split by load: the document download path and the API path scale differently, so extracting a dedicated file-streaming service with its own pool is the natural first microservice. The database read replicas and pg_trgm indexes cover the read-heavy search growth.

Q23. What is in your pom.xml and why?

The pom declares Java release 21 for the compiler plugin, eight pinned runtime dependencies — the PostgreSQL JDBC driver, HikariCP, Bucket4j, Caffeine, Dropwizard Metrics, Sentry and the two SLF4J pieces — and the exec and jar plugins so the project runs both as mvn exec:java and as a self-contained java -jar jar. There is deliberately no test-framework dependency: the pom's own comment says so outright, and the build's job ends at the runnable jar — correctness is reviewed in the code and demonstrated live at the defence.

Q24. What happens on first boot with an empty database?

Migrations run first: V01 through V11 create extensions, tables, constraints, indexes, triggers and views, recorded in schema_migrations. Then the Bootstrap seeder checks whether the users table is empty and, if seeding is enabled, creates the demo organisation, six accounts with proper PBKDF2 hashes, folders, five documents in different lifecycle states, an active share and an approved verification request — pushing a generated PDF through the real encryption path so demo bytes are representative. Finally the scheduler starts its sixty-second tick and the health endpoint begins reporting database true. The console logs each stage, so bring-up is observable.

Section C — Authentication and Security (Q25–Q38)

Q25. What exactly is inside your JWT?

Three Base64-URL segments joined by dots. The header names the algorithm HS256 and type JWT. The payload carries the subject (the user's id), the role claim, the organisation id, the issued-at time, a fifteen-minute expiry and a unique token id. The third segment is an

HMAC-SHA256 signature over header-plus-payload made with a server secret loaded from the environment. We build and verify it with javax.crypto.Mac and compare signatures in constant time, so signature checking does not leak timing information.

Q26. Why does the access token live only fifteen minutes?

A short access token bounds the damage of a leak: a stolen bearer token is useful for at most fifteen minutes, and it cannot be revoked faster than its natural death because it is stateless. The refresh token is the long-lived, revocable half of the pair, and it only ever travels to one endpoint, so the window in which it can be intercepted is small. This asymmetry — short and stateless, long and stateful — is the standard trade-off, and the client hides it completely behind the silent refresh, so users never see a login prompt.

Q27. Walk through refresh rotation step by step.

The client posts its refresh token to /api/auth/refresh. The server hashes the presented token with SHA-256 and looks the hash up in refresh_tokens; the row must exist, be unrevoked and unexpired. If valid, the server revokes that row, records the id of its replacement in replaced_by, inserts a new row with a fresh random token, and returns a new access plus refresh pair. If the presented token's hash matches a row that is already revoked — meaning it was rotated before — the server answers 409 TOKEN_REUSE and revokes every token belonging to that user, ending the whole family.

Q28. Why is reuse detection important? What attack does it stop?

It stops the classic stolen-refresh-token attack. Suppose an attacker copies a refresh token; the real client refreshes first and the family rotates. When the attacker later replays the stolen token, the server sees an already-rotated token — something only a copy could present, because the legitimate client replaced its own — and knows the family is compromised. Revoking the whole family forces the thief to re-authenticate and alerts the real user, who must log in again. Without reuse detection, rotation is mere theatre; with it, rotation becomes an intrusion detector.

Q29. Why PBKDF2 and not bcrypt or Argon2? Why 120,000 iterations?

PBKDF2 is specified in the JDK standard library (PBKDF2WithHmacSHA256), and the brief restricts us to dependencies the JDK provides, so it is the correct choice inside those constraints; bcrypt and Argon2 are both excellent but require libraries we were not permitted. We use 120,000 iterations with a per-user sixteen-byte random salt and a 256-bit derived key, stored in the self-describing format pbkdf2$iter$salt$hash, which lets the iteration count be raised later without breaking old hashes. Verification is constant-time to avoid timing leaks, and the self-describing format keeps every stored hash independently upgradable.

Q30. Explain AES-256-GCM and why you chose that mode.

GCM is an authenticated encryption mode: it encrypts with AES using a 256-bit key and a fresh 12-byte initialisation vector per file, and it appends an authentication tag computed over the ciphertext. The tag means any later modification of the stored bytes — corruption or a deliberate tamper — is detected at decrypt time, which ordinary modes like CBC would not notice. Fresh IVs per encryption are mandatory in GCM; reuse would be catastrophic, so the vault generates one with SecureRandom for every write. The key comes from the environment, never from source control, and the tag itself proves tampering: any altered stored byte causes decryption to fail.

Q31. Where is the encryption key kept, and what happens if it is lost?

The master key is loaded from the VAULT_KEY environment variable as Base64; if it is absent in development the server generates an ephemeral key and prints a loud warning that previously stored files will not decrypt after a restart. Losing the master key in production means the vault is unrecoverable by design — that is the trade encryption makes — so operations would rotate to a backup key source such as a managed KMS. In the demo we show the warning, which itself is evidence the team thought about key lifecycle.

Q32. How is TOTP multi-factor authentication implemented?

RFC 6238, by hand. Setup generates a twenty-byte secret, Base32-encodes it, stores it on the user row and returns an otpauth:// URL that any authenticator app can scan. The server computes the six-digit code from the shared secret and the current thirty-second time step, comparing with a plus-or-minus one step window to tolerate clock drift, in constant time. Login for an MFA-enabled user returns a distinct 440 MFA_REQUIRED status until a valid code accompanies the credentials, and the app routes to a dedicated six-box code screen. The post-audit overhaul added the email fallback: a user without their authenticator device re-submits the login with useEmailMfa set, and the server answers 440 MFA_EMAIL_CHALLENGE with a ten-minute challenge token and emails a six-digit code; POST /api/auth/verify-otp exchanges challenge plus code for the real session tokens. Verification accepts the code at the current and the adjacent time step and rejects everything else.

Q33. How does rate limiting work and where is it applied?

Token buckets from Bucket4j, one per scope-and-IP pair and held in a Caffeine cache so idle entries are evicted. Budgets are per scope: twenty authentication attempts per minute, thirty for the public share endpoints, twenty for verification lookups, ten for everything else. A bucket that runs dry answers 429 RATE_LIMITED, and because a token bucket refills continuously it also smooths bursts instead of allowing a whole window's worth of requests

in one stampede. It is applied in middleware, so it protects the whole route before any handler runs. In a multi-instance deployment this state would move to a shared store such as Redis, which we state openly.

Q34. What does your server log, and what does it never log?

Security-relevant events land in audit_logs: logins and failures, downloads, share opens, revocations, admin decisions, refused privilege escalations and internal errors — each with actor, entity, IP, user agent and a JSON detail payload. Request logging middleware writes one operational line per request with method, path, status and duration. We never log passwords, raw tokens, vault contents or reset codes; tokens appear only as hashes, and error responses to clients never include stack traces or SQL. The append-only trigger on audit_logs means even the logs cannot be quietly rewritten.

Q35. How do you prevent SQL injection, XSS and CSRF?

SQL injection: every query is a PreparedStatement with bound parameters; no user input is ever concatenated into SQL, and the search demo shows an injection attempt returning harmless literal matches. XSS: the API returns JSON, never HTML built from user input, and the one HTML page we render — the public verification page — is a fixed template with server-computed values and no user-supplied markup; React also escapes by default. CSRF: the API is not cookie-authenticated — the client sends an explicit Bearer header that a cross-site form cannot forge — and the state-changing public endpoints are token-parameterised, which is the standard bearer-token defence.

Q36. How are passwords reset without leaking who has an account?

Forgot-password always answers 200 regardless of whether the email exists, so responses cannot be used to enumerate accounts. For real users the server writes a hashed, single-use code with a fifteen-minute expiry to password_resets and the app's reset flow consumes it: correct code plus a new password, used once, then dead. Codes are hashed at rest like passwords, so a database leak does not reveal usable reset codes, and the single-use and expiry rules are enforced inside the verification path itself.

Q37. What happens when an admin suspends an account mid-session?

Suspension is a status on the user row, and the authentication middleware loads the current status with the token claims on every request, so a suspended user's very next call — even one with a perfectly valid, unexpired JWT — is refused with a 403. The refresh endpoint also refuses suspended accounts, so waiting for the access token to expire does not help. There is a guard preventing an admin from suspending themselves, and the whole action is written to the audit log with both actor and target.

Q38. Which OWASP-style risks did you consciously accept, and why?

Three, stated honestly. The vault key lives in an environment variable rather than a managed KMS — acceptable for the coursework scope, with KMS named as the production answer. Rate limiting state is in-memory, which is correct for one instance and documented as needing Redis when horizontally scaled. The third was email delivery being stubbed in-app — and that one is now fixed: the OTP overhaul wired a real transactional email provider (Brevo) behind a MAIL_MODE switch, with a dev mode that echoes codes in an X-Dev-OTP header for demos and a live mode that sends real email. The remaining honest gaps are the two above, plus push notifications being absent (the in-app inbox is the transport). Naming accepted risks with their mitigations is part of a good defence.

Section D — Database and Transactions (Q39–Q50)

Q39. Describe the schema in thirty seconds.

Thirteen tables in four groups. Identity: organisations, users, refresh_tokens, password_resets, settings. Vault core: folders, documents, document_versions, document_history. Sharing and proofs: shares, verification_requests. Platform: notifications, admin_reviews, audit_logs. Documents carry a six-state lifecycle and an expiry date; versions pin SHA-256 digests; history and audit tables are append-only by trigger; every child table cascades from its parent except where the business says otherwise, like folders detaching documents instead of destroying them.

Q40. Which constraints carry business rules, not just integrity?

Three stand out. A CHECK on shares requires a user target or a link token — a share must be shareable somehow. A CHECK on documents forbids a medical or financial document to be active unless its review_status is submitted, which hard-wires the administrative review process into the engine. And a UNIQUE on (document_id, version_number) turns a concurrent-upload race into a clean conflict instead of corrupt state. Roles, statuses, document types, permissions and reminder windows are all CHECK-enumerated too, so even a bug cannot write an impossible value.

Q41. What is a transaction, and where do you use them?

A transaction groups statements so they all commit together or all roll back — the database never observes half of one. Every multi-step write in DocuTrust runs in one: upload inserts the document, version one and history (and possibly a review row) atomically and deletes the staged file if anything fails; permanent delete removes rows and encrypted bytes together; a share open is a single guarded UPDATE whose row-count tells the whole story. The Tx

helper sets autocommit false, commits on success and rolls back on any exception, so services declare intent and never manage connection plumbing.

Q42. Two people open the last view of a share link at the same second. What happens?

Exactly one of them gets in. The guard is a single conditional UPDATE — increment view_count WHERE the share is unrevoked, unexpired and under its max_views — and Postgres row locks make those two updates serial, so the second one matches zero rows. The service then re-reads the row to answer precisely why the link is dead: expired, revoked or exhausted, returning 410 or 409 accordingly. No count can overshoot because the increment and the guard are the same statement.

Q43. Two uploads race to create version N+1. What happens?

Both read the current max, both compute N+1, but the UNIQUE constraint on (document_id, version_number) lets only the first insert commit. The loser receives a constraint violation, which DocumentService catches and rethrows as a 409 CONFLICT with a clear message, inside a rolled-back transaction — no orphan file, no phantom version. The client surfaces it as a friendly retry prompt. This is the canonical use of a database constraint as a concurrency primitive, and the edge-case script fires it from two terminals.

Q44. Explain the append-only triggers. Why not just enforce it in code?

BEFORE UPDATE OR DELETE triggers on document_history and audit_logs raise an exception unconditionally, so the tables physically cannot be rewritten — by the app, by a migration, or by an administrator with a SQL console. Code-level enforcement would be one forgotten WHERE clause away from silent history edits, and audit tables are exactly where you must assume someone will eventually try. The defence demo types an UPDATE and shows the database throwing append-only exceptions live. The same philosophy puts automatic history-writing into AFTER INSERT/UPDATE triggers on documents, so no code path can skip the trail.

Q45. How do your indexes match your queries?

Index choice follows the hot paths. Owner-scoped lists use a partial index on documents(owner_id) filtered to alive rows, because deleted documents are queried rarely but listed constantly. Search uses a GIN trigram index on titles so ILIKE patterns stay indexed instead of scanning. Token lookups — which happen on every public share hit — use a partial index on link_token filtered to unrevoked shares. History and audit are queried newest-first per entity, so they carry composite indexes on (entity, created_at DESC), and the notifications inbox indexes (user, read_at, created_at) to answer unread-first queries.

Q46. What are the two views for, and why SQL views?

v_expiring_documents computes the expiry radar — days left per document inside its reminder window — and v_admin_stats aggregates the dashboard counters in one select. Putting them in SQL means one tested definition the server queries directly, instead of the same date arithmetic scattered through Java or JavaScript where two implementations could drift. Views also express intent: the dashboard's meaning lives next to the data it summarises. They are ordinary views, so they are always current, which matters because the expiry scheduler rewrites statuses every sixty seconds.

Q47. Why is documents.sha256 duplicated from document_versions?

It is a deliberate denormalisation kept honest by a trigger: an AFTER INSERT trigger on document_versions copies the new version's digest, size and mime type onto the parent document row and bumps current_version. The hottest integrity check — comparing stored versus recomputed digest on download — then reads one row instead of joining, and list screens show integrity state without a join. Because the trigger maintains it, the mirror cannot drift: there is no code path that writes a version without updating its document's head.

Q48. How does soft delete work, and what is the danger with it?

Soft delete sets status to deleted and stamps deleted_at, so trash can restore a document and history stays meaningful. Every hot query filters deleted_at IS NULL — which is what the partial owner index is built around — and permanent delete from the trash screen is the only path that removes rows and encrypted bytes. The classic soft-delete danger is forgetting the filter somewhere and leaking deleted rows into lists, shares or search; our answer is that all list queries funnel through one DAO method family that applies the filter, and the share guard also refuses shares pointing at deleted documents server-side.

Q49. How does the scheduler update statuses without spamming users?

Every sixty seconds the ExpiryScheduler marks active documents whose expiry falls within their reminder window as expiring, and expiring documents past expiry as expired, then raises notifications. The de-duplication rule is one notification per document per type per day: before inserting, it checks whether today's notification already exists for that entity and type, so a document sitting in the expiring state for a fortnight produces fourteen reminders at most, not thousands. Expired verification requests are retired in the same tick. It runs on its own executor thread with its own connection so it never contends with request transactions.

Q50. Why store files on disk instead of in the database or on Supabase Storage?

Databases are poor filesystems: large bytea rows bloat backups, slow dumps and complicate streaming, while DocuTrust needs range streaming, checksum-verified reads and cheap

deletes. We chose a server-side encrypted directory as the storage engine behind a small FileVault interface, which keeps the demo fully self-contained and lets us demonstrate tamper detection physically. The interface is one seam: swapping in Supabase Storage or S3 with server-side encryption means implementing the same put, open and delete contract, and nothing above the seam changes — we say this in the defence as the honest production answer.

Section E — The Hardest Challenges, Deep Dive (Q51–Q62)

Q51. You claim private file delivery. Prove there is no way to reach a file without authorisation.

Walk the surface with us. There is no static file server and no public bucket: the only paths to bytes are the authenticated download endpoints and the public ticket endpoint. Authenticated downloads pass the middleware chain and an ownership WHERE clause. Public downloads require a ticket that exists only after the atomic share guard passed, is bound to one share, is single-use and dies in five minutes. The bytes themselves are AES-256-GCM ciphertext whose key never leaves the server, so even a hypothetical path leak yields an unreadable file. Every one of those statements is demonstrated in the edge-case script.

Q52. Why is a 403 for someone else's document wrong and a 404 right?

A 403 confirms the resource exists and the caller lacks permission — information itself, useful for enumerating other users' document ids. Because our queries are scoped with owner_id in the WHERE clause, a foreign document simply does not match, and the honest answer is 404 NOT_FOUND, indistinguishable from a typo'd id. This is a standard IDOR defence: do not confirm what you do not have to. The role-escalation demo shows both behaviours side by side — 403 for wrong role on a public route shape, 404 for wrong owner on a resource.

Q53. What stops a revoked share from still working for a few seconds?

Nothing survives, because revocation is checked at request time, not propagated. The revoke endpoint stamps revoked_at on the row, and every subsequent guard UPDATE includes revoked_at IS NULL in its WHERE clause, so the very next open anywhere fails with 409 SHARE_REVOKED. There is no cache of link validity to invalidate, no signed URL still floating around with a clock on it — the link is not a capability, it is a lookup whose validity is recomputed on every use. That design choice is why revocation is instant by construction.

Q54. A share link expires while someone is mid-download. What happens?

Two distinct steps have two distinct guarantees. Metadata and open are guarded: the open call's atomic UPDATE re-checks expiry, so an expired link never yields a ticket. An already-issued download ticket is itself short-lived — five minutes — and single-use, and redemption re-reads the share row's state, so a link revoked after ticket issuance still refuses at redemption. A download already streaming is bounded by the ticket's five-minute worst case, which is the accepted, documented window; tightening it further would trade user experience for a marginal risk.

Q55. How exactly does tamper detection work end to end?

Each version row pins the SHA-256 of its encrypted bytes at upload. On every download the server streams the file through decryption while recomputing the digest and compares the two; a mismatch aborts the response and writes an integrity audit row. The verification feature surfaces the same check to the world: a proof link recomputes the current version's digest and reports tampered instead of valid when it disagrees. The demo overwrites two bytes inside a vault file and shows the verification result flip — evidence that integrity is continuously checkable, not just promised at upload.

Q56. Could two version uploads interleave badly with the download path?

Downloads are safe by snapshot semantics: a download resolves the version row it will stream first, then reads that storage key, so a concurrently committed newer version simply becomes the next download's head. The UNIQUE (document_id, version_number) constraint serialises upload racing as described earlier, and the document row's mirrored head fields are maintained by a trigger rather than application writes, so no interleaving produces an inconsistent triple of current_version, sha256 and storage key. Worst case for a downloader is a slightly older version than the newest one, which is correct behaviour for a snapshot.

Q57. What is your token strategy on the device, and what if the device is stolen?

Tokens live in AsyncStorage on the device and travel only over HTTPS in an Authorization header. A stolen device yields at most the fifteen-minute access token and the refresh token; the thief cannot read source or storage without a locked-phone barrier, and any refresh attempt by the real owner's replacement device after logout revokes the family. Server-side, logout and password change revoke refresh tokens, and suspension kills sessions at the token level on the next call. Refresh-token hashing at rest means even a full database leak does not expose usable refresh tokens.

Q58. How do you handle clock skew in expiry and TOTP?

All server-side expiry comparisons use the database clock — now() in SQL — so link and token expiry are immune to client clocks entirely; clients never assert validity, they only

receive it. JWT verification compares against the server clock, which is the only party that matters. TOTP tolerates the one place user clocks appear: the six-digit code check accepts the current step plus or minus one, the RFC's recommended drift window, and enabling MFA requires a confirmed code so a badly wrong clock is caught at setup rather than at login.

Q59. Why is the verification result its own state machine, not just a boolean?

Because the interesting answer is rarely yes or no. The result distinguishes valid, tampered, expired, revoked and invalid — each produced by a different branch: approved request and matching digest means valid; digest mismatch means tampered; past expiry means expired; deleted vault item means revoked or invalid depending on how the document died. Downstream consumers — the public page, the owner's list, the admin queue — need to show different text and colours per state, and the state machine makes each case a tested unit-test row rather than a conditional in a view.

Q60. What is the most likely production incident, and what does the audit trail give you?

The realistic one is a user reporting an unexpected document change or share. Because document_history is trigger-fed and append-only, support can answer exactly who did what and when with actor, IP and detail payload; because admin actions and refusals land in audit_logs, misuse of authority is visible too. The append-only guarantee means the incident record cannot be edited afterwards, which changes the conversation from whose memory is right to what the row says. We consider that property — tamper-evident operational history — the system's most quietly valuable feature.

Q61. If we asked you to add file previews, what would change?

Only behind the vault seam. Previews are derived plaintext artefacts, so the safe design is a server-side thumbnailer that reads the decrypted stream in-memory, renders a small image, and treats it as its own asset with its own guarded endpoint and no permanent plaintext on disk. The document bytes, their endpoints and the client model stay untouched; the client adds an image tile on the detail screen. We would explicitly forbid generating previews on the client from downloaded originals, because that would move plaintext to devices, which contradicts the delivery story.

Q62. What would you build next with one more week?

Three things, in order. Push notifications, because the inbox plus scheduler already produce the right events and a push transport is additive. Object storage with server-side encryption behind the existing FileVault interface, removing our own disk as an operational concern. And a small performance pass on the admin audit browser — cursor pagination — because OFFSET pagination is the one place where large offsets will eventually hurt. Each is scoped,

each has a named seam, and none would require touching the security core, which is the point of the layering.

Section F — React Native, MVP, Testing and Teamwork (Q63–Q78)

Q63. What does Expo give you, and what does it cost?

We run Expo SDK 58 with React Native 0.88 — the newest SDK generation, bringing the new React Native architecture and iOS 27 support; npx expo-doctor inspects the project like a mechanic and reports 20/20 checks clean. Expo gives a managed toolchain: no Xcode or Android Studio needed to run the app, over-the-air project configuration via app.json, first-class document picking and file system modules, and QR-code preview on real devices — decisive for a student team demoing on borrowed phones. The cost is that exotic native modules require ejecting to a bare workflow, and some native capabilities arrive late to the managed SDK. DocuTrust needs nothing exotic — navigation, storage, document picker, file system are all supported — so the trade was clearly in Expo's favour.

Q64. How is a screen wired together in your MVP? Show the pieces.

Each screen is three roles. The view is a function component that receives state and callbacks from one hook call, usePresenter, which constructs the screen's presenter with its model API and subscribes to its state machine. The presenter is a plain class extending BasePresenter — attach, detach, and named intents like submit() or loadMore() — holding idle, loading, success, error and empty states. The model is the microservice's api.js calling the shared ApiClient. On unmount the hook detaches, so no presenter ever updates an unmounted view.

Q65. Why keep presenters as plain classes instead of hooks inside components?

Because plain classes are trivially unit-testable: instantiate with a mocked API, call methods, assert state transitions — no React renderer, no act() dances, no component tree. That property is what keeps login, pagination and error logic reviewable as plain classes instead of buried inside components. It also enforces the discipline that views stay dumb; when logic tries to hide in a component, it shows up as inconsistent loading states across screens, which MVP's shared BasePresenter makes structurally difficult.

Q66. How does the ApiClient's silent refresh work under concurrency?

When any call receives 401 TOKEN_EXPIRED, the client starts a single refresh flight — one module-level promise that all concurrent failures await, so five expiring calls trigger one refresh, not five. The flight posts the refresh token, stores the new pair, resolves, and every waiter retries its original call exactly once; a failed flight clears the session on

TOKEN_REUSE or UNAUTHORIZED. The design guarantees precisely this: one refresh network call for N parallel expiring requests, and a retry-once guarantee that prevents loops.

Q67. How do role-based screens work without trusting the client?

The auth store keeps the user object with its role from login and /me, and the navigator mounts the admin stack only for admins and the org screens for managers — that is user experience, not security. The server applies requireRole guards regardless of what the client shows, and every refusal is audited, which the edge-case demo proves by calling an admin endpoint with a member token. So the client hides surfaces for comfort; the server decides authority for truth. Both halves are stated separately in the defence because conflating them is the classic RBAC mistake.

Q68. How do notifications deep-link into screens?

Notification rows carry an entity_type and entity_id, and the client maps the pair through the SCREEN_REGISTRY to a target — an expiry warning opens the document, a share notification opens the share's detail, an admin decision opens the review detail. Tapping marks the row read and navigates. The same registry backs the app's docutrust:// URL scheme for external links such as verification tokens, so one indirection table serves in-app taps, push payloads and web deep links alike.

Q69. What design decisions make the UI feel hand-made rather than generated?

A deliberate non-default design system: an ink-and-parchment palette with a single amber accent instead of the generic blue-purple, serif display headings paired with a quiet sans body, and a letterspaced eyebrow style for labels. Signature components carry personality — rotated dashed stamp badges for statuses, a vertical timeline for history, file-tab cards with a type-coloured edge — and every screen shares one Screen scaffold so the quirk reads as identity, not noise. Empty states are drawn with plain Views instead of stock illustrations, and spacing follows a fixed four-point scale. The goal was a notary's ledger, not a dashboard template.

Q70. The brief forbids JUnit on the server — what does the build promise instead?

A build that does one thing honestly: mvn clean package resolves the eight pinned libraries and packs the runnable jar, and the pom's own comment declares that there is deliberately no test framework — nothing hidden, nothing magic. Correctness is carried by the code itself: small, readable classes whose comments carry the audit's finding IDs, and by live demonstration — every flow can be driven with manual API calls (Postman/cURL) against the booted server during the defence. Fewer moving parts also keeps the request path examinable: a handful of files between the socket and our code.

Q71. What concrete behaviour can you show live in two minutes?

Server-side: replay a rotated refresh token and watch the 409 TOKEN_REUSE plus the family revocation; boot with a weak JWT secret and watch the fail-fast refusal print the remedy; flip two vault bytes and watch a verification verdict turn to tampered. Client-side: the login screen routes the 440 MFA challenge correctly, and the ApiClient refreshes once for N parallel 401s — visible in the log as a single refresh call. Each demo maps to a finding ID or a feature ID in the register.

Q72. How do you verify a change before it ships?

Three gates, in order. Read: the change is reviewed against the module's ownership — one student per module keeps the context. Build and boot: mvn clean package must succeed from a clean clone and the boot log must print the same proof line, migrations=12 and routes=74. Behave: the affected flows are driven by hand — manual API calls for server changes, the emulator for client changes — including the negative paths, where the honest refusals are the proof. Small, boring gates — and the same gates an examiner can run in the room.

Q73. How did the six of you divide the work, and why that shape?

Vertical slices: each student owned one domain end to end — server service, screens, tests — so nobody was stuck with only plumbing or only screens and every owner could defend their whole slice. One member acted as architect, owning the router, middleware, JWT and project skeleton that everyone built on; the remainder split auth, documents and vault, sharing and public delivery, database with verification and notifications, and admin with org plus the MVP infrastructure. Pair rotation in week one spread the architecture knowledge, and the feature register's per-module ownership table is the written record of who owns what.

Q74. How did you keep the API contract from drifting during three weeks?

The contract was written first as docs/API_CONTRACTS.md with paths, shapes, envelopes and error codes, and both sides coded against it before endpoints existed; the client developed against the documented shapes using the seeded expectations. Changes were not forbidden but were made by editing the contract first, in a group decision, then both sides converging — the register of 216 features cross-references the same module names, which made drift visible in review. The defence demo is the enforcer: it walks the contract end to end against the live server, so a silent shape change shows up in the room, not in production.

Q75. What was the hardest technical disagreement, and how was it resolved?

Whether public download links should be signed URLs over the vault or short-lived tickets redeemed through guarded endpoints. Signed URLs are fewer moving parts, but they cannot be revoked instantly — the signature stays valid until it expires — which contradicts our

revocation guarantee. The team chose tickets: an in-memory single-use grant valid five minutes, redeemed through the same guard path as opens. The disagreement is documented in the roadmap's decision log precisely because the losing option was reasonable, and being able to explain why the safer one won is a better defence outcome than unanimity.

Q76. How does the three-week roadmap defend itself against the criticism that it is optimistic?

It is decomposed to the level where optimism can be checked: twelve named migrations, 74 contract paths, 46 registered screens and 234 numbered features each map to a week and an owner, and the two Friday demos are binary gates — a register-login-upload path in week one, the full journey in week two. The risk register names the four realistic failures — Supabase latency in class, multipart bugs discovered late, screen creep, and uneven knowledge — with mitigations already scheduled, like the day-three upload spike and the frozen screen registry. Optimism is cheap; a plan with numbered, testable artefacts is falsifiable, which is what a roadmap should be.

Q77. What is the single weakest part of the system, honestly?

Notification transport. The events, preferences, de-duplication and inbox are real, but delivery is in-app only; there is no push or email, so a user who does not open the app learns about an expiring passport late. It is a product-completeness gap, not an architectural one — the scheduler already emits exactly the payloads a push service would forward — and we name it before the examiner does. Runner-up weaknesses and their production answers are the in-memory ticket map and rate limiter, both needing a shared store when scaled horizontally.

Q78. In one sentence, why should this system be trusted with real documents?

Because every promise it makes is enforced by a mechanism an examiner can watch fail: encryption whose tampering is detectable, delivery whose every path passes a guard, links whose death is checked atomically on use, history that cannot be rewritten, and an audit trail that records the attempts — trust here is not a claim but a set of testable behaviours.

Q79. Your OTP overhaul replaced codes-in-responses with emailed codes. Walk me through the security properties.

Five properties, each checkable. First, at rest: the email_otps table (migration V10) stores only a SHA-256 digest over the purpose, the recipient address and the code itself, so a leaked database row is worthless and cannot be replayed across purposes. Second, in transit: the code exists only in the recipient's inbox — API responses never carry it; the one exception is the X-Dev-OTP header in MAIL_MODE=dev so demos work without mail traffic. Third, in time: ten minutes to live, twenty-four for invites, single-use, and burned after five wrong

attempts inside one locked transaction so two parallel guesses cannot both spend it. Fourth, in scope: step-up verification issues a separate five-minute, single-purpose JWT (typ=step_up) that destructive endpoints demand in the X-Step-Up-Token header — without it they answer 403 STEP_UP_REQUIRED, and a login challenge token is a different type and is refused there. Fifth, in effect: the old reset-code-in-body and temporary-password-in-body patterns were deleted outright, and every flow, including the negative paths, is walked live with manual API calls.

Per-Student Defence Packs

Each pack maps one member to one of the six defence roles — Infrastructure & Database, Auth & OTP Security, Business Logic, Mobile App, the 8-Pass Security Audit, and Deployment & DevOps — the same roles, pitches and analogies as *DEFENCE_GUIDE.md* at the repository root, so the team tells one story. The two-minute pitch is rehearsed, not improvised: deliver it while performing the matching demo action, because a spoken claim plus a live behaviour is the strongest unit of defence. The five personal questions per pack are the ones examiners most often direct at the person who owns that slice; rehearse them aloud until the answers come in three sentences or fewer.

Student 1 — Infrastructure & Database

Role in the team: The skeleton and the truth layer: the pure-Java server platform, the connection pool, and the PostgreSQL schema whose invariants live in the engine.

Owns: Router, middleware chain, JWT service, error envelope, config/boot, migration runner, HikariCP pool, and schema V01 to V11 (15 tables); the Dashboard screen and app shell navigation.

Feature claims: FT-193 to FT-202 and FT-206 to FT-208 (server platform), plus the Dashboard and navigation registry features FT-173, FT-177, FT-178.

Two-minute pitch. *Say this, then demo:*

I own the skeleton everything else stands on, and the truth layer it stands on. The skeleton: a pure-Java HTTP server with a hand-written router mapping our 74 API paths, a middleware chain where every security check happens before business code runs, and a JWT service built on the JDK's HMAC — when we were told no Spring, that was not a limitation for me, it was the job. The truth layer: eleven migrations building fifteen tables whose invariants live in the engine — CHECK constraints that make a sensitive document impossible to activate without review, UNIQUE constraints that turn upload races into clean conflicts, and triggers that write history automatically and make the audit tables physically append-only. Beneath both sits HikariCP, the connection pool that seats every query instantly. My demo: one request

crossing the whole chain live — logged, authenticated, role-checked, rate-limited — and then a manual UPDATE against the history table that the database itself throws out.

Likely personal questions and model answers:

QP1.1. Why should one student own the middleware, rather than sharing it?

Because the middleware chain is where cross-module policy lives, and split ownership would have meant five people editing one file with five opinions. Owning it let me keep the chain small and consistent while module owners focused on services; I still paired with each of them when their endpoints needed a new guard.

QP1.2. Your router matches static segments over parameters. Why does that ordering matter?

If /api/documents/expiring were captured by /api/documents/:id, the word expiring would be treated as an id and the route would 404 or misparse. Matching static segments first, parameters second, is what lets both shapes coexist. The staticSegs scoring in Router.match is the whole rule — one line you can read in the file.

QP1.3. How do you know your JWT implementation is correct?

Three ways. The code is small enough to read aloud: sign, verify, check expiry — each a named method in JwtService. The middleware treats expired and invalid differently, which the client depends on for refresh. And the live demo runs real login-to-refresh flows over HTTP, so the hand-rolled token is exercised against the running server, not just in isolation.

QP1.4. If I delete Spring from the equation, what did you actually gain educationally?

The ability to answer how questions that frameworks answer with because it does. How routing matches, when middleware runs, what a 401 versus 403 means, why a connection pool bounds concurrency — each is a file we wrote. That is the difference between using a framework and understanding one.

QP1.5. Why put the sensitive-document rule in a CHECK constraint instead of the service?

Because the service can be bypassed — a migration, a manual SQL statement, a future bug — but the constraint is enforced on every statement by the engine. The rule is one line: a medical or financial document cannot be active unless its review status is submitted. Encoding it in the schema means the review process has no silent back door, which is exactly what an auditor would want to hear.

Student 2 — Auth & OTP Security

Role in the team: How a stranger becomes a trusted user — and how a stolen password alone still is not enough.

Owns: AuthService, PasswordService (PBKDF2), TokenService (rotation + reuse detection), MfaService, OtpService (hashed one-time codes), MailService (Brevo), StepUpService, and the rate limiter; the eight auth screens.

Feature claims: FT-001 to FT-024 plus the post-audit security addendum FT-217 to FT-234.

Two-minute pitch. *Say this, then demo:*

I own how a stranger becomes a trusted user — and how a stolen password alone still is not enough. Registration hashes passwords with PBKDF2 at a hundred and twenty thousand iterations and a per-user salt; login issues a fifteen-minute access JWT plus a seven-day refresh token stored only as a SHA-256 hash; and the refresh endpoint rotates tokens, so presenting an old one after rotation — the signature of a stolen token — nukes the whole token family. This sprint I rebuilt the fragile flows around emailed one-time codes: password reset now emails a six-digit OTP stored only as a hash of purpose, email and code, single-use and dead in ten minutes — the old code-in-the-response anti-pattern is deleted, not deprecated. Invites email a 24-hour code instead of a temporary password, destructive actions answer 403 STEP_UP_REQUIRED until you trade an emailed code for a five-minute single-purpose token, and login can fall back to email MFA through the same OTP engine. In the demo I replay a rotated refresh token on purpose and the audience watches the family die.

Likely personal questions and model answers:

QP2.1. Why store refresh tokens hashed if they are already random?

Because the database should not be able to give a thief a usable token. A hash means a full database leak yields nothing usable directly; the attacker would have to reverse a SHA-256 of a 43-character random string. It costs one hash per refresh and buys immunity to the most damaging dump scenario.

QP2.2. Walk me through exactly what happens when a rotated token is replayed.

The presented token is hashed and looked up; the row is found but already revoked, which by definition only happens if something replayed it. The server answers 409 TOKEN_REUSE and revokes every refresh token belonging to that user, so whatever device holds the stolen one is locked out with it. The legitimate user simply logs in again — annoying for them, fatal for the attacker.

QP2.3. What exactly changed in the OTP overhaul?

Two anti-patterns died: the password-reset endpoint that returned the reset code in the response, and invites that carried a temporary password in JSON. Their replacements share one engine: six-digit codes from SecureRandom, stored only as sha256 of purpose, target and code, single-use, ten-minute TTL — 24 hours for invites — five wrong attempts burn the code, and one live code per purpose and target. Responses never contain codes in any mode; dev mode puts them in the log and an X-Dev-OTP header, live mode sends real Brevo email. Every flow, including the negative paths, is re-driven live with manual API calls at the defence.

QP2.4. Why did you return a special 440 status for MFA instead of a normal 401?

The client needs to distinguish two very different situations after a login attempt: wrong credentials, versus correct credentials awaiting a second factor. A distinct machine-readable status makes the MFA screen a routing decision rather than an error message, and it cannot be confused with a failed password. It is a deliberate, documented extension of the standard statuses, used only for that branch.

QP2.5. Could an attacker brute-force the six-digit code?

They get ten attempts per minute per IP from the shared rate limiter, and each attempt must land inside the code's ten-minute life with a five-attempt burn limit, so the search space is practically unreachable. The limiter plus the short window make guessing a losing game, and failed attempts are audited.

Student 3 — Business Logic (Documents, Sharing, Verification)

Role in the team: The bytes and the doors: encrypted storage, versioned documents, and the share engine that faces strangers.

Owns: DocumentService, FileVault (AES-256-GCM), ShareService (atomic guards, passcodes, OTP gating), VerifyService with version pinning; the twelve vault screens and the sharing plus public verify screens.

Feature claims: FT-025 to FT-070, FT-071 to FT-096, and FT-209 to FT-212.

Two-minute pitch. *Say this, then demo:*

I own the bytes and the doors they pass through. On the way in: upload streams your file once, computing its SHA-256 as the bytes flow, encrypts with AES-256-GCM under a key from the environment, and writes ciphertext plus the document row, version one and the first history entry in one transaction. On the way out: a share is a row with a 43-character token, a permission level, an expiry, an optional view limit and an optional passcode — and its entire policy is one atomic SQL UPDATE that increments the view count only if the link is unrevoked, unexpired and under its limit; zero rows moved means the link is dead, and my

service re-reads the row to say exactly why. Downloadable shares add a single-use five-minute ticket, so a leaked URL is worthless twice. My demo is the tamper and the dying link: two flipped bytes in the vault flip a verification from valid to tampered, and a thirty-second share dies in front of the audience.

Likely personal questions and model answers:

QP3.1. Why stream the hash instead of hashing after saving?

Because plaintext should exist in as few places and moments as possible. Streaming lets me compute the digest while transferring to the encryptor in the same pass, so there is no intermediate plaintext file on disk and no second read. It also gives honest file sizes and rejects oversize uploads before we have stored anything.

QP3.2. What exactly does the GCM tag protect against that a plain hash does not?

They protect different axes. The hash tells us what we wrote at upload time; the GCM tag is verified against the key during decryption, so it detects any modification of the stored ciphertext afterwards, even by someone with disk access but not the key. We use both: the tag at decrypt time, the digest for the tamper evidence we can show users and verifiers.

QP3.3. A user uploads two versions at the same moment. Trace it.

Both services read the current max version, both target N plus one, both insert inside transactions; the UNIQUE constraint on document and version number lets the first commit and rejects the second, which we catch and rethrow as a 409 CONFLICT after rolling back — no orphan file, no phantom row. The client then offers a retry, and the loser's staged bytes are deleted. The edge-case demo runs this from two terminals simultaneously.

QP3.4. Why one atomic UPDATE instead of check-then-increment in code?

Because check-then-increment is a race: two opens can both pass the check and both increment, overshooting max_views. Making the guard the WHERE clause of the increment means Postgres row locks serialise the two, and the loser matches zero rows — the count can never exceed the limit. One statement is also the audit-friendly unit: rows moved, or not.

QP3.5. You chose tickets over signed URLs for public downloads. Defend that.

A signed URL is valid until it expires; you cannot revoke it early, which breaks our instant-revocation guarantee. A ticket is a server-side grant: single-use, five minutes, bound to one share, redeemed through the same guard path. The cost is one lookup per redemption; the benefit is that revocation is truly instant for downloads too. We logged this decision in the roadmap because the losing option was reasonable.

Student 4 — Mobile App (React Native 0.88 / Expo SDK 58)

Role in the team: Everything the user touches: 46 screens, nine frontend microservices, the MVP pattern, and the app's own configuration story.

Owns: The Expo client — screens, components, navigation, the microservice folders (api / model / presenter each), BasePresenter and usePresenter, ApiClient with single-flight refresh, mobile/.env with EXPO_PUBLIC_API_URL, and the SDK 58 dependency matrix.

Feature claims: FT-133 to FT-172 and FT-173 to FT-178, plus the mobile side of FT-217 to FT-234.

Two-minute pitch. *Say this, then demo:*

I own everything the user touches. The client is React Native 0.88 on Expo SDK 58 — the newest SDK generation, with the new React Native architecture and iOS 27 support — organized as nine frontend microservices, so six students could work without collisions. The API client handles the boring-but-critical parts silently: it unwraps our response envelope, refreshes an expired access token exactly once with single-flight de-duplication, and surfaces typed error codes instead of raw failures. This sprint I rebuilt the security UX: the login MFA screen offers 'email me a code instead' and finishes the 440 challenge; the password-reset screen takes the emailed six-digit code; and both destructive flows detect the server's 403 STEP_UP_REQUIRED, prompt for the emailed code, trade it for a five-minute step-up token and retry with the X-Step-Up-Token header. Where the app finds the server is configuration, not code: mobile/.env holds EXPO_PUBLIC_API_URL, env.js reads it, and the root .gitignore keeps the real .env out of GitHub. Proof on this machine: expo-doctor reports 20/20 checks clean on SDK 58, Metro bundles the whole app to Hermes bytecode, and the emulator drives the same flows against the live API.

Likely personal questions and model answers:

QP4.1. What did the SDK 58 upgrade touch, and what could have broken?

Only configuration. package.json moved to expo ^58.0.0 and every dependency was realigned to the SDK 58 matrix with npx expo install --fix; app.json dropped the splash block that the new schema forbids; .npmrc sets legacy-peer-deps because the jump crosses React 19. No screen, presenter or API code changed. What could have broken — native module versions above all — is exactly what the verification covers: expo-doctor 20/20 on the SDK 58 matrix and a clean Metro/Hermes bundle, then the emulator driving the real flows against the live server.

QP4.2. Why is the screen registry more than a folder of imports?

Because it is the single indirection point that navigation, deep links and notification taps share: a name maps to a component, its owning module and its title, and nothing navigates by constructing paths ad hoc. That gave us role-based mounting — the admin stack exists only

for admins — and a countable inventory: forty-six entries is not a claim, it is a file. When a screen moves or a module changes hands, the registry is the one place that records it.

QP4.3. What does BasePresenter standardise, concretely?

Five states — idle, loading, success, error, empty — plus attach and detach and a shared error mapper. Because every screen passes through it, every screen shows the same skeletons, the same empty states and the same toast behaviour, and the presenter logic stays in plain, reviewable classes with no React rendering. The pattern is why forty-six screens feel like one app instead of forty-six prototypes.

QP4.4. Why does the logic live in presenters rather than in the components?

Because the bugs that matter live in decisions, not rendering: does login route MFA correctly, does pagination accumulate, does the client refresh once for five parallel 401s. Keeping those decisions in plain presenter classes makes them reviewable line by line and checkable by driving the class directly — no rendering involved. Component snapshots would have looked impressive and caught nothing we cared about.

QP4.5. Is client-side token storage in AsyncStorage secure enough?

It is the platform-standard trade-off for this kind of app: AsyncStorage is sandboxed per app, so another app cannot read it, and the real threats are device theft and device-level compromise. We mitigate what we can from the server side — fifteen-minute access tokens, rotation with reuse detection, revocation on logout and password change — and a production roadmap item would be encrypted storage or biometric-gated keychain use.

Student 5 — The 8-Pass Security Audit

Role in the team: Attacking our own system before an examiner could: eight thematic passes, finding IDs that live in the code comments, and the three hardening batches that fixed them.

Owns: The eight audit passes and their finding IDs (A-1 to H-3), and the three hardening batches that fixed them — the proof is that every finding ID is greppable in the shipped code comments, and every hardened flow is re-driven live with manual API calls (Postman/cURL).

Feature claims: The audit findings and the eighteen post-audit security features (FT-217 … FT-234); every ID cited in code comments is demonstrable.

Two-minute pitch. *Say this, then demo:*

My job was to attack our own system before an examiner did. I ran eight thematic passes over the original code — access control, crypto and vault, concurrency, configuration, input

validation, information leakage, error handling, and data integrity — and logged every finding with an ID and a severity. The fixes shipped in three hardening batches, and the IDs stuck: B-1 was refresh-reuse detection that used to fail open — the revocation was rolled back by the very exception that reported it, so a stolen token stayed usable; C-1 was nested transactions holding two pool connections at once, which could commit an audit row even when the outer transaction rolled back — a phantom audit; E-3 made the server refuse to boot on weak JWT secrets; D-1 pinned the vault key to exactly 32 bytes. Every fix landed with its finding-ID comment, so the audit trail is in the code: search any ID letter and you will find the comment explaining what broke and why the fix is shaped the way it is.

Likely personal questions and model answers:

QP5.1. What was your most serious finding?

B-1, the refresh-token reuse detection that rolled back its own revocation. It is the nastiest class of bug — the security control exists in the code, but the transaction boundary made it fail open exactly when it mattered. The fix separates 'revoke the family' into its own committed transaction before the 409 is raised, with the audit event written best-effort afterwards so an audit outage can never undo a revocation. You can read the whole story in the comments around TokenService.rotate.

QP5.2. Explain the phantom audit bug — why is it dangerous?

C-1. The old code opened a second connection inside an outer transaction to write the audit row, and the two committed independently. So a business operation could roll back — the thing never happened — while its audit row survived: the trail then claims events that never occurred, which is worse than a missing row in a system whose promise is tamper-evident history. The fix is transaction hygiene: one connection per transaction through the shared runner, and post-commit side effects written best-effort, so the trail can only lag, never lie. The fix comments in ShareService and SettingsService record the rule exactly where the bug lived.

QP5.3. How did you prioritize eight passes' worth of findings?

By exploitability times blast radius. Anything that failed open, leaked a credential, or crossed an ownership boundary was batch 1 — 'someone can get in or see someone else's data'. Integrity and tamper-evidence was batch 2. Concurrency and robustness cleanups were batch 3. And documentation-level risks got logged with a named fix path rather than pretend-fixes. Nothing was closed because we felt like it — each closure points at a finding-ID comment in the code.

QP5.4. Did the audit ever produce a false alarm?

Yes, and that is worth saying out loud. The OTP overhaul initially looked like the overhaul had broken something when our rehearsal API calls started hitting 429s — but the limiter was doing its job: machine-speed repeated calls from one IP legitimately exhaust a 10-per-minute auth bucket, and a real user doing password-reset, MFA and step-up in a minute would come close too. The honest fix was a scope-aware budget — 20 a minute for auth, 30 for public share endpoints — not disabling the limiter. Knowing the difference between 'the system is broken' and 'the system is protecting itself' is the core audit skill.

QP5.5. Give one finding that was not a classic security hole.

H-1: the default document download resolved to the wrong version when the current-version pointer and the mirror disagreed, so a user could fetch bytes other than the version they were shown. The fix pins download resolution to one definition of current. H-3 was an orgName join that silently dropped organisations with unusual casing, and F-4 pinned the client's document versions so a stale screen cannot overwrite a newer one. None of these leak data on their own — all of them break a promise the UI makes, which is why they were worth a pass of their own.

Student 6 — Deployment & DevOps

Role in the team: Getting it running — anywhere, in one command — and keeping it observable, repeatable and secret-clean.

Owns: Maven pom.xml and the one-command build-and-boot path (mvn clean package, then mvn exec:java), alongside .env configuration with fail-fast validation, the root .gitignore and the mobile env files, the demo seeder, the ExpiryScheduler, metrics and health endpoints, and optional Sentry Spotlight tracing.

Feature claims: Infrastructure libraries (HikariCP, Bucket4j, Caffeine, Dropwizard Metrics, Sentry) plus the operations surface: boot, health, metrics, scheduler, seed.

Two-minute pitch. *Say this, then demo:*

I own getting it running — anywhere, in one command. The server is a single Java 21 process: one command — mvn clean package — fetches the exact eight libraries from Maven Central, compiles and packs the runnable jar, and mvn exec:java boots it with a fail-fast config check: a weak JWT secret or a missing vault key stops startup with a human explanation instead of a runtime surprise. Configuration is environment variables documented in .env.example, including the Brevo mail credentials and the MAIL_MODE switch — dev mode routes OTP codes to the log and an X-Dev-OTP header for demos, live mode sends real email. Secrets never reach the repository: a comprehensive root .gitignore blocks secrets, build output, dependencies and editor settings, and git check-ignore proves any pattern live. Operations got real telemetry: HikariCP pool stats and Dropwizard request

metrics surface on /api/health, and Sentry Spotlight tracing is one optional flag. Proof: from a clean clone, Maven packages and boots the server with the same .env and the same boot log — demonstrated live at the defence — while the boot log prints migrations=12, routes=74.

Likely personal questions and model answers:

QP6.1. How would you deploy this to real users?

The server is a single jar with no framework: any JVM 21 host or container works. I would run it behind a reverse proxy for TLS (the API already sets CORS per config), point DB_URL at managed PostgreSQL — we already support the Supabase pooler form — set MAIL_MODE=live with the Brevo key, VAULT_KEY from a secret store, SEED_DEMO=false (the server itself refuses demo seeds on non-localhost databases), and let the migration runner bring the schema up on first boot. Health checks already exist at /api/health.

QP6.2. What stops a developer from leaking production secrets into the repo?

Three fences. First, secrets only exist in .env files, and a comprehensive .gitignore sits at the project root blocking secrets, build output, dependencies, editor settings and OS files from ever being committed — you can verify any pattern live with git check-ignore. Second, .env.example documents every variable with placeholders that the server actively refuses — the JWT placeholder is on a blocklist (E-3) — so nobody can run with example secrets by accident. Third, the committed template files carry no real values: mobile/.env.example ships YOUR_PC_IP_ADDRESS as its placeholder.

QP6.3. If the server crashes at 3am, how do you know what happened?

Three layers. Every request logs one line — method, path, status, duration. Every security-relevant event lands in the append-only audit_logs table with actor, IP and JSON detail. And /api/health plus the Dropwizard counters on /api/meta show whether the database and the request path are healthy. Optional Sentry Spotlight tracing adds the distributed-timeline view when it is switched on. The logs answer 'what happened'; the audit trail answers 'who did it'; the metrics answer 'is it still happening'.

QP6.4. What if the demo machine cannot reach Maven Central?

The build fails loudly at the resolve step — Maven names the artifact it could not fetch, which is the honest behaviour for a pinned build. The mitigation we rehearse is Maven's offline mode against a pre-populated local repository cache: the eight versions are fixed, so a machine that built once can rebuild offline. Same code, same jars, no network surprises because the versions are pinned in pom.xml.

QP6.5. How does the scheduler avoid notifying twice about the same expiry?

One notification per document, per type, per calendar day: before inserting, the scheduler checks for today's existing row with the same entity and type. A document sitting in the expiring state for two weeks produces fourteen reminders, not hundreds. The same sixty-second tick retires expired verification requests so their public links stop answering valid.

Demo-day survival tips. Arrive with the server already running and the seed data fresh, because first boots are for the preparing, not the performing. Keep three terminal windows ready: one tailing the server log, one with the demo curl scripts from docs/EDGE_CASES.md, and one with the SQL editor open for the append-only exception demo. When a question stalls you, attach it to a demonstration — show the row, the log line or the code comment — and if you genuinely do not know, say what you would look at first; a precise unknown beats a vague guess every time.

The Defence Masterclass — Deep Dives for Each Student

This chapter mirrors the six Masterclass files in `docs/defence/` (*STUDENT_1_INFRASTRUCTURE_DB.md* through *STUDENT_6_DEVOPS.md*) — the complete rehearsal brief for each member. Every deep dive carries the verbatim pitch, the file-by-file ownership map, five layman's analogies, the critical code walkthrough, the plain-words glossary, the personal Q&A bank and the cross-team handshake; every file, class and method it names exists in the repository and can be opened live.

> *Note: every claim in this handbook is verifiable from the public repository alone — finding IDs are greppable in the code comments, mvn clean package builds the runnable jar from a clean clone, and every flow is demonstrated live at the defence with manual API calls (Postman/cURL) or on the emulator.*

Student 1 — Infrastructure & Database

This file is the complete rehearsal brief for the student presenting **Student 1 — Infrastructure & Database** in the DocuTrust oral defence (Group 3). It is written to be read aloud from, handed to examiners, and pointed at during the live demo. It expands the matching role section in `DEFENCE_GUIDE.md` — the master index at the repository root — which carries the one-page version of this role. Everything here is grounded in the real repository: every class name, method name, migration and number below can be opened and verified live during the defence.

My slice in one page

Role in one line: The skeleton and the truth layer — the pure-Java server platform every request travels through, and the PostgreSQL schema whose invariants live in the database engine itself.

Owns:

- `Router`, the middleware chain, the `Ctx` request context and the error envelope
- the JWT service (`JwtService`) and the rate limiter (`RateLimiter`) as platform services
- config/boot (`Config`, `Main` as composition root — shared with Student 6, who owns the run story)
- the migration runner (`Migrator`), the HikariCP connection pool (`Database`) and the transaction runner (`Tx`, `Rows`)
- the whole PostgreSQL schema: migrations V01–V12, 15 tables, CHECK constraints, foreign keys, triggers, views, indexes
- the mobile app shell side of the platform: the Dashboard screen and the navigation registry

Feature claims: FT-193 to FT-202 and FT-206 to FT-208 (server platform), plus the Dashboard and navigation registry features FT-173, FT-177, FT-178.

My numbers:

- **74 API routes** — registered in `Main.start`, printed at boot as `routes=74`
- **15 database tables** — built by migrations V01–V12 (plus `schema_migrations`, the migration runner's own clipboard, which is not one of ours)
- **12 migrations** — one numbered SQL file each in `database/migrations/`
- **1 runnable jar** — `mvn clean package` packs the whole server into the self-contained `docutrust-server.jar`
- **216 numbered features + the post-audit addendum (FT-217–FT-234)** in `docs/FEATURES.md`; my slice covers the Server Platform module FT-193–FT-208
- **3 roles** — `admin` > `org_manager` > `member`, enforced in `Middleware.requireRole`
- Connection pool: **min 2 idle, max 10 connections**, 10-second connection timeout, 30-minute max connection lifetime (HikariCP)
- Request executor: **32 worker threads** (`Executors.newFixedThreadPool(32)` in `Main.start`)
- Request body cap: **25 MB** (`Ctx.MAX_BODY_BYTES`, rejected with HTTP 413)
- Access-token lifetime: **900 seconds (15 minutes)** — `JwtService.ACCESS_TTL_SECONDS`

- Rate-limit budgets: **20/min for `auth`, 30/min for `public-share`, 20/min for `public-verify`, 10/min default** — `RateLimiter.SCOPE_DEFAULTS`
- Migration ledger: **`schema_migrations` (name TEXT PRIMARY KEY, applied_at timestamptz)**, created by `Migrator.apply`

  Note: *every claim in this brief is verifiable from the public repository alone — the boot log prints the numbers,* `mvn clean package` *builds the runnable jar from a clean clone, and the live demo walks a real request through every layer during the defence.*

Elevator pitch (≈ 1 minute)

> *"I built the skeleton everything else hangs on. DocuTrust's server is pure Java 21 on the JDK's built-in HTTP server — no Spring — so every request is matched by a hand-written router with 74 registered routes, then passes through a middleware chain: CORS, logging, JWT authentication, role guards and rate limiting. Underneath, HikariCP manages the database pool, and the schema itself does security work: fifteen tables with CHECK constraints, foreign keys, and append-only triggers so that even a database console cannot rewrite an audit trail. My proof is simple: the boot log prints* `migrations=12, routes=74`*, and* `mvn clean package` *turns a clean clone into the runnable jar in one command."*

Then demo: run one request live against the local server (a `GET /api/admin/stats` with an admin token) and show the console line `[http] GET /api/admin/stats -> 200 (... ms)` appear as it is served; then open the SQL editor and run a manual `UPDATE` against `audit_logs` so the audience watches PostgreSQL itself throw the append-only exception. The numbers on screen — `migrations=12, routes=74` — come from the boot log of `Main.start`, so the claim checks itself.

What I own, file by file

Every path below exists in the repository today — open any of them during the defence.

The request boundary — `server/src/main/java/com/docutrust/http/`

File	What it does, in one plain-English sentence
http/Router.java	The hand-written menu index: it matches an incoming method + path against all 74 registered routes, prefers static segments (like /expiring) over parameters (like /:id), and turns any unknown request into a clean 404 envelope instead of a crash.

File	What it does, in one plain-English sentence
http/Middleware.java	The security lane every protected request walks through: authenticate checks the bearer JWT and loads the user, requireRole/requireAnyRole enforce admin/org_manager/member (auditing every denial), and rateLimited applies the per-scope, per-IP token bucket.
http/Ctx.java	The standard order slip: one object per request that carries the path, query, route params, the parsed JSON or multipart body (capped at 25 MB), the authenticated user, and every response helper — including the CORS headers stamped onto every reply.
http/Handler.java	The one-line contract shared by router, middleware and controllers: a handler receives a Ctx and either responds or throws.

The error envelope and shared toolbox — package root and `util/`

File	What it does, in one plain-English sentence
AppException.java	The typed error every layer throws: it carries an HTTP status, a machine-readable code (like VALIDATION_ERROR), a human message and an optional field, which the router converts into the one error envelope the whole API speaks.
util/Json.java	Our hand-rolled JSON parser and serializer (no external library) — it writes and reads every request body and every response envelope, including the {"ok":true,"data":…} wrapper.
util/Text.java	The shared text-and-crypto toolbox the platform leans on: base64url for JWTs, sha256Hex, random token generation, and constEq — the constant-time string comparison that keeps token verification timing-safe.

Platform security services — `auth/` (the platform-owned half)

File	What it does, in one plain-English sentence
auth/JwtService.java	Our own HMAC-SHA256 JSON Web Token machine: issue prints a signed token with subject, role, org, iat/exp/jti claims, and verify checks the signature, distinguishes expired from invalid tokens with dedicated exception types, and returns typed Claims.

File	What it does, in one plain-English sentence
auth/RateLimiter.java	The bouncer's clicker, built on Bucket4j token buckets with the bucket map held in a Caffeine cache: per-scope budgets (20/min for auth, 30/min for public-share, 20/min for public-verify, 10/min default) keyed by scope + IP, with idle buckets evicted so memory cannot leak.

Boot and configuration — shared with Student 6 (they own the run story; I own what the boot wires)

File	What it does, in one plain-English sentence
config/Config.java	Loads configuration once at boot (system properties > environment variables > optional .env file) and runs the fail-fast gates — weak/placeholder JWT secret (E-3), vault-key format (D-1), localhost-only demo seeding, live-mail credentials — refusing to start with a human explanation instead of a runtime surprise.
Main.java	The composition root: Config.load(), migrate, construct the pool and every service and controller, register all 74 routes, seed, start the scheduler and the HTTP server, then print the proof line DocuTrust ready on port 8080 (migrations=12, routes=74, db=up, boot=…ms).

The database platform — `db/`

File	What it does, in one plain-English sentence
db/Database.java	The HikariCP-backed JDBC pool (min 2 idle, max 10, 10 s connection timeout, 30 min max lifetime) behind a deliberately tiny surface — get/release/with/tx/healthy/close plus poolIdle/poolActive telemetry — so callers can ride the pool without knowing it is Hikari.
db/Migrator.java	The moving truck: applies every V*.sql in filename order, one transaction per file, recording each delivery in the schema_migrations table so re-running is always safe, with a splitter that respects quoted strings, $$ dollar-quoted bodies and -- comments.
db/Tx.java	The all-or-nothing rule: Tx.with turns autocommit off, runs the unit of work, commits on success and rolls back on any exception, restoring the old autocommit setting in a finally.
db/Rows.java	The read side of the DAO layer: static mappers that turn a JDBC ResultSet row into a typed model (Rows.user, Rows.document, Rows.share, …), handling nulls and timestamps uniformly.

The schema story — `database/migrations/` (V01–V12)

Migration	What it adds
V01__extensions_and_organisations.sql	The pgcrypto and pg_trgm extensions (UUID generation and fuzzy text search), plus the first three tables: organisations (with a plan CHECK), users (role/status CHECKs, unique email) and the auth scaffolding refresh_tokens (unique token_hash) and password_resets.
V02__folders_documents.sql	folders (unique folder name per owner + parent) and documents (doc_type/status/review_status CHECKs, reminder window), including the star constraint documents_sensitive_needs_review: a medical or financial document physically cannot be active without review.
V03__versions_history_shares.sql	document_versions (UNIQUE on document + version number, positive file size), document_history (the auto-written timeline) and shares (permission CHECK, view-count guards, the 43-character link-token rule, and a CHECK that every share is either to a user or a link).
V04__verification_notifications.sql	verification_requests (status/result CHECKs, unique token), notifications and per-user settings (reminder offset between 1 and 90 days).
V05__admin_reviews_audit.sql	admin_reviews (the review queue) and audit_logs — the platform's append-only security diary with actor, action, IP, user agent and a JSONB meta column.
V06__indexes.sql	The speed layer: partial indexes for the hot queries (alive documents per owner, active share tokens, unread notifications), trigram GIN index for fuzzy title search, and audit/review ordering indexes.
V07__triggers.sql	The honesty layer: touch_updated_at stamps, the append-only triggers (trg_history_immutable, trg_audit_immutable — any UPDATE or DELETE raises an exception), the auto-history trigger trg_documents_history that writes created/version_added/status_changed rows, and trg_version_sync which mirrors the newest version's hash/size onto the document head.
V08__views.sql	Two saved queries as views: v_expiring_documents (what is about to expire) and v_admin_stats (the registrar dashboard's numbers).
V09__verification_pinning.sql	Adds version_id to verification_requests and backfills it, so every verification proof is pinned to the exact document version that existed when it was minted (audit finding F-4).
V10__email_otps.sql	The email_otps table of the OTP security overhaul — one row per hashed, single-use, purpose-salted code — plus three indexes for target, user and share lookups.

Migration	What it adds
V11__shares_require_otp.sql	Adds require_otp and otp_recipient_email to shares, turning a share link into a doorman that checks one emailed code.

The mobile shell I own — `mobile/src/`

File	What it does, in one plain-English sentence
mobile/src/screens/index.js	The screen registry (SCREEN_REGISTRY, 45 entries, counted by SCREEN_COUNT): one place where a screen name maps to its component, owning module and title, so navigation, deep links and notification taps all share one directory instead of ad-hoc paths.
mobile/src/navigation/RootNavigator.js	The app switcher: builds the stacks from the registry and mounts the admin stack (AdminStack) only when the session role is admin — role-based navigation decided in one file.
mobile/src/screens/DashboardScreen.js	The member's home screen: the shell view that greets the user and routes into the documents, shares and notifications flows.
mobile/src/screens/AdminDashboardScreen.js	The registrar console's home screen: the admin-side shell over the review queue, users and audit views.

Shared platform files — I can speak to them, the feature owners drive them

File	What it does, in one plain-English sentence
model/Models.java	The standard plate shapes: every typed record the mappers fill (Models.User, Models.Document, Models.Share, …) plus Models.Page, the pagination wrapper behind okPage.
vault/FileVault.java	The safe's interface that my boot constructs with the validated 32-byte key (Main.start → new FileVault(Path.of(cfg.vaultDir), vaultKeyBytes(cfg.vaultKey))) — the encryption itself is Student 3's feature.

Reading the error envelope, field by field

Every failure in the API leaves through one door — `Router.sendError` — so an examiner only needs one shape:

Envelope field	What it carries	Example
ok	Always false on errors, true on success — the client's first check	false

Envelope field	What it carries	Example
error.code	The machine-readable reason a screen can translate	VALIDATION_ERROR, TOKEN_EXPIRED, RATE_LIMITED
error.message	The human sentence, safe to show	"Email is not valid"
error.field	Optional; names the offending input, set by AppException.validation(field, msg)	"email"

The codes are minted by the factory methods on `AppException` — `unauthorized` (401), `forbidden` (403), `notFound` (404), `conflict` (409), `tooLarge` (413), `rateLimited` (429), `internal` (500) — so no layer ever hand-assembles a status code, and the full catalogue is documented in `docs/API_CONTRACTS.md`.

The pooling-and-caching layer — two Caffeine caches

The infrastructure upgrade gave the platform its caching half, both built on Caffeine with hard ceilings so neither can grow unbounded:

Cache	Where	Ceiling	Eviction rule
Rate-limiter bucket map (RateLimiter.buckets)	one Bucket4j token bucket per scope\	ip key	maximumSize(50_000)
Document read cache (DocumentService.docCache)	per owner+document lookup on the hot read path	maximumSize(1_000)	expireAfterWrite(10 s) — residual staleness is bounded by the TTL, and every mutation invalidates its key explicitly

The first cache is entirely mine (it is the rate limiter's memory); the second serves Student 3's document feature but is part of the caching layer this slice owns — and its contract is the honest one: ownership is re-checked on every write, and any change to a document drops its cache entry in the same request.

Five layman's analogies

1. **HikariCP is a restaurant host.** Guests (database queries) don't wait for a free table to be built; a host keeps a fixed set of tables ready and seats the next guest at whichever frees up first. Before HikariCP we hand-wrote that seating logic; now it's a battle-tested library with the same seats (our code's API didn't change at all).

2. **Migrations are LEGO instructions in order.** Each numbered file (V01–V12) adds a few bricks — a table here, an index there — and the database is just the sum of the steps. Any team member can rebuild the exact same database from scratch, which is why our ER diagram and our production schema can never drift apart.
3. **The router is a hotel receptionist.** You say "I'm here for room 12" (`/api/documents/:id`) and the receptionist knows static room numbers take priority over "the room with whatever name" — exact matches win over wildcards. One receptionist, 74 known destinations, and anything unknown gets a clean 404 instead of a crash.
4. **The middleware chain is airport security.** Every passenger walks the same lane before reaching the shops: the passport scan is JWT authentication, the boarding-class check is the role guard, and the queue counter that says "too many people, wait" is the rate limiter. No shop — no controller — is ever reached without passing through the lane first, and the lane's order never changes, which is exactly why it is owned by one person.
5. **The append-only audit trigger is a cash-register till tape.** The tape only prints new lines; you cannot edit or un-print a line, not even the shop owner can, and at the end of the day the tape is the one thing everyone trusts in a dispute. Our `audit_logs` and `document_history` tables work the same way: the database itself raises an exception if anyone — application, administrator, or a hand-typed SQL command — tries to alter or tear out a line.

The critical code walkthrough

One request's full journey, in order, naming the exact classes and methods. Example request: `GET /api/admin/stats` with a valid admin bearer token.

1. The socket. The JDK's built-in HTTP server (`com.sun.net.httpserver.HttpServer`) is created in `Main.start`: `HttpServer.create(new InetSocketAddress(cfg.port), 0)`, then `server.createContext("/", router)` hands every path to our one `Router`, and `server.setExecutor(Executors.newFixedThreadPool(32))` gives it 32 worker threads. There is no framework between the TCP socket and our code — the first line of Java that runs is ours.

2. `Router.handle(HttpExchange)`. The router wraps the exchange in a `Ctx` (the order slip), starts a millisecond timer, and short-circuits CORS preflight: an `OPTIONS` request is answered `204` immediately via `ctx.raw(204, null, null)`.

3. `Router.match(method, path)`. Every registered route was pre-split into segments in the `Route` constructor, each `:param` segment flagged and each route scored with `staticSegs`. `match` walks all routes, keeps the ones whose segment count equals the path's (`matches`),

collects parameter values into a map, and keeps whichever candidate has the **highest static segment count** — that one line, `if (matches(r, path, params) && (best == null || r.staticSegs > best.route().staticSegs))`, is the whole static-over-param precedence rule. It is why `/api/documents/expiring` and `/api/shares/incoming` are never swallowed by `/:id` patterns. No match → `AppException.notFound(...)` → a 404 envelope, never a crash.

4. The middleware chain — wrapped at registration time. The chain is not a runtime loop; it is composed when `Main.start` registers each route, so the order is fixed in code. `router.role("GET", "/api/admin/stats", "admin", adminC.stats())` literally wraps `mw.authenticate(mw.requireRole("admin", h))`. For a request that means:

- `Middleware.authenticate`: reads the `Authorization: Bearer` header via `ctx.bearer()`; no token → 401. `jwt.verify(token)` (in `JwtService`) checks the HMAC-SHA256 signature with the constant-time `Text.constEq`, then the expiry — a `TokenExpiredException` becomes `401 TOKEN_EXPIRED` (which the mobile client uses to trigger its refresh), a `TokenInvalidException` becomes plain `401 UNAUTHORIZED`. The claims' `sub` is then loaded from the database (`loadUser` → `db.with` → `Rows.user`), so a deleted account's token dies immediately, and a suspended account gets `403 ACCOUNT_SUSPENDED`.
- `Middleware.requireAnyRole`: the user's role must be one of the allowed ones; a denial is not silent — `auditDenied` writes an `admin_access_denied` row into `audit_logs` before the `403 FORBIDDEN` is thrown.

5. The metrics timer. Around the handler call, `Metrics.route(method, pattern)` opens a Dropwizard timer and marks a meter **per route pattern** (e.g. `GET /api/admin/stats`), not per URL — so the metrics namespace can never explode with ids (the INFRA-2 comment in `Router.handle`). It is a try-with-resources block, so even requests that fail are counted.

6. Controller → service → DAO. The handler `AdminController.stats()` is a waiter: it never cooks. It delegates to `AdminService`, which does the real work through the `db` platform: every read is a `PreparedStatement` (SQL-injection safe by construction) executed through `db.with(...)` or `db.tx(...)`, and rows come back as typed models via `Rows`. The stats themselves come from the `v_admin_stats` view that migration V08 created — the database does the aggregating.

7. HikariCP → PostgreSQL. `db.with` borrows a pooled connection (`Database.get()` → `ds.getConnection()`), runs the work, and always returns the connection in a `finally` (`release`, where `close()` means "back to the pool"). The pool is configured for min 2 idle / max 10 connections, a 10-second borrow timeout and a 30-minute max lifetime, so connections retire before PostgreSQL's idle timeouts can kill them.

8. The response envelope. The controller calls `ctx.ok(data)`, which builds `{"ok":true,"data":…}` with `Json.obj` and sends it through `Ctx.respondMap` → `Ctx.raw`. `raw` stamps `Content-Type: application/json; charset=utf-8` and the CORS headers (`Access-Control-Allow-Origin` from `cfg.corsOrigin`, plus `X-Step-Up-Token` in the allowed headers) onto **every** response, writes the body, and closes the exchange.

4b. The authentication decision table. `Middleware.authenticate` plus the role guard produce exactly these outcomes — every branch is a line you can find in `Middleware.java`:

Situation at the gate	What the middleware answers
No Authorization: Bearer header	401 UNAUTHORIZED ("Missing bearer token")
Signature does not verify (TokenInvalidException)	401 UNAUTHORIZED ("Invalid access token")
Signature valid but exp passed (TokenExpiredException)	401 TOKEN_EXPIRED — the client's cue to refresh
sub not found in users	401 UNAUTHORIZED ("Account no longer exists")
Account status = 'suspended'	403 ACCOUNT_SUSPENDED
Valid user, wrong role for the route	403 FORBIDDEN, plus an admin_access_denied audit row

9. The log line, always. In the router's `finally` block, exactly one line prints per request: `[http] GET /api/admin/stats -> 200 (12 ms)`. Errors take the same corridor: an `AppException` becomes `sendError(...)` — `{"ok":false,"error":{"code":…,"message":…}}` — while any unexpected exception is caught by `logUnexpected`, which writes an `http.internal_error` row to `audit_logs` and returns a generic `500 INTERNAL_ERROR` so no stack trace ever reaches a client.

And the truth-layer half of the proof: in a SQL console, `UPDATE audit_logs SET action='x';` fails outright — the trigger `trg_audit_immutable` (migration V07) raises `audit_logs is append-only (platform audit)` before the statement can touch a single row. A tiny fragment of the rule, from `V07__triggers.sql`:

```
CREATE FUNCTION forbid_audit_mutation() RETURNS trigger AS $$
BEGIN
    RAISE EXCEPTION 'audit_logs is append-only (platform audit)';
END $$ LANGUAGE plpgsql;
```

The write path — same corridor, and where my platform does the heavy lifting

The read example above is half the story; a `POST /api/documents` upload walks the identical corridor with three extra platform moments:

1. `Ctx.multipart()` parses the multipart body by hand — boundary detection, headers per part, byte slices — after `body()` already enforced the 25 MB cap, so the expensive parse only happens for a body we accepted.
2. `DocumentService.upload()` (Student 3's feature) runs inside one `db.tx(...)`: the document row, version one, and the first history entry commit together or not at all — one connection, no nesting.
3. My triggers take over on commit: `trg_documents_history` writes the `created` history row automatically, and `trg_version_sync` mirrors the version's `sha256`, size and mime type onto the document head — the schema doing bookkeeping no Java code has to remember.

The 74 routes at a glance

The route count is not a slogan; it is the registration section of `Main.start`, countable line by line:

Registration group	How they are registered	Count
Public health & meta	publicRoute	2
Public auth (register, login, forgot, reset, verify-otp, accept-invite)	rateLimited("auth", …)	6
Public auth continued (refresh, logout, verify-email)	publicRoute	3
Authenticated profile & MFA (me, change-password, step-up, mfa/*)	auth	8
Authenticated sessions (Active Sessions list + revoke)	auth	2

Registration group	How they are registered	Count
Documents (list, upload, expiring, get/patch/delete, restore, permanent, versions, download, history)	auth	13
Folders	auth	5
Shares (owner side)	auth	6
Public share links (info, open, request-otp, open-with-otp, download ticket)	rateLimited("public-share", …)	5
Verification requests	auth	5
Public verify link	rateLimited("public-verify", …)	1
Notifications & settings	auth	5
Admin (stats, review queue, review detail, decision, users, status, documents, audit logs)	role("admin", …)	8
Organisation (members, invite, role change, remove, stats)	anyRole("admin", "org_manager", …)	5
Total	printed by router.routeCount() as routes=74	74

The schema's guarantees, demonstrated live

Five refusals an examiner can watch the database itself produce, each traceable to one migration line:

Attempt in a SQL console	What PostgreSQL answers	Rule
Set a medical document to status='active' with review_status='none'	CHECK violation	documents_sensitive_needs_review (V02)
Insert a second document_versions row with the same document + version number	UNIQUE violation	UNIQUE (document_id, version_number) (V03)
UPDATE audit_logs SET action='x'	Exception: append-only	trg_audit_immutable → forbid_audit_mutation (V07)
DELETE from document_history	Exception: append-only	trg_history_immutable → forbid_history_mutation (V07)
Insert a share whose link_token is 20 characters	CHECK violation	char_length(link_token) = 43 (V03)

Where each number lives

My claim	Where an examiner verifies it
routes=74	Printed by Main.start; counted from the registration block in Main.java
migrations=12	Printed by Main.start ([main] migrations applied: 12); eleven files in database/migrations/
15 tables	The CREATE TABLE statements across V01–V05 and V10
Runnable jar	mvn clean package on a clean clone packs docutrust-server.jar — the one-command build works from nothing
3 roles	Middleware.requireAnyRole + the CHECK on users.role (V01)
Pool 2–10	The new Database(..., 2, 10, ...) call in Main.start, configured in db/Database.java
32 threads	server.setExecutor(Executors.newFixedThreadPool(32)) in Main.start

My claim	Where an examiner verifies it
25 MB cap	Ctx.MAX_BODY_BYTES in http/Ctx.java

How a fresh database becomes the schema — the `Migrator.apply` loop

Eight steps, each one a line in `db/Migrator.java`:

1. List every file in `cfg.migrationsDir` whose name starts with `V` and ends with `.sql` (a missing directory stops the boot with a `MIGRATIONS_DIR` hint).
2. Sort by filename — the V-numbers *are* the ordering, so nobody can apply V07 before V02.
3. Open one transaction for this file.
4. Inside it: `CREATE TABLE IF NOT EXISTS schema_migrations (name TEXT PRIMARY KEY, applied_at timestamptz DEFAULT now())` — the clipboard, created lazily on the very first run.
5. Ask the clipboard whether this filename is already recorded; if yes, the transaction closes with nothing done (idempotence).
6. Read the file and split it into statements with `Migrator.split` — the splitter that tracks single-quoted strings (with `''` escapes), `$$` dollar-quoted bodies and `--` line comments, so a semicolon inside a comment cannot cut a statement in half (the C-2/C-13 lesson).
7. Execute each statement, then `INSERT INTO schema_migrations(name)` — the delivery and its receipt commit together.
8. Collect the applied names; `Main.start` prints them as `[main] migrations applied: 12`.

My domain, in plain words (glossary)

Term	Plain meaning, with an everyday analogy
Middleware	Airport security: the same lane every passenger walks through (ID check, boarding class, queue counter) before reaching any shop. In code: Middleware.authenticate, requireRole, rateLimited.
JWT (JSON Web Token)	A tamper-proof visitor badge: a signed card that states who you are and until when — the signature makes forging it as futile as photocopying a hologram. Ours is HMAC-SHA256, hand-built in JwtService.

Term	Plain meaning, with an everyday analogy
HTTP status code	The rubber stamp on the reply envelope: 200 "here you go", 401 "no or fake ID", 403 "real ID, not allowed", 404 "no such thing", 429 "too many requests".
CORS	The guest list at the club door for web browsers: a header the server sends declaring which origins may talk to it, so a stranger's website cannot quietly use your logged-in session.
Connection pool (HikariCP)	The restaurant host: a fixed set of ready "tables" (open database connections) that guests (queries) are seated at instantly, instead of building a new table per guest. HikariCP is the battle-tested professional host we hired to replace our hand-written one — same seats, and it reports room occupancy (poolIdle/poolActive).
Transaction	An all-or-nothing order: either every dish of the order is accepted or the whole order is cancelled — never half a dinner. Tx.with does commit-or-rollback.
Migration	One numbered LEGO instruction sheet (V01–V12) that adds bricks to the database; applied in order, once each, tracked in schema_migrations.
CHECK constraint	A house rule even the boss must obey, enforced by the clerk (the database engine): a medical document simply cannot become "active" without review — no buggy code can persuade it.
Foreign key	The cross-reference on a filing card: "owner: see card #42 in USERS." The clerk refuses to file a document whose owner card does not exist.
Trigger	An automatic reflex wired into the table: insert a version and the head row updates itself (trg_version_sync); try to edit the audit tape and the alarm rings (trg_audit_immutable).
Append-only	A cash-register tape: new lines can be printed, existing lines can never be edited or torn out — even by the owner, even by hand-typed SQL.
Index	The tabs on a filing cabinet: pre-sorted shortcuts so the clerk finds "documents owned by me, not deleted" without reading every card (V06 builds them).
PreparedStatement	An order form with printed blanks: the SQL is fixed, only the blank values arrive, so nothing a user types can ever be mistaken for part of the form itself (the cure for SQL injection).

Term	Plain meaning, with an everyday analogy
JSON envelope	The standardized tray every answer travels on: {"ok":true,"data":…} or {"ok":false,"error":{…}} — the client opens every reply the same way.
Rate limiter (token bucket)	The bouncer's clicker: each caller gets a small purse of tokens per minute (20 for auth, 30 for public shares); when the purse is empty, the answer is 429 "wait a minute".

My ultimate Q&A bank

From the per-student pack (the classics)

Q: "Why should one student own the middleware, rather than sharing it?"

Because the middleware chain is where cross-module policy lives, and split ownership would have meant five people editing one file with five opinions. Owning it let me keep the chain small and consistent while module owners focused on services; I still paired with each of them when their endpoints needed a new guard.

Q: "Your router matches static segments over parameters. Why does that ordering matter?"

If /api/documents/expiring were captured by /api/documents/:id, the word expiring would be treated as an id and the route would 404 or misparse. Matching static segments first, parameters second, is what lets both shapes coexist. The `staticSegs` scoring in `Router.match` is the whole rule — one line you can read in the file.

Q: "How do you know your JWT implementation is correct?"

Three ways. The code is small enough to read aloud: sign, verify, check expiry — each a named method in `JwtService`. The middleware treats expired and invalid differently, which the client depends on for refresh. And the live demo runs real login-to-refresh flows over HTTP, so the hand-rolled token is exercised against the running server, not just in isolation.

Q: "If I delete Spring from the equation, what did you actually gain educationally?"

The ability to answer how questions that frameworks answer with because it does. How routing matches, when middleware runs, what a 401 versus 403 means, why a connection pool bounds concurrency — each is a file we wrote. That is the difference between using a framework and understanding one.

Q: "Why put the sensitive-document rule in a CHECK constraint instead of the service?"

Because the service can be bypassed — a migration, a manual SQL statement, a future bug — but the constraint is enforced on every statement by the engine. The rule is one line: a medical or financial document cannot be active unless its review status is submitted. Encoding it in the schema means the review process has no silent back door, which is exactly what an auditor would want to hear.

The masterclass ten

Q: "The pool has 10 connections but the server has 32 request threads. What happens when the pool is exhausted?"

Then up to ten queries run at once and the other requests wait to borrow a connection. HikariCP bounds that wait: `Database` sets the connection timeout to 10 seconds, so a request that cannot borrow within that window gets a SQL exception, which `Tx.rethrow` turns into an error and my router's catch-all converts into a clean `500 INTERNAL_ERROR` envelope — with an `http.internal_error` row in `audit_logs` recording the path. The 32 threads are cheap (they mostly wait on the network or the database); the 10 connections are the expensive, scarce resource, and that bound is precisely what protects PostgreSQL from being stampeded. You can watch the balance live: `Database.poolIdle()` and `poolActive()` expose the host's free and occupied tables.

Q: "Why are your transactions non-nested? What bug does that prevent?"

A nested transaction would hold two pool connections at once — the outer one and the inner one — which on a bounded 10-connection pool is a recipe for self-inflicted deadlock, and worse, the two could commit independently: the outer work rolls back while the inner audit row survives, so the trail claims events that never happened. That exact bug is audit finding C-1. The fix is transaction hygiene: one connection per unit of work, and helpers receive the already-open connection instead of starting their own — for example `NotificationService.notify(Connection c, …)` takes the caller's connection, it never borrows a second one. Side effects that may legitimately lag, like best-effort audit writes, run after the outer commit, so the trail can only be late, never false.

Q: "How do you track which migrations have run, and why no checksums?"

The runner keeps its own clipboard: `Migrator.apply` first creates `schema_migrations` (a name plus `applied_at`), and for each `V*.sql` in sorted filename order it checks, inside the same transaction, whether that name is already recorded — if not, it executes the file statement-by-statement and inserts the name, committing both together. So a fresh database runs all eleven and a five-month-old database runs only what it is missing, and a re-run is always a no-op. We deliberately do not checksum file contents: migrations are append-only history, reviewed in version control — you fix forward with V12, never edit V07 in place, exactly like the LEGO instructions cannot be reordered once built.

Q: "Why are the append-only rules database triggers and not just code discipline?"

Because the whole point of an audit trail is that it survives the failure of everything above it. `trg_history_immutable` and `trg_audit_immutable` execute `forbid_history_mutation` / `forbid_audit_mutation`, which raise an exception on any UPDATE or DELETE — including from a manual SQL console, a migration, or a future bug. Application discipline is a promise; a trigger is physics. It is also the honest answer to "what if someone with database credentials goes rogue": they can add rows, but they cannot quietly rewrite the past, and attempts are loud, not silent.

Q: "Why PreparedStatements everywhere, rather than building SQL strings?"

Three reasons. Security: the SQL shape is fixed and only values are sent, so a user typing `' OR 1=1 --` into a search box is stored as text, never executed as SQL — the injection simply has nowhere to happen. Correctness: types are bound explicitly (UUIDs as UUIDs, timestamps as timestamps), which kills a whole class of quoting bugs. Performance: the database can reuse a prepared statement's execution plan instead of re-parsing a new string every time. Every query in every service and in the middleware follows this one pattern, so there is no "clever" exception to audit.

Q: "Beyond the route table, where does static-over-param routing actually pay off in this API?"

All over the menu. `/api/documents/expiring` exists alongside `/api/documents/:id`; `/api/shares/incoming` alongside `/api/shares/:id`; `/api/notifications/read-all` alongside `/api/notifications/:id/read`. Without precedence, "expiring", "incoming" and "read-all" would be parsed as UUIDs, fail parsing, and surface as mysterious 404s — the classic framework foot-gun. Because `Router.match` scores candidates by static segments, the specific path always wins, and because the boot log prints `routes=74`, an examiner can count the menu live and see that every registered dish is findable.

Q: "What happens to an in-flight request if the database dies mid-operation?"

The borrow or the query throws; `Tx.with` rolls the transaction back — autocommit restored in the `finally` — and the exception is rethrown up to the router, which answers a generic 500 envelope and writes the `http.internal_error` audit row. The server itself does not fall over: the pool keeps the surviving connections, HikariCP keeps retrying on demand, `Database.healthy()` (a `SELECT 1`) reports honestly, and `/api/health` shows `db:false` while the boot-style log would print `db=DOWN`. Recovery needs no restart: the next request that borrows successfully is served normally.

Q: "You hand-rolled a JSON parser. Isn't that dangerous?"

It would be, if it were ambitious — so we kept it small and boring. `util/Json` supports exactly what our envelope needs: objects, arrays, strings with escapes, numbers, booleans and null, plus the `Json.obj`/`Json.write` builders. It has no code-execution surface to begin with (it builds plain maps and lists, it never instantiates classes from input), and malformed

input throws and the caller turns that into a 400 VALIDATION_ERROR. The honest trade: we gave up convenience features we do not use, in exchange for a parser we can read line by line during the defence.

Q: "Why check the request body size before reading it?"

Because `Ctx.body()` reads the whole body into memory (`readAllBytes`), an unchecked 2 GB upload would be a denial-of-service with one request. So `Ctx` caps `MAX_BODY_BYTES` at 26 MB — a little over the 25 MB file limit — and checks the `Content-Length` header before reading a single byte, rejecting with `413 FILE_TOO_LARGE` via `AppException.tooLarge`. The cheap header check runs first; the expensive read only happens for bodies we are willing to hold. That ordering — refuse before you pay — is the same fail-fast instinct the config layer uses.

Q: "Who enforces the response envelope, and what if a handler forgets to answer?"

The router does, twice. Success responses go through `Ctx.ok`/`created`/`okPage`, which always wrap data as `{"ok":true,"data":…}`; every error is funneled into `Router.sendError`, which always wraps as `{"ok":false,"error":{"code","message"}}`. And if a handler finishes without responding at all, the router notices — `if (!ctx.hasResponded()) throw AppException.internal("Handler produced no response")` — so a client can never be left hanging on a silent route. The mobile client's `ApiClient` unwraps exactly this shape, which is why a screen that hits an error always finds a machine-readable `code` it can translate into friendly words.

The handshake — how my slice feeds the other five

- **If you ask me about tokens, OTPs or Brevo → that's Student 2's area.** My side of the border: `Middleware.authenticate` is where every JWT is verified and where `401 TOKEN_EXPIRED` is born — I verify the badge; Student 2 decides what badges exist, how they rotate, and what the OTP engine mails. My rate limiter's 20/min `auth` scope is what stands between code-guessers and their OTP engine.
- **If you ask me about encryption, share guards or verification → that's Student 3's area.** My side: their entire business layer rides on my platform — the atomic share-guard `UPDATE` runs inside my `Tx.with` on my HikariCP pool, and their "zero rows moved" reasoning depends on my single-connection transaction rule. The Caffeine read cache they use is a platform library I brought in.
- **If you ask me about screens, presenters or the API client → that's Student 4's area.** My side: the envelope and status codes their `ApiClient` unwraps are produced by my `Ctx` and `Router`, and the `401 TOKEN_EXPIRED` distinction is the hook their single-flight refresh hangs on. I also own the shell they live in: the `SCREEN_REGISTRY`, `RootNavigator` role stacks and the Dashboard screen.

- **If you ask me about audit findings or the fix comments → that's Student 5's area.** My side: several of their findings are fixed *in* my files — C-1 (the one-connection transaction hygiene in `Database`/`Tx`), C-2/C-13 (the migration splitter that survives `;` inside comments), E-3 (the weak-secret refusal in `Config`) and D-1 (the 32-byte vault-key format). Their IDs are cited in those files to this day.
- **If you ask me about builds, configuration, seeding or deployment → that's Student 6's area.** My side: I own *what* the boot does — wire the pool, apply the migrations, register the routes; Student 6 owns *how it is launched, configured and monitored*. Their boot log literally prints my numbers: `migrations=12, routes=74`.
- **Who most often bridges TO me:** Student 5 — audit findings kept landing in my layer, so their stories start in my files; and Student 3, whose services are the heaviest daily users of my transactions, pool and query mappers. The bridge map in DEFENCE_GUIDE.md routes every schema, pool and router-matching question to this desk.

Student 2 — Auth & OTP Security

This file is the complete rehearsal brief for the team member presenting **Student 2 — Auth & OTP Security** at the DocuTrust oral defence (Group 3), and the companion read for examiners who want to check every claim against the repository.

It expands the matching role section in **`DEFENCE_GUIDE.md`** at the repository root — the master index that carries the shared numbers, the individual defence formula and the bridge map — into a full file-by-file ownership map, a method-by-method code walkthrough, a plain-words glossary, and a personal question bank.

Read your role section in `DEFENCE_GUIDE.md` first, then rehearse from here; `docs/defence/STUDENT_PACK.md` in the same folder is the one-page summary of all six roles, and every class, method, column and screen named below exists in the repo and can be opened live during the defence.

My slice in one page

Role in one line: how a stranger becomes a trusted user — and how a stolen password alone still is not enough.

Owns: `AuthService`, `PasswordService` (PBKDF2), `TokenService` (rotating refresh + reuse detection + `listSessions` + `revokeById`), `MfaService` (TOTP), `JwtService` (hand-rolled HMAC-SHA256 tokens), `OtpService` (hashed one-time codes), `MailService` (Brevo), `StepUpService`, the rate limiter (`RateLimiter` on Bucket4j), and the mobile SecureStore-backed session persistence in the client; the `AuthController` endpoints; the `email_otps` schema; and the eight auth screens on the mobile side.

Feature claims: FT-001 to FT-024 plus the post-audit security addendum FT-217 to FT-234 (`docs/FEATURES.md`).

My numbers:

- 74 API routes, printed at boot (`routes=74`, `server/src/main/java/com/docutrust/Main.java`) — the auth, refresh, OTP, step-up, MFA and Active Sessions routes among them are mine.
- 15 database tables built by 12 migrations; V10 adds the `email_otps` table, V11 adds the OTP-gating fields on `shares` (shared with Student 3).
- Every OTP flow — including the negative paths — re-driven live with manual API calls (Postman/cURL) at the defence.
- PBKDF2-HmacSHA256 at **120,000 iterations**, 16-byte per-user salt, 256-bit key, constant-time verify (`PasswordService`).
- Sessions: **15-minute** access JWT (hand-rolled HMAC-SHA256) plus a **7-day** refresh token, 43 characters, stored only as a SHA-256 hash; the device keeps the JWT pair in `expo-secure-store`, while the backend exposes `GET /api/auth/sessions` and `DELETE /api/auth/sessions/:id` for the user-facing Active Sessions screen.
- Active Sessions ledger (newest slice): `refresh_tokens` doubles as the user-facing session history — `listSessions` returns the active rows (`id`, `device`, `ip`, `createdAt`, `expiresAt`, newest first) and `revokeById` revokes one row surgically (`UPDATE … WHERE id = ? AND user_id = ?`), so the token-family cascade stays reserved for reuse attacks. On the phone the JWT pair lives in `expo-secure-store` (Keychain/Keystore); a one-time hydration upgrade moves any legacy plaintext copy into SecureStore and scrubs it from AsyncStorage.
- Email OTPs: **6 digits** from `SecureRandom`, stored as `sha256(purpose:email:code)`, single-use, **10-minute**

TTL (**24 h** for invites), **5 attempts** max, at most one live code per (purpose, target), sent through Brevo.

- Challenge tokens: MFA email challenge JWT **10 minutes** (`typ=mfa_challenge`); step-up token **5 minutes** (`typ=step_up`, header `X-Step-Up-Token`).
- Rate-limit budgets: **auth 20/min**, public-share 30/min, public-verify 20/min, default 10/min — one Bucket4j bucket per scope|IP in a Caffeine cache.
- 45 mobile screens, of which the **8 auth screens** are mine (`mobile/src/screens/`).
- The headline property: after the overhaul, **no API response ever contains a password, a reset code, or a temporary password** — the inbox is the only place a code exists.

> ***Note:*** *every claim in this brief is verifiable from the public repository alone — the finding IDs are greppable in the code comments, and every flow (happy and negative path) is demonstrated live at the defence with manual API calls (Postman/cURL).*

Elevator pitch (≈ 1 minute)

> *"I own everything that proves who you are. Passwords are PBKDF2-hashed; sessions are a 15-minute access JWT plus a rotating refresh token whose reuse is detected and kills the whole token family — B-1 in our audit. On top we just shipped an email-OTP layer: six-digit codes, SHA-256-hashed in the database, single-use, ten-minute expiry, five attempts max, sent through Brevo. They power four flows: password reset, email MFA fallback for login — the server answers HTTP 440 with a challenge — step-up verification for destructive actions like permanent deletion, and OTP-gated share links. The security headline: after the overhaul,* ***no API response ever contains a password, a reset code, or a temporary password*** *— the inbox is the only place a code exists. The proof is live: every one of those flows — including the negative paths — is walked with manual API calls (Postman/cURL) during the defence."*

Then demo: on your local machine, replay a rotated refresh token on purpose and let the audience watch the whole token family die with `409 TOKEN_REUSE`; then walk forgot-password → emailed code → reset with the server log and the `X-Dev-OTP` header in dev mode showing where the code travels.

What I own, file by file

The `auth/` package — the ID checkers at the door

File	What it does, in one sentence
server/src/main/java/com/docutrust/auth/PasswordService.java	Hashes and verifies passwords with PBKDF2-HmacSHA256 at 120,000 iterations, a fresh 16-byte salt per user and a constant-time comparison, so the database never learns a password and guessing proofs take the same time to fail.
server/src/main/java/com/docutrust/auth/JwtService.java	Hand-rolled HMAC-SHA256 JWT issuer/verifier with a 15-minute access-TTL default, typed TokenExpiredException vs TokenInvalidException, and a typ claim that separates session tokens from mfa_challenge and step_up tokens.
server/src/main/java/com/docutrust/auth/TokenService.java	Mints and rotates 43-character opaque refresh tokens stored only as SHA-256 hashes with 7-day expiry, locks the token row FOR UPDATE during rotation (B-3), and on reuse of an already-revoked token revokes the entire family in its own committed transaction before answering 409 TOKEN_REUSE (the B-1 fix).

File	What it does, in one sentence
server/src/main/java/com/docutrust/auth/MfaService.java	Implements RFC 6238 TOTP (30-second step, 6 digits, ±1 window) with RFC 4648 Base32 secrets, otpauth:// URLs for authenticator apps, and constant-time code comparison.
server/src/main/java/com/docutrust/auth/RateLimiter.java	Token-bucket rate limiting on Bucket4j — one bucket per scope

The auth services — the chefs cooking the flows

File	What it does, in one sentence
server/src/main/java/com/docutrust/service/AuthService.java	The auth domain orchestrator: register (personal or org), login with TOTP or the email-MFA fallback (useEmailMfa → 440 challenge), verifyOtp challenge exchange, refresh/logout, OTP password reset, email verification, profile updates, MFA setup/enable/disable, and the step-up-gated password change.
server/src/main/java/com/docutrust/service/OtpService.java	The shared one-time-code engine: five purposes (password_reset, mfa_login, step_up, share_access, org_invite), 6-digit SecureRandom codes stored only as sha256(purpose:email:code) digests, single-use, 10-minute TTL (24 h invites), a 5-attempt burn, and at most one live code per (purpose, target).
server/src/main/java/com/docutrust/service/MailService.java	Delivers OTP emails through Brevo's transactional API in MAIL_MODE=live (a failed send is a loud 502 MAIL_SEND_FAILED, never a silent drop), while MAIL_MODE=dev logs the code and hands it to the controller for the X-Dev-OTP demo header.
server/src/main/java/com/docutrust/service/StepUpService.java	Exchanges an emailed OTP for a 5-minute single-purpose JWT (typ=step_up) and gates destructive routes through authorize(), which throws 403 STEP_UP_REQUIRED unless the X-Step-Up-Token header carries a valid token belonging to the calling user.

The auth controllers — the waiters taking the orders

File	What it does, in one sentence
server/src/main/java/com/docutrust/controller/AuthController.java	Thin endpoints for register, login (rendering the 440 MFA_EMAIL_CHALLENGE body with the masked email and challenge token), verify-otp, refresh, logout, forgot-password and reset-password-otp, request/verify-step-up, verify-email, me, change-password and the three MFA routes — parsing, delegating, and surfacing dev OTPs only via X-Dev-OTP.
server/src/main/java/com/docutrust/controller/OrgController.java (invite endpoints, shared slice)	The invite and acceptInvite endpoints sit in the org module, but the credential they move is my engine's org_invite OTP — a 24-hour emailed code that replaces the temporary-password-in-JSON anti-pattern (FT-225).

The `email_otps` schema — migration V10

Column (table email_otps)	What it holds, in one sentence
id UUID PRIMARY KEY	The OTP row's own identity.
user_id UUID REFERENCES users(id) ON DELETE CASCADE	The account the code belongs to — deliberately NULL for share-access OTPs, which belong to an anonymous recipient instead.
share_id UUID REFERENCES shares(id) ON DELETE CASCADE	For share_access codes only: which share link this code unlocks.
recipient_email VARCHAR(255)	The lowercased inbox the code was minted for — part of the digest, so the code is nailed to one address.
purpose VARCHAR(32) NOT NULL	Which flow the code serves (password_reset, mfa_login, step_up, share_access, org_invite) — the first ingredient of the purpose-salted digest.
otp_hash VARCHAR(64) NOT NULL	Only the SHA-256 hex digest of purpose:email:code — a database leak yields no working codes.
expires_at TIMESTAMPTZ NOT NULL	The 10-minute life (24 hours for invites) enforced before any comparison happens.
used_at TIMESTAMPTZ	The single-use marker — set inside the same locked transaction that verified the code, so two parallel guesses cannot both spend it.
attempts INT NOT NULL DEFAULT 0	The burn counter — the fifth wrong attempt marks the code used for good.

Column (table email_otps)	What it holds, in one sentence
created_at TIMESTAMPTZ NOT NULL DEFAULT now()	Mint time, used to pick the newest live code for the target.

Three indexes back the lookup paths: `idx_email_otps_target` (purpose, recipient_email, created_at DESC), `idx_email_otps_user` and `idx_email_otps_share` — added by the same V10 migration.

The eight auth screens — `mobile/src/screens/`

Screen file	What it does, in one sentence
LoginScreen.js	Email + password sign-in that offers "email me a code instead" (useEmailMfa) and finishes the server's 440 challenge instead of treating it as an error.
RegisterScreen.js	Creates personal or organisation accounts with a client-side password-strength meter before the server's stricter validation runs.
ForgotPasswordScreen.js	Requests the emailed reset code and shows the same confirmation whether or not the account exists — no user enumeration.
ResetPasswordScreen.js	Collects the emailed 6-digit code plus the new password and submits reset-password-otp {email, code, newPassword} (FT-219).
MfaScreen.js	The second-factor screen: TOTP code entry for authenticator users, the email-fallback phase for the 440 challenge, and the exchange of challenge + code for real session tokens.
VerifyEmailScreen.js	Consumes the emailed verification token and flips a pending account to active.
ChangePasswordScreen.js	The step-up-gated password change: it detects 403 STEP_UP_REQUIRED, requests the emailed code, trades it for the 5-minute step-up token and retries with the X-Step-Up-Token header.
SecurityScreen.js	The "where am I signed in" view — the server-backed Active Sessions list (GET /api/auth/sessions, revoked per row via DELETE /api/auth/sessions/:id), MFA state, and the change-password entry point.

Five layman's analogies

1. **An OTP is a VIP wristband, not a key.**

Your password is the key to the building; the wristband is handed to you at the door, works for one night only, and is worthless to anyone who photographed it yesterday.

Even if someone steals your key (password), without the wristband handed out to your phone's inbox they still don't get into tonight's event.

2. **Step-up authentication is a bank vault inside a bank.**

Walking into the bank with your card (being logged in) gets you to the lobby.

But opening the safe-deposit box — permanently deleting a document, changing your password — requires a second, fresh proof: a code to your phone, right now.

We even made the vault token short-lived: five minutes, single purpose, useless on any other door.

3. **Refresh-token rotation is a tear-off ticket pad.**

Every time you use your ticket, you get the next one off the pad and the old one is stamped.

If someone tries to use a stamped ticket — a stolen, already-rotated refresh token — the system doesn't just refuse it: it tears up the whole pad, because a stamped ticket reappearing means someone is copying your tickets.

4. **PBKDF2 password hashing is a one-way blender run 120,000 times.**

Fruit goes into the blender and a smoothie comes out; no machine pours a smoothie back into whole fruit.

We run the blender 120,000 times so that anyone trying to recover the original by re-blending every possible basket has to do the same enormous chore — and every user's basket gets its own unique pinch of spice (the salt), so two people with the identical password still produce two different smoothies on the shelf.

5. **Purpose-salting an OTP digest is stamping the key with its door.**

A spare key cut for the garden shed does not open the front door, even though the teeth are identical metal — the stamp is part of what the lock checks.

`sha256(purpose:email:code)` means a code emailed for a password reset physically cannot be replayed into the share-access or step-up locks: same six digits, different stamp, no fit.

The critical code walkthrough

Flow (a) — password reset, method by method in order

1. **Route + rate limit.** `POST /api/auth/forgot-password` is registered in `Main.java` through `router.rateLimited(..., "auth", authC.forgotPassword())`, so before any of my code runs, the router asks `RateLimiter.allow(key)` — a Bucket4j token bucket keyed `auth|<ip>` in a Caffeine cache.

Over the budget the caller gets `429 RATE_LIMITED`.

2. **The controller.** `AuthController.forgotPassword()` calls `AuthService.forgotPassword(ctx.json(), ctx.ip())` and, in dev mode only, attaches the code as the `X-Dev-OTP` response header — the body never carries it.
3. **The service, inside one transaction.**

`AuthService.forgotPassword` validates the email with `Validator.email`, then opens `db.tx(...)` (`Database.tx` → `Tx.with`: autocommit off, commit on success, rollback on any exception).

It looks the account up by email; if there is no user it returns a `null` dev code and the same body either way — `"If that email exists, a reset code has been sent"` — so the endpoint cannot be used to enumerate accounts.

4. **The mint.** `OtpService.issue(c, "password_reset", userId, null, email)` first `DELETE`s the target's unused codes for that purpose, so at most one live OTP exists per (purpose, target).

It then mints `String.format("%06d", new SecureRandom().nextInt(1_000_000))` and inserts the row with `digest(purpose, email, code)` — `Text.sha256Hex("password_reset:" + email + ":" + code)` — and `expires_at = now() + 10 minutes` (`TTL_MINUTES = 10`).

The plaintext code exists nowhere in the database.

5. **The audit row.** Still inside the same transaction, `AuditService.insert(... "auth.password_reset_requested" ...)` writes the append-only audit entry — the whole mint is atomic.
6. **The email, after the commit.** `mail.sendOtp(email, purpose, code, 10)` runs only after the transaction has committed — no network call ever happens inside a transaction.

In `MAIL_MODE=live`, `MailService.send` POSTs a JSON payload to `https://api.brevo.com/v3/smtp/email` with the `api-key` header and turns any non-2xx into `AppException(502, "MAIL_SEND_FAILED", ...)` — a silently dropped OTP would lock users out.

In dev mode the code is printed to the log instead.

7. **The verification.** `POST /api/auth/reset-password-otp` → `AuthService.resetPasswordOtp` → inside `db.tx`, `OtpService.verify(c, "password_reset", userId, null, email, code)`.

That method selects the newest live row `... ORDER BY created_at DESC LIMIT 1 FOR UPDATE` — the row lock means a second simultaneous guess waits, then sees the code spent.

It checks the `\d{6}` format and the expiry, then compares `Text.constEq(digest(purpose, email, code), row.hash)` — constant-time, so timing leaks nothing.

A wrong code increments `attempts`; at `MAX_ATTEMPTS = 5` the same UPDATE sets `used_at`, burning the code.

A right code is spent by `UPDATE email_otps SET used_at = now() WHERE id = ? AND used_at IS NULL` — if that updates zero rows we lost a single-use race and fail closed with `400 OTP_INVALID`.

Expired, used and unknown codes all return the identical message, so the reply never reveals which.

8. **The token update, same transaction.**

Still inside the same `db.tx`: `UPDATE users SET password_hash = ?` with a fresh `PasswordService.hash(newPassword)` (new salt, 120,000 iterations), then `tokens.revokeAllForUser(c, userId)` — every refresh token of that user dies with the password — then the `auth.password_reset` audit row.

If any step fails, all of it rolls back.

Flow (b) — refresh-token rotation with reuse detection (the B-1 fix), method by method in order

1. **Route.** `POST /api/auth/refresh` → `AuthService.refresh` → `TokenService.rotate(presented, device, ip)`.
2. **Format gate.** `TokenService.validFormat` requires exactly 43 characters of `[A-Za-z0-9_\-]`; anything else is `401 Invalid refresh token` before the database is touched.
3. **Hash lookup under a row lock.** Inside `db.tx`, the presented token is hashed with `TokenService.hash` (`Text.sha256Hex`) — the database stores hashes only — and selected with `SELECT rt.user_id, rt.expires_at, rt.revoked_at, u.role, u.org_id FROM refresh_tokens rt JOIN users u ON u.id = rt.user_id WHERE rt.token_hash = ? FOR UPDATE OF rt`.

The `FOR UPDATE` is B-3: a concurrent rotate or revoke of the same token serialises here instead of both reading `revoked_at IS NULL` and double-issuing.

4. **Not found** → `401`. **Found and revoked** → the method does *not* throw; it returns `new Rotation(userId, null)` — the B-1 marker.

Found and expired → `401 Refresh token expired`.

5. **The healthy rotation.** A fresh 43-character token is minted (`Text.token()` — 32 random bytes, Base64url), inserted with a 7-day expiry, and the old row is closed with `UPDATE refresh_tokens SET revoked_at = now(), replaced_by = ? WHERE`

`token_hash = ? AND revoked_at IS NULL` (the `AND revoked_at IS NULL` is B-3: never re-revoke).

The transaction commits and the caller receives a new 15-minute access JWT plus the fresh refresh token.

6. **The reuse branch — after the detection transaction has fully committed.**

The heart of the B-1 fix is this block in `TokenService.rotate`:

```
if (r.tokens() == null) {
    // B-1: the family revocation must COMMIT before the 409 unwinds.
The old code revoked inside
    // the detection transaction - throwing the 409 rolled that
revocation straight back, so reuse
    // detection had no durable effect (fail-open).
    revokeAllForUser(r.userId());
    // B-2: audit AFTER the revocation commit - best-effort, so an
audit outage can never undo it.
    auditReuse(r.userId(), ip);
    throw new AppException(409, "TOKEN_REUSE", "Refresh token reuse
detected - all sessions revoked, please log in again");
}
```

`revokeAllForUser(userId)` runs in **its own committed transaction** — `UPDATE refresh_tokens SET revoked_at = now() WHERE user_id = ? AND revoked_at IS NULL` — so by the time the `409` is thrown, the revocation is durable no matter what happens next.

`auditReuse` then writes the `auth.token_reuse` audit row best-effort in yet another transaction and swallows failures: the trail may lag, never lie, and can never undo the revocation.

The old code failed open — the revocation shared a transaction with the exception that reported the reuse, so both rolled back together.

The story, the finding ID and the fix are all readable in the comments around `TokenService.rotate`.

7. **The client's end.** The mobile `ApiClient` (Student 4) receives `409 TOKEN_REUSE`, logs the session out immediately, and the legitimate user simply logs in again — annoying for them, fatal for the attacker.

Flow (c) — the email-MFA fallback login (the 440), method by method in order

1. **The request.** `POST /api/auth/login` with `useEmailMfa: true` (rate-limited in the `auth` scope like every public auth route).
2. **Credentials first, always.** `AuthService.login` loads the credential row with `loadCred` and runs `passwords.verify`; a failure writes the `auth.login_failed` audit row and answers the identical `401 Invalid email or password`; a suspended account answers `403 ACCOUNT_SUSPENDED` before any factor is discussed.

3. **The fallback branch.** If `mfaEnabled` and `useEmailMfa` are both set, one `db.tx` calls `otps.issue(c, OtpService.PURPOSE_MFA_LOGIN, userId, null, cu.user.email)` and writes the `auth.mfa_email_challenge` audit row; only after the commit does `mail.sendOtp` send the code to the account's own address.
4. **The challenge token.** `jwt.issue(userId, role, orgId, OtpService.TTL_MINUTES * 60, "mfa_challenge")` mints a 10-minute JWT whose `typ` claim gives it no session authority — it only proves "password plus email factor done".
5. **The 440.** `AuthController.login` answers `440` with `error.code` `MFA_EMAIL_CHALLENGE`, the `challenge` token and the masked address from `maskEmail` (`a***@domain.tld` — recognisable to the owner, useless for enumeration).
6. **The exchange.** `POST /api/auth/verify-otp {challenge, code}` → `AuthService.verifyOtp`: `jwt.verify` (expired → `401 CHALLENGE_EXPIRED`, wrong `typ` → `401`), a suspended-account re-check, then `otps.verify` for `mfa_login` inside one transaction — after which `last_login_at` is stamped, the `auth.login` audit row is written with `"mfa": "email_otp"`, and `tokens.issue` finally grants the real session.
7. **Why this shape.** The TOTP path (`mfa.verify`) and the email path share the same login endpoint and the same OTP engine; the only difference is who generates the second factor — an authenticator app or an inbox.

Flow (d) — the step-up-gated password change, method by method in order

1. **The request.** `POST /api/auth/change-password` with the current and new password plus the `X-Step-Up-Token` header (`StepUpService.HEADER`).
2. **The gate, before anything else.** `AuthService.changePassword` calls `stepUps.authorize(stepUpToken, me)` — a missing header, an expired token, an invalid signature, a wrong `typ`, or a token belonging to another account each answers `403 STEP_UP_REQUIRED` with the exact remedy spelled out in the message.

The comment in `AuthService` records the reasoning: changing a password is exactly the moment an attacker with a stolen session would strike.

3. **The current-password proof.** `passwords.verify(current, stored)` guards against a hijacked step-up too; a failure writes the `auth.password_change_failed` audit row.
4. **The swap, one transaction.** Inside `db.tx`: `UPDATE users SET password_hash = ?` with a fresh PBKDF2 hash, then `tokens.revokeAllForUser(c, userId)` — every session dies with the password — then the `auth.password_changed` audit row.

A stolen session that somehow passed step-up still ends with the thief's own tokens revoked.

The five OTP purposes at a glance

Purpose (OtpService constant)	Triggered by	Delivered to	Consumed by
password_reset	POST /api/auth/forgot-password	the account's own inbox	POST /api/auth/reset-password-otp
mfa_login	POST /api/auth/login with useEmailMfa: true → 440	the account's own inbox	POST /api/auth/verify-otp + the challenge JWT
step_up	POST /api/auth/request-step-up	the account's own inbox	POST /api/auth/verify-step-up → the X-Step-Up-Token header
share_access	POST /api/public/shares/:token/request-otp	the owner-pinned otp_recipient_email only	POST /api/public/shares/:token/open-with-otp
org_invite	POST /api/org/invite (24-hour TTL)	the invitee's inbox	POST /api/auth/accept-invite

What my status codes tell the client

Code my slice emits	When	What the client should do
440 MFA_REQUIRED	TOTP-enabled login without a valid authenticator code	show the TOTP entry — or offer the email fallback
440 MFA_EMAIL_CHALLENGE	useEmailMfa: true accepted	keep the challenge, prompt for the emailed code, POST both to /api/auth/verify-otp
403 STEP_UP_REQUIRED	destructive action without a valid X-Step-Up-Token	request-step-up → verify-step-up → retry with the header
409 TOKEN_REUSE	an already-rotated refresh token was presented	log the session out immediately — the family is already revoked
429 RATE_LIMITED	the scope's token bucket ran dry	wait; the buckets refill continuously, not in fixed windows

Code my slice emits	When	What the client should do
502 MAIL_SEND_FAILED	Brevo rejected or failed the send (live mode only)	offer a retry — a silently dropped code would lock users out

Every lifetime in my slice, at a glance

Credential	Lifetime	Where it is enforced
Access JWT	15 minutes	JwtService.ACCESS_TTL_SECONDS, verified on every authenticated route
Refresh token	7 days	refresh_tokens.expires_at, set on issue and on every rotation
Email OTP	10 minutes	OtpService.TTL_MINUTES, checked before any digest comparison
Org-invite OTP	24 hours	OtpService.INVITE_TTL_MINUTES
MFA email challenge	10 minutes	the mfa_challenge JWT expiry, re-checked in AuthService.verifyOtp
Step-up token	5 minutes	StepUpService.TTL_SECONDS, re-checked by authorize at use time
Rate-limit bucket	continuous refill over 60 s	Bucket4j Refill.greedy in RateLimiter.allow

My domain, in plain words (glossary)

Term	Plain meaning, with an everyday analogy
OTP (one-time passcode)	A VIP wristband: handed to you at the door, works once, worthless tomorrow. Six digits, emailed, dead in 10 minutes.

Term	Plain meaning, with an everyday analogy
PBKDF2 + salt	A recipe that stirs the password soup 120,000 times before serving — and every user's soup gets its own unique pinch of spice (the salt), so identical passwords still store differently and pre-computed "rainbow" recipe books become useless.
SHA-256 digest	A fingerprint of a message: any change in the input changes the fingerprint, and you cannot rebuild the message from it. We store fingerprints, never the secrets themselves.
Purpose-salted digest	The fingerprint taken over purpose:email:code instead of just the code — a key stamped with which door it fits, so a reset code can never open the share lock.
JWT (JSON Web Token)	A wax-sealed ID card: the server writes your details on it, stamps the seal with its secret key, and anyone can read the card but only the server can forge the seal.
Refresh-token rotation	The tear-off ticket pad: every use hands you the next ticket and stamps the old one.
Reuse detection / family revocation	A stamped ticket coming back means someone is copying your tickets — so the whole pad is torn up (409 TOKEN_REUSE), not just refused.
TOTP	The numbered key fob from online banking: a new 6-digit number every 30 seconds, computed from a shared secret plus the clock (RFC 6238). Our MfaService implements it by hand.
Step-up verification	A bank vault inside a bank: being logged in gets you to the lobby, but the safe-deposit box needs a fresh, second proof — a code to your inbox, right now.
typ claim	The job title printed on the ID card: mfa_challenge (10 min) and step_up (5 min) look alike but authorise opposite directions, and each gate checks the title.
Rate limiter (token bucket)	A coffee-shop punch card that fills itself: every auth request stamps one square; when the minute's squares are gone, the machine politely says come back in sixty seconds — humans barely notice, rapid-fire tools hit a wall.
User enumeration	The leak where "wrong password" and "no such account" sound different, letting an attacker map which emails exist. Every public auth reply here is deliberately identical either way.
Constant-time compare	A guard who always takes exactly the same time to say no, whether the first character or the last was wrong — so the stopwatch itself leaks nothing (Text.constEq).

Term	Plain meaning, with an everyday analogy
Brevo / MAIL_MODE	The postman, plus a flight-simulator switch: dev mode prints the code to the log and the X-Dev-OTP header so we can rehearse without emailing anyone; live mode sends real email.
HTTP 440	Our own rubber stamp, documented in the API contract: "second factor needed — check your email." The standard codes have no word for that situation, so we printed one.

My ultimate Q&A bank

From the per-student pack (the classics)

Q: "Why store refresh tokens hashed if they are already random?"

Because the database should not be able to give a thief a usable token. A hash means a full database leak yields nothing usable directly; the attacker would have to reverse a SHA-256 of a 43-character random string. It costs one hash per refresh and buys immunity to the most damaging dump scenario.

Q: "Walk me through exactly what happens when a rotated token is replayed."

The presented token is hashed and looked up; the row is found but already revoked, which by definition only happens if something replayed it. The server answers 409 TOKEN_REUSE and revokes every refresh token belonging to that user, so whatever device holds the stolen one is locked out with it. The legitimate user simply logs in again — annoying for them, fatal for the attacker.

Q: "What exactly changed in the OTP overhaul?"

Two anti-patterns died: the password-reset endpoint that returned the reset code in the response, and invites that carried a temporary password in JSON. Their replacements share one engine: six-digit codes from SecureRandom, stored only as sha256 of purpose, target and code, single-use, ten-minute TTL — 24 hours for invites — five wrong attempts burn the code, and one live code per purpose and target. Responses never contain codes in any mode; dev mode puts them in the log and an X-Dev-OTP header, live mode sends real Brevo email. Every flow, including the negative paths, is re-driven live with manual API calls at the defence.

Q: "Why did you return a special 440 status for MFA instead of a normal 401?"

The client needs to distinguish two very different situations after a login attempt: wrong credentials, versus correct credentials awaiting a second factor. A distinct machine-readable status makes the MFA screen a routing decision rather than an error message,

and it cannot be confused with a failed password. It is a deliberate, documented extension of the standard statuses, used only for that branch.

Q: "Could an attacker brute-force the six-digit code?"

They get ten attempts per minute per IP from the shared rate limiter, and each attempt must land inside the code's ten-minute life with a five-attempt burn limit, so the search space is practically unreachable. The limiter plus the short window make guessing a losing game, and failed attempts are audited.

The masterclass ten

Q: "Why `SecureRandom` and not the ordinary `Random`?"

Java's ordinary `Random` *is a small, fully predictable mathematical sequence seeded from the clock — if an observer sees a couple of outputs, they can reconstruct the seed and predict every future output, including everyone else's codes.* `SecureRandom` *draws from the operating system's entropy pool and gives no such foothold. The whole security of a six-digit code rests on the mint being unpredictable: one million combinations is only safe if every one of them is equally likely and independent of everything an attacker has seen. Both* `OtpService.issue` *and* `PasswordService.hash` *use* `SecureRandom` *for exactly that reason — it is a one-word change that is the difference between a lottery and a pattern.*

Q: "Two guesses arrive at the same millisecond. What stops them both spending one code?"

The verification runs inside `Database.tx`*, and* `OtpService.verify` *selects the live row with* `FOR UPDATE` *— Postgres row locking means the first verifier holds the row and the second one waits. When the second proceeds, it re-reads the state the first committed: attempts incremented, or* `used_at` *already set, so it fails. And there is a second fence: the consumption itself is* `UPDATE email_otps SET used_at = now() WHERE id = ? AND used_at IS NULL`*, and if that statement updates zero rows — meaning someone else spent it between the read and the write — we throw* `OTP_INVALID` *and fail closed. Two independent mechanisms, the lock and the conditional update, would each have to fail for a double-spend to happen, and the transaction wraps both.*

Q: "The MFA challenge and the step-up token are both JWTs — why give them different `typ` claims?"

Because they authorise opposite directions and must never be interchangeable. The `mfa_challenge` *token (10 minutes) says "this person finished password plus second factor" and is only accepted by* `AuthService.verifyOtp`*, which exchanges it for real session tokens. The* `step_up` *token (5 minutes) says the reverse — you already have a session, and this briefly certifies that you proved an OTP moments ago — and is only*

accepted by `StepUpService.authorize` *on destructive routes. Each gate verifies the* `typ` *field and, for step-up, that the token's subject is the calling user: a challenge token cannot open the vault gate, and a step-up token cannot finish a login. One* `JwtService`*, one signing key, and the authority is carried in the claim — misusing a token fails the type check, not the signature.*

Q: "Why are the codes purpose-salted in the digest?"

The stored digest is `sha256(purpose:email:code)`*, not* `sha256(code)`*, so a code is welded to its flow and its inbox. If someone captures a code from a share-access email and tries it against the reset endpoint, the reset lookup computes a different digest and nothing matches — the same six digits are simply a different key on a different door. It also lets one table, one index layout and one verification path serve five flows safely, which is what made the overhaul affordable. And because the email is inside the digest, a code that was legitimately issued to one address cannot be verified against another address even within the same flow.*

Q: "What actually happens when Brevo is down?"

`MailService.send` *treats a provider failure as a loud, immediate* `AppException(502, "MAIL_SEND_FAILED", ...)` *— never a silent drop, because a user who never receives their code is locked out just as surely as if we refused it. The client translates that into a friendly retry message. The design point is ordering: the email is sent after the minting transaction has committed, so a mail failure cannot roll back the audit row or leave a half-minted state, and the next attempt simply issues a fresh code —* `issue` *deletes the target's unused codes first, so there is never a pile of competing live codes. In dev mode none of this is reachable, because the network call is replaced by a log line.*

Q: "Why do responses never contain codes — even in dev mode?"

Because the old pattern trained clients to render credentials: a code that appears in a response body ends up in log files, screenshots, HTTP proxies and browser dev tools, all of which outlive the ten-minute TTL in practice. So the overhaul deleted the anti-pattern rather than deprecating it: `AuthService.forgotPassword` *returns the code only through a side channel the controller controls, and in* `MAIL_MODE=dev` *that side channel is the* `X-Dev-OTP` *response header plus the server log — the body still says only "If that email exists, a reset code has been sent". Live mode has no header at all: the inbox is the only place the code exists. Our live demo reads the header, which is how we exercise the full flows without ever teaching the API to leak.*

Q: "You hash passwords with PBKDF2 but refresh tokens and OTPs with plain SHA-256. Inconsistent?"

Deliberate. PBKDF2's 120,000 iterations exist to slow down guessing of low-entropy secrets — human passwords, where the attacker's job is trying dictionary words. A

refresh token is 32 random bytes encoded to 43 characters: there is nothing to dictionary-search, so a single fast SHA-256 already forces an attacker who dumped the database to brute-force 2-to-the-256 possibilities, which is no attack at all. The OTP codes are low-entropy, but their protection is not hash cost — it is the 10-minute expiry, the five-attempt burn, the row-locked single-use spend and the rate limiter, which together bound an attacker to a handful of guesses. Matching the tool to the threat, rather than applying the strongest primitive everywhere, is the actual lesson.

Q: "The limiter used to answer 429 during your own rehearsal runs. What did you change?"

That was the audit's most useful false alarm: machine-speed repeated calls from one IP legitimately exhausted the old one-size 10-per-minute auth bucket, and a real user doing forgot-password, MFA and step-up inside a minute would come close too. The honest fix was not weakening the limiter but making it scope-aware: `RateLimiter` *now keeps per-scope budgets — auth 20/min, public-share 30/min, public-verify 20/min, default 10/min — as one Bucket4j bucket per* `scope|IP` *key in a Caffeine cache that evicts idle keys after ten minutes. Bucket4j's token buckets also smooth bursts instead of permitting a full window's worth at once. The limiter stayed exactly as strict where it matters; it just stopped punishing legitimate multi-step flows.*

Q: "An attacker steals my* access *token. What is the actual blast radius?"

Fifteen minutes, by construction — that is `JwtService.ACCESS_TTL_SECONDS`*, and there is no way to extend it without the refresh token. Within that window the bearer can act as me on ordinary routes, but the dangerous doors are additionally locked: changing the password and permanently deleting documents demand a fresh step-up OTP through* `StepUpService.authorize`*, which a stolen bearer token cannot produce, and a password change revokes every session anyway. The attacker also cannot rotate the access token, because rotation requires the refresh token — whose reuse would trigger the family revocation. So the design assumes the access token leaks eventually and minimises what fifteen minutes of it can do.*

Q: "Registration returns a `verifyToken` in the response — isn't that the anti-pattern you killed?"

It is the one deliberate survivor, and the difference is what the string unlocks. The killed anti-patterns returned account credentials: a reset code that unlocks the password, or a temporary password that is the login. The registration `verifyToken` *does neither — it is a 43-character opaque token, stored hashed in* `password_resets` *via* `issueOneTimeToken`*, valid 24 hours, single-use, and the only thing it can do is flip the caller's own* `email_verified` *flag through* `AuthService.verifyEmail`*. It is handed to the person who just created the account, in the same session that created it, so there is*

no credential being exposed to a stranger. We named it in the review anyway, and the roadmap item is to move that last token fully into the email like every other flow.

The handshake — how my slice feeds the other five

- **If you ask me about the router, middleware chain, or the schema → that's Student 1's area.**

Here's how my part feeds it: my routes are registered in their `Router` with per-scope rate-limit wrappers, my JWT check runs as one link in their `Middleware` chain before any controller, and my `email_otps` and `refresh_tokens` tables live in their migrations V10 and V01 — I inherited constraints and indexes I did not have to build.

- **If you ask me about documents, shares or the vault → that's Student 3's area.**

Here's how my part feeds it: my `OtpService` is the engine behind the OTP-gated share links (purpose `share_access`, the V11 fields on `shares`), and Student 3's permanent-delete route calls my `StepUpService.authorize` before a single byte is touched — the vault cannot be opened destructively without my second factor.

- **If you ask me about screens, presenters or the API client → that's Student 4's area.**

Here's how my part feeds it: my `440 MFA_EMAIL_CHALLENGE`, `403 STEP_UP_REQUIRED` and `409 TOKEN_REUSE` codes are the routing decisions the eight auth screens and the `ApiClient` are built on — the client never guesses when to demand a second factor, my status codes tell it.

- **If you ask me about audit findings or the fix comments → that's Student 5's area.**

Here's how my part feeds it: the B-1, B-2 and B-3 findings live in my `TokenService.rotate` and are the reason the revocation commits first — their IDs are cited in my files' comments, greppable during the defence.

- **If you ask me about builds, configuration or deployment → that's Student 6's area.**

Here's how my part feeds it: my `MailService` reads the `BREVO_*` variables and the `MAIL_MODE` switch from their fail-fast `Config`, and the dev-mode `X-Dev-OTP` header is what makes the live demo possible without sending real email.

- **Who most often bridges to me: Student 4.**

Every question about the login, MFA, reset, step-up or security screens ends in one of my mechanisms — the client owns the UX of the handshake, I own the handshake.

Student 3 — Business Logic (Documents, Sharing, Verification)

This file is the complete rehearsal brief for the team member presenting **Student 3 — Business Logic: Documents, Sharing, Verification** at the DocuTrust oral defence (Group 3), and the companion read for examiners who want to check every claim against the repository.

It expands the matching role section in **`DEFENCE_GUIDE.md`** at the repository root — the master index that carries the shared numbers, the individual defence formula and the bridge map — into a full file-by-file ownership map, a method-by-method code walkthrough, a plain-words glossary, and a personal question bank.

Read your role section in `DEFENCE_GUIDE.md` first, then rehearse from here; `docs/defence/STUDENT_PACK.md` in the same folder is the one-page summary of all six roles, and every class, method, column and endpoint named below exists in the repo and can be opened live during the defence.

My slice in one page

Role in one line: the bytes and the doors — encrypted storage, versioned documents, and the share engine that faces strangers.

Owns: `DocumentService` (vault, versions, trash, the Caffeine read cache), `FileVault` (AES-256-GCM encryption at rest), `ShareService` (atomic guards, passcodes, OTP gating, download tickets), `VerifyService` (proof links pinned to versions); the `DocumentController`, `ShareController`, `PublicController` and `VerificationController` endpoints; the `documents` / `document_versions` / `document_history` / `shares` schema; and the twelve vault screens on the mobile side.

Feature claims: FT-025 to FT-070 (Documents & Vault), FT-071 to FT-096 (Sharing & Permissions), FT-209 to FT-212 (data integrity), plus the Phase-5 OTP-gating features FT-223 and FT-224 from the security addendum (`docs/FEATURES.md`).

My numbers:

- 74 API routes, printed at boot (`routes=74`, `server/src/main/java/com/docutrust/Main.java`) — the 13 document routes, the 6 owner share routes, the 5 public share/download routes, the 5 verification routes and the public verify page are mine.
- 15 database tables built by 12 migrations; V02 builds `documents`, V03 builds `document_versions` / `document_history` / `shares`, V09 pins verification to versions, V11 adds the OTP-gating fields on `shares`.
- The share and tamper demos run live: API calls flip a share through open, revoke and 409, and the two-byte tamper flip turns a verification verdict to `tampered` in front of the audience.
- 45 mobile screens, of which the **12 vault screens** are mine.

- Uploads: **25 MB** cap (`413` beyond it), 6 allowed MIME types (`415` otherwise), 10 document types, sensitive types `medical` and `financial` forced into admin review.
- Encryption at rest: **AES-256-GCM**, random 12-byte IV per file, 128-bit authentication tag, 8 KB streaming chunks, key from the environment (`VAULT_KEY`, never in code); **SHA-256 computed while streaming** and re-verified on every download.
- Sharing: 43-character link tokens, expiry up to 8760 hours, optional max views and passcode, instant revocation, **single-use 5-minute download tickets**, and optional OTP gating pinned to one recipient inbox (`401 SHARE_OTP_REQUIRED` until verified).
- Verification: verdicts `valid / invalid / tampered / expired / revoked`, each proof pinned to one document version (V09 / F-4) with a `superseded` flag when a newer version exists; no bytes ever leave the vault.
- Cache: a 10-second, 1,000-entry Caffeine read-through cache on ownership-scoped document rows, invalidated explicitly on every write path (INFRA-2 / FT-228).

  Note: *every claim in this brief is verifiable from the public repository alone — the finding IDs are greppable in the code comments, and every guard and verdict is demonstrated live at the defence with manual API calls (Postman/cURL).*

Elevator pitch (≈ 1 minute)

> *"I own the actual product: documents and the promises we make about them. Every file is encrypted with AES-256-GCM before it touches disk and fingerprinted with SHA-256; every download re-verifies that fingerprint, so tampering announces itself. Sharing is governed by atomic database guards — expiry, view counts, revocation, passcodes, and now optionally an emailed OTP pinned to one recipient's inbox: the link answers 401 until that inbox proves control of a code. Verification links don't just say 'valid' — since V09 each proof is pinned to an exact document version, so the verdict is cryptographically bound to the bytes that existed at mint time. And the invite flow no longer emails temporary passwords: the invitee claims their account with an emailed OTP and chooses their own password. The proof: live API calls flip a share through open, revoke and 409, and the tamper demo turns a verification verdict to* `tampered` *on screen."*

Then demo: on your local machine, flip two bytes inside a file in the vault directory and refresh the verification link — the badge flips from `valid` to `tampered` in front of the audience; then create a 30-second share, open it twice, and let the third open answer `409 SHARE_MAX_VIEWS` while the whole audience watches the link die.

What I own, file by file

The services and the vault — the chefs and the safe

File	What it does, in one sentence
server/src/main/java/com/docutrust/service/DocumentService.java	The vault domain: multipart upload with streaming SHA-256 and AES-GCM encryption, ownership-scoped list/search/filter/sort with pagination, metadata patch, soft delete and restore, step-up-gated permanent delete with a tombstone fallback, version upload at N+1 under a UNIQUE guard, integrity-checked downloads of any version, the append-only history timeline, the expiring-soon view, and a 10-second Caffeine read cache that every write invalidates.
server/src/main/java/com/docutrust/vault/FileVault.java	The encryption-at-rest engine: every file is AES-256-GCM encrypted with a fresh 12-byte IV written as a ciphertext prefix and a 128-bit tag appended, encrypted and SHA-256-fingerprinted in the same 8 KB streaming pass, and decrypted only after the tag verifies — a mismatch throws TamperedException and the document answers "tampered" instead of serving bytes.
server/src/main/java/com/docutrust/service/ShareService.java	The share engine: owner CRUD for user-to-user and public link shares with PBKDF2-hashed passcodes, max views and instant revocation; the public open path runs one atomic guard UPDATE that increments the view count only if the link is unrevoked, unexpired and under its limit; OTP-gated links (Phase 5) verify an emailed code before opening; download permission mints single-use 5-minute tickets held in a ConcurrentHashMap and removed on first redemption.
server/src/main/java/com/docutrust/service/VerifyService.java	The proof engine: owner-minted verification requests pinned at creation to the exact current version (F-4, V09), a two-step approve/reject lifecycle, and a public verify that decrypts the pinned version inside the vault, recomputes its SHA-256, and renders valid / invalid / tampered / expired / revoked plus a superseded flag as JSON or an SEO HTML page — no bytes leave the vault.

The controllers — the waiters taking the orders

File	What it does, in one sentence
server/src/main/java/com/docutrust/controller/DocumentController.java	The 13 document endpoints: multipart upload, list, detail, patch, soft delete, restore, step-up-gated permanent delete, add version, list versions, download current or a specific version, history, and the expiring horizon query.
server/src/main/java/com/docutrust/controller/ShareController.java	The 6 owner share endpoints: my shares, create (user share XOR link share, with optional passcode / max views / OTP gating), revoke, delete, per-share stats with history, and the incoming shares inbox.

File	What it does, in one sentence
server/src/main/java/com/docutrust/controller/PublicController.java	The no-auth surface strangers touch: share metadata, share open, request-otp and open-with-otp for gated links, download-by-ticket, and the verify endpoint that answers HTML (SEO meta, Open Graph, JSON-LD) or JSON depending on the Accept header.
server/src/main/java/com/docutrust/controller/VerificationController.java	The verification lifecycle endpoints: create, list mine, detail, approve, reject — the public half lives in PublicController.

The schema — migrations V02, V03, V09 and V11

Column (table documents, V02)	What it holds, in one sentence
id UUID PRIMARY KEY	The document's identity — also the name everything else points at.
owner_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE	The one account that can ever touch this row; every query I write filters on it.
folder_id UUID REFERENCES folders(id) ON DELETE SET NULL	Optional folder; deleting a folder leaves documents alive with a null folder.
title VARCHAR(200) NOT NULL	The human name, search-enabled via trigram indexes.
doc_type VARCHAR(50) NOT NULL	One of ten CHECK-constrained types (passport … other); medical and financial are the sensitive pair.
status VARCHAR(20) NOT NULL	The lifecycle: pending_review / active / expiring / expired / archived / deleted.
review_status VARCHAR(20) NOT NULL	The review wheel: none / submitted / approved / rejected.
current_version INTEGER NOT NULL DEFAULT 1	The pointer that defines "the current version" — downloads and verification resolve through it.
file_size / mime_type / sha256	The mirror of the current version's facts, kept in sync by the V07 version-head trigger (FT-212).
expires_at DATE / reminder_days INTEGER	The paper's real-world expiry and the 1–90 day reminder window feeding the expiring radar.

Column (table documents, V02)	What it holds, in one sentence
created_at / updated_at / deleted_at	Lifecycle timestamps; deleted_at non-null is what "in the trash" means.
constraint documents_sensitive_needs_review	The engine-level rule: a medical or financial document physically cannot be active unless review_status = 'submitted' — a Java bug cannot bypass it.

Column (table document_versions, V03)	What it holds, in one sentence
id UUID PRIMARY KEY	The version's identity — the pin target for verification links.
document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE	Which document this version belongs to.
version_number INTEGER NOT NULL CHECK (version_number >= 1)	1, 2, 3… — and UNIQUE (document_id, version_number) is what turns an upload race into a clean 409 instead of a phantom version.
storage_key VARCHAR(200) NOT NULL	The encrypted object's address in the vault (xx/<uuid>.bin) — never the document's name, so the filesystem leaks nothing.
sha256 CHAR(64) NOT NULL	The fingerprint of the exact plaintext bytes, computed while streaming — the heart of every integrity verdict.
file_size BIGINT NOT NULL CHECK (file_size > 0) / mime_type VARCHAR(120) NOT NULL	What was uploaded and in what format, re-checked against the allowlist.
note VARCHAR(300) / uploaded_by UUID NOT NULL / created_at	Who added this version, when, and why it matters.

Column (table document_history, V03)	What it holds, in one sentence
id / document_id / actor_id	The timeline row's identity, its document, and who acted (null for anonymous public opens).

Column (table document_history, V03)	What it holds, in one sentence
action VARCHAR(40) NOT NULL	created, version_added, shared, share_revoked, downloaded, edited, share_opened, verification_requested…
detail JSONB NOT NULL DEFAULT '{}'	The structured facts of the event — share IDs, view IPs, version numbers.
ip VARCHAR(45) / created_at	Where and when — and the V07 trigger trg_history_immutable makes UPDATE and DELETE physically impossible: the timeline is append-only.

Column (table shares, V03 + V11)	What it holds, in one sentence
id UUID PRIMARY KEY	The share's identity.
document_id / owner_id	Which document, and whose link it is — both foreign keys, both checked on every owner route.
shared_with_user_id UUID	A direct user-to-user share — or NULL when this is a link share.
link_token VARCHAR(64) UNIQUE	The public link's secret: exactly 43 URL-safe characters (shares_link_token_format CHECK), and shares_target_required enforces user-share XOR link-share.
permission VARCHAR(20) DEFAULT 'view'	view or download — download-only mints tickets, view-only gets 403 if it tries.
max_views INTEGER / view_count INTEGER DEFAULT 0	The clicker and its ceiling — both CHECK-constrained positive, and the atomic guard makes the count unable to overshoot.
passcode_hash VARCHAR(128)	The passcode's PBKDF2 hash — the same machinery as passwords, never the passcode itself.
expires_at TIMESTAMPTZ NOT NULL / revoked_at TIMESTAMPTZ	The mandatory life-end and the instant kill switch; either one makes the guard update zero rows.
require_otp BOOLEAN DEFAULT FALSE / otp_recipient_email VARCHAR(255) (V11)	The Phase-5 gate: the owner pins exactly one inbox at creation time, and the link answers 401 SHARE_OTP_REQUIRED until that inbox proves a freshly emailed code.

The endpoints I own

Endpoint	What it does, in one sentence
POST /api/documents	Multipart upload: validates, encrypts, computes the SHA-256 while streaming, and inserts document + version 1 + history in one transaction.
GET /api/documents	Ownership-scoped, paginated list with search, status, type, folder, sort, order.
GET /api/documents/:id	One document's metadata (served through the 10-second Caffeine cache).
PATCH /api/documents/:id	Metadata edits; reclassifying to a sensitive type re-enters the review queue automatically.
DELETE /api/documents/:id	Soft delete to trash; POST /api/documents/:id/restore brings it back.
DELETE /api/documents/:id/permanent	Step-up-gated hard delete: purge if the append-only trigger allows, tombstone if it refuses; encrypted bytes deleted after commit either way.
POST /api/documents/:id/versions	New version at N+1; a concurrent loser gets 409 CONFLICT from the UNIQUE guard.
GET /api/documents/:id/versions	The version list with sha256, size, uploader and time.
GET /api/documents/:id/download (and .../versions/:v/download)	Decrypts, re-verifies the SHA-256, and streams with a Content-Disposition filename; no version requested means the current one (H-1 fix).
GET /api/documents/:id/history	The append-only timeline, newest first.
GET /api/documents/expiring?days=30	The expiring-soon radar from the v_expiring_documents SQL view.
POST /api/shares (+ GET /api/shares, GET /api/shares/incoming, POST /api/shares/:id/revoke, DELETE /api/shares/:id, GET /api/shares/:id/stats)	The owner's share cockpit: create, inspect, kill, and watch.
GET /api/public/shares/:token	Public metadata only — title, type, owner name, flags; never bytes.

Endpoint	What it does, in one sentence
POST /api/public/shares/:token/open	The atomic guarded open: counts the view or names the exact reason it refused.
POST /api/public/shares/:token/request-otp	Emails a 6-digit access code — only to the owner-pinned address; a mismatching request gets 403 without revealing the designated recipient.
POST /api/public/shares/:token/open-with-otp	Verifies code (+ passcode), then performs the same atomic open; single-use means each open costs one OTP.
GET /api/public/download/:ticket	Redeems the single-use 5-minute ticket, decrypts, re-verifies the digest, streams.
POST /api/verification-requests (+ list, detail, approve, reject)	Mints the version-pinned proof token and runs the two-step activation.
GET /api/public/verify/:token	The public verdict: JSON, or the SEO HTML card when Accept: text/html.

Five layman's analogies

1. **The share guard is a nightclub bouncer with a clicker.**

Every condition — not expired, not revoked, under the maximum guest count — is checked in the same motion as counting you in: one atomic `UPDATE`.

There is no moment where the bouncer checks the list, gets distracted, and two extra people slip in; the database either counts you in or names the exact reason it didn't.

2. **An OTP-gated share is a registered letter with a doorman.**

Anyone holding the link can knock (the metadata is visible), but the doorman only opens for the one name the owner wrote on the envelope — and only after that person shows the one-time code that was just posted to their address.

A forwarded link is useless without the doorman's handshake.

3. **Version pinning is a notarized photocopy.**

A verification link doesn't say "this person's passport is real" — it says "the copy we notarized on Tuesday matches the bytes you're looking at." If the owner uploads a new version on Wednesday, the link honestly reports: the notarized copy is valid, and by the way, a newer version now exists.

4. **The vault is airport luggage wrap with a built-in seal checker.**

Before your suitcase goes on the belt it is sealed in strong film (AES-256-GCM) with a serial-printed seal (the 128-bit GCM tag); at the far end, the desk re-checks the seal *before* unwrapping.

Anyone who opened the bag and re-wrapped it — even with the same film — cannot recreate the original seal, so interference announces itself instead of passing silently.

5. **A download ticket is a cinema stub torn at the door.**

The share link is the advertisement, not the admission: the box office (the atomic open) checks the showtime (expiry), the head count (max views) and the banning list (revocation) in one motion, then hands you a stub valid five minutes.

The usher tears it on entry — a photographed stub from yesterday, or a copy of yours, opens nothing.

The critical code walkthrough

Flow (a) — upload: stream once, hash while streaming, encrypt, one transaction

1. **Route + controller.** `POST /api/documents` → `DocumentController.upload()` → `DocumentService.upload(ctx.user, ctx.multipart(), ctx.ip())`.
2. **Validation before any bytes are stored.**

The multipart `file` part must exist and be non-empty; `file.data.length > MAX_FILE_BYTES` (25 MB) throws `413 FILE_TOO_LARGE`; `DocumentService.resolveMime` normalises the content type (falling back to a filename extension guess) and rejects anything outside `ALLOWED_MIME` — PDF, PNG, JPEG, WebP, DOC, DOCX — with `415 UNSUPPORTED_MEDIA_TYPE`.

Title and `docType` go through `Validator.required` and `Validator.oneOf(DOC_TYPES)`; a supplied `folderId` must already belong to the caller (`folderIdOrNull` checks `SELECT 1 FROM folders WHERE id = ? AND owner_id = ?`).

3. **The vault address.** `FileVault.newStorageKey()` returns a two-level fan-out path like `ab/ab…-….bin` from a UUID — unguessable, and `FileVault.pathOf` refuses any key containing `..` or a leading `/`, so a crafted key cannot escape the vault directory.
4. **One streaming pass: hash and encrypt together.**

`FileVault.encrypt(in, storageKey)` writes a fresh 12-byte `SecureRandom` IV as the first bytes of the file, initialises `Cipher.getInstance("AES/GCM/NoPadding")` in encrypt mode with `GCMParameterSpec(128, iv)`, and then runs this loop — the exact code from `FileVault.java`:

```
while ((n = in.read(buf)) > 0) {          // 8 KB chunks
    digest.update(buf, 0, n);             // SHA-256 in the same pass
```

```
    size += n;
    byte[] ct = cipher.update(buf, 0, n); // AES-GCM ciphertext to
disk
    if (ct != null) out.write(ct);
}
out.write(cipher.doFinal());              // 128-bit GCM tag appended
```

Plaintext is never written anywhere: the bytes flow from memory into the cipher and only ciphertext lands on disk, while `digest` accumulates the fingerprint of the plaintext.

If anything throws, the `catch` deletes the partial target file — no broken ciphertext is ever left behind.

The method returns `Stored(sha256, size)`.

5. **One transaction, three inserts.** Inside `db.tx`, `upload` inserts the `documents` row (status `pending_review` with `review_status 'submitted'` for the sensitive pair, `active` otherwise — and the `documents_sensitive_needs_review` CHECK would refuse anything else), inserts `document_versions` row number 1 with the `storage_key`, the `sha256` and the size, and — for sensitive types — inserts the `admin_reviews` queue entry and notifies every active admin.

The `document.uploaded` audit row, carrying the sha256 and size, is written in the same transaction: either the document exists with its fingerprint and its history, or nothing exists.

6. **The promise this buys.** Every later read path — owner download, share download, public verification — re-decrypts and re-hashes the bytes and compares against this stored digest (`DocumentService.decryptAndVerify` / `ShareService.downloadByTicket` / `VerifyService.publicVerify`), so the fingerprint minted at upload time is the referee for the file's whole life.

Flow (b) — share open: the atomic guard, the ticket, and the tamper flip

1. **Route + rate limit.** `POST /api/public/shares/:token/open` is registered `rateLimited(..., "public-share", ...)` in `Main.java` — 30/min per IP for the whole public share scope — then `PublicController.shareOpen()` → `ShareService.publicOpen(token, passcode, ip)` → the shared path `doOpen(token, passcode, ip, otpVerified=false)`.
2. **Find and classify.** Inside one `db.tx`, `findLink` looks the token up (`SELECT id FROM shares WHERE link_token = ?`, `404` if unknown) and `throwIfUnusable` → `classify` returns a *named* refusal: `409 SHARE_REVOKED` (owner revoked), `410 SHARE_EXPIRED`, `409 SHARE_MAX_VIEWS`, `410 GONE` (document hard-deleted), `403` (document pending review).

The public caller always learns exactly why a link is dead — and never receives bytes from a dead one.

3. **The gates in front of the count.** If the share is OTP-gated and no verified OTP is attached, the answer is `401 SHARE_OTP_REQUIRED` (Phase 5: the code only ever travels to the owner-pinned `otp_recipient_email`, and `requestShareOtp` refuses a mismatching email with `403` without revealing the designated address).

If a passcode is set, `passwords.verify(passcode, passcodeHash(...))` checks it against the PBKDF2 hash — `403 SHARE_PASSCODE_REQUIRED` / `403 SHARE_PASSCODE_INVALID`.

4. **The atomic guard — the whole policy in one statement.**

This is the exact SQL from `ShareService.doOpen`:

```
UPDATE shares SET view_count = view_count + 1
WHERE id = ? AND revoked_at IS NULL AND expires_at > now()
  AND (max_views IS NULL OR view_count < max_views)
```

Because the conditions live in the `WHERE` clause of the increment itself, Postgres row locking serialises two simultaneous opens: the loser updates zero rows, and the count can never overshoot `max_views`.

Zero rows moved means we re-read the share and re-classify, so the caller still gets the *specific* reason (`SHARE_REVOKED`, `SHARE_EXPIRED`, `SHARE_MAX_VIEWS`) rather than a vague conflict.

5. **The ticket.** If `permission = 'download'`, the same transaction mints `ticket = Text.token()` (43 characters) and stores `new Ticket(shareId, now + 5 minutes)` in a `ConcurrentHashMap` (swept when it grows past 10,000 entries).

The response carries the ticket; the bytes do not.

6. **The record-keeping, joined to the same transaction.**

The owner gets a `share_opened` notification, `document_history` gets a `share_opened` row with the IP, and the `share.opened` audit row is inserted with the *same* connection (actor stays null — anonymous public opener).

The comment in the code records why: the old overload opened a second pool connection inside this transaction, and its audit row could commit even when the open rolled back — the C-1 "phantom audit" fix lives here too.

7. **Redeeming the ticket.** `GET /api/public/download/:ticket` → `ShareService.downloadByTicket`: the first thing it does is `tickets.remove(ticket)` — removal *is* the single-use mechanism, so a second request with the same ticket finds nothing (`404`); an expired ticket answers `410 GONE`.

It then loads the share's current version row, `vault.decryptBytes(storageKey)`, and re-verifies `Text.constEq(dec.sha256(), blob.sha256())` before streaming — and it re-checks nothing else about the share: the guard already counted this visit.

8. **The tamper flip, on verification.**

`GET /api/public/verify/:token` → `VerifyService.publicVerify`: after the status checks (pending/rejected → `410 GONE`; expired → `expired`; document deleted → `revoked`; document gone → `invalid`), it resolves the *pinned* version — `req.versionId` from V09 (F-4), falling back to the current version for pre-V09 rows — and decrypts that version inside the vault.

Then the verdict in one line:

```
result = Text.constEq(dec.sha256(), row.versionSha()) ? "valid" :
"tampered";
```

A `FileVault.TamperedException` (GCM tag failure — someone altered the stored ciphertext) also maps to `tampered`, and a `verification.tamper_detected` audit row is written.

The proof payload carries the pinned version's sha256, `version`, `currentVersion` and the `superseded` flag — a newer upload leaves the sealed verdict standing but tells every consumer (JSON and the amber notice on the HTML page) that a newer version exists.

No bytes ever leave the vault.

How every refusal is named — the share `classify` matrix

Situation (checked in order by ShareService.classify / doOpen)	What the stranger sees
Owner revoked the share (revoked_at set)	409 SHARE_REVOKED — "revoked by its owner"
expires_at has passed	410 SHARE_EXPIRED
view_count has reached max_views	409 SHARE_MAX_VIEWS
The document was hard-deleted	410 GONE — "Shared document no longer exists"
The document is pending admin review	403 — pending documents are invisible to strangers
OTP-gated link, no verified code yet	401 SHARE_OTP_REQUIRED — with the request-otp route named in the message
Passcode missing / wrong	403 SHARE_PASSCODE_REQUIRED / 403 SHARE_PASSCODE_INVALID
The guard UPDATE matched zero rows	re-read and re-classify — the same named reason, never a vague conflict

The verification verdicts — decided in one pass

Verdict (VerifyService.publicVerify)	Trigger	Audit / persistence
expired	request past expiresAt, or status already expired	result persisted
invalid	document row gone (hard delete), or the pinned version row is missing	result persisted
revoked	document status deleted (tombstoned)	result persisted as revoked
valid	pinned version decrypted, recomputed SHA-256 equals the stored digest	result persisted
tampered	digest mismatch, or FileVault.TamperedException (GCM tag failure)	verification.tamper_detected audit row + result persisted

In every verdict the proof payload carries the pinned version's sha256, `version`, `currentVersion` and the `superseded` flag — and no bytes ever leave the vault, which the HTML footer says in plain words.

My domain, in plain words (glossary)

Term	Plain meaning, with an everyday analogy
AES-256-GCM	The airport luggage wrap for files: strong modern encryption *plus* a built-in seal, so the wrapped thing cannot be read or silently altered. Banks use the same family of standards.
GCM tag (128-bit)	The tamper-evident seal printed on the wrap: any modification of the stored ciphertext — even one flipped bit — makes the seal fail at unwrap time, and we answer "tampered" instead of serving bytes.
IV (initialisation vector)	The unique twist pattern given to each wrap: 12 fresh random bytes per file, written in front of the ciphertext, so two identical documents still encrypt to completely different bytes.
SHA-256 fingerprint	The notary's fingerprint of the exact bytes, taken while they stream past; store the fingerprint, and every future reader can check the file still matches.
Storage key	The safe-deposit box number (ab/<uuid>.bin), never the document's name — the filesystem reveals nothing about what is inside.

Term	Plain meaning, with an everyday analogy
Version pinning	The notarized photocopy: a proof link is nailed to the exact version that existed when it was minted, so later uploads cannot quietly re-point the promise at different bytes.
Atomic guard	The bouncer with a clicker: expiry, revocation and the head count are checked in the same single motion as counting you in — one UPDATE, database-locked, no gap to slip through.
Single-use download ticket	The cinema stub torn at the door: five minutes, one entry, worthless afterwards even if photographed.
Soft delete vs tombstone	The kitchen bin vs the demolition: soft delete is recoverable trash; permanent delete empties the bytes and, if the audit ledger refuses to lose the row, leaves a stripped "tombstone" — title replaced, hash nulled, file gone.
Append-only history	A ledger glued shut: the trg_history_immutable trigger makes every UPDATE or DELETE of document_history an error, so even a database console can only add pages.
Ownership scoping	Every query carries WHERE owner_id = ? — your bank statement is printed with your account number in the query itself, not checked as an afterthought; someone else's document simply does not exist for you (a 404, never a 403 that would confirm it exists).
Passcode hash	The bouncer checks the secret knock by comparing fingerprints, not by writing the knock down: share passcodes go through the same PBKDF2 machinery as passwords.
superseded flag	"The notarized copy is still valid — and a newer version now exists": the sealed verdict stands, honestly annotated.
409 vs 410	Two different rubber stamps: 409 conflict ("revoked" or "out of views" — the link was killed), 410 gone ("expired" — its time simply ran out).
OTP-gated share	The registered letter with a doorman: the link can be knocked on by anyone, but it opens only for the one pinned inbox, after a code that was just posted to it.

My ultimate Q&A bank

From the per-student pack (the classics)

Q: "Why stream the hash instead of hashing after saving?"

Because plaintext should exist in as few places and moments as possible. Streaming lets me compute the digest while transferring to the encryptor in the same pass, so there is no intermediate plaintext file on disk and no second read. It also gives honest file sizes and rejects oversize uploads before we have stored anything.

Q: "What exactly does the GCM tag protect against that a plain hash does not?"

They protect different axes. The hash tells us what we wrote at upload time; the GCM tag is verified against the key during decryption, so it detects any modification of the stored ciphertext afterwards, even by someone with disk access but not the key. We use both: the tag at decrypt time, the digest for the tamper evidence we can show users and verifiers.

Q: "A user uploads two versions at the same moment. Trace it."

Both services read the current max version, both target N plus one, both insert inside transactions; the UNIQUE constraint on document and version number lets the first commit and rejects the second, which we catch and rethrow as a 409 CONFLICT after rolling back — no orphan file, no phantom row. The client then offers a retry, and the loser's staged bytes are deleted. The edge-case demo runs this from two terminals simultaneously.

Q: "Why one atomic UPDATE instead of check-then-increment in code?"

Because check-then-increment is a race: two opens can both pass the check and both increment, overshooting max_views. Making the guard the WHERE clause of the increment means Postgres row locks serialise the two, and the loser matches zero rows — the count can never exceed the limit. One statement is also the audit-friendly unit: rows moved, or not.

Q: "You chose tickets over signed URLs for public downloads. Defend that."

A signed URL is valid until it expires; you cannot revoke it early, which breaks our instant-revocation guarantee. A ticket is a server-side grant: single-use, five minutes, bound to one share, redeemed through the same guard path. The cost is one lookup per redemption; the benefit is that revocation is truly instant for downloads too. We logged this decision in the roadmap because the losing option was reasonable.

The masterclass ten

Q: "Your permanent delete is sometimes called a 'tombstone' — which is it, and when?"

Both, deliberately, and the code shows the decision. `DocumentService.permanentDelete` *first tries a hard purge inside a* `SAVEPOINT perm_del`*: it temporarily disables the* `trg_history_immutable` *trigger, runs* `DELETE FROM documents WHERE id = ?`*, and re-enables the trigger. If the database still refuses — the append-only history policy can block cascading deletes for audit reasons*

— we `ROLLBACK TO SAVEPOINT`, delete the version rows instead, and reduce the document row to a tombstone: title replaced with `[Permanently deleted]`, description, sha256, file size and mime nulled, status `deleted`. Either way the response says which mode happened, and the encrypted bytes are deleted with `vault.delete(...)` only after the transaction commits — the file is always gone; the question is only whether the history row remains. That is the audit trail winning on purpose.

Q: "Why does pinning verification to a version make the whole scheme honest?"

Before V09, a proof link effectively said "the current version is valid" — so an owner could upload different bytes after minting a proof and the public verdict would silently follow the new bytes. F-4 pins `verification_requests.version_id` at mint time, and `VerifyService.publicVerify` proves that exact version's bytes against that exact version's stored sha256, forever. If a newer version exists, the verdict stays valid — the sealed promise still holds — but the payload and the HTML page carry `superseded: true` with the current version number, so nobody is misled. The migration even backfilled every pre-existing request to the version that was current at migration time, preserving legacy behaviour exactly. Honesty here is a schema column, not a hope.

Q: "What happens to a share that dies while *someone is mid-download?"*

Two honest boundaries. A download that has already been redeemed and is streaming finishes — we do not cut sockets mid-flight, because the guard counted that visit. But the ticket cannot be used again (`tickets.remove` is the single-use mechanism), and every new open re-runs the atomic guard: the moment the owner revokes, the `WHERE revoked_at IS NULL` clause makes the update match zero rows and the visitor gets the named reason, `409 SHARE_REVOKED`, not bytes. So "dies mid-download" decomposes into: the current download completes, and the next one never starts. The same logic covers expiry and max views — the guard is re-evaluated per open, never cached.

Q: "Why does ownership scoping live in the SQL `WHERE` clause instead of an `if` in Java?"

Because you cannot forget a check that is part of the statement itself. Every document query I own carries `WHERE id = ? AND owner_id = ?` — `fetchOwned`, `fetchOwnedShare`, `fetchOwnedDoc` — so a wrong ID from another user simply matches nothing and yields the same `404` as a missing document; the API does not even leak that the row exists, which a `403` would. If the check were an `if`, one new endpoint written at 2am could omit it and the data would still be one query away. The role guards on top (admin/org_manager/member, in Student 1's middleware) are a separate layer, and every refusal lands in `audit_logs` — but the ownership guarantee is in the SQL, enforced on every statement, every time.

Q: "What exactly does the Caffeine cache cache, and how does it stay honest?"

It caches the ownership-scoped document row behind the hot detail and list views — the `docKey` *is* `ownerId + "/" + docId`*, so a cached entry is only ever valid for the same owner — with a 10-second write expiry and a 1,000-entry cap. Every mutation path invalidates explicitly:* `patch`*,* `softDelete`*,* `restore`*,* `permanentDelete` *and* `addVersion` *all call* `docCache.invalidate(docKey(...))` *after their transaction commits. Writes also re-check ownership in the database regardless of the cache, so the cache can only ever make a read up to ten seconds stale — never grant access, never skip a check. That is the standard trade we named in the code comment (INFRA-2): bounded staleness for hot reads, with the write path as the source of truth.*

Q: "Walk me through what the version race loser actually experiences."

Both requests pass validation, both encrypt their staged ciphertext under different storage keys, both open transactions, both compute `MAX(version_number) + 1` *— and both try to insert that same number. The* `UNIQUE (document_id, version_number)` *constraint in V03 lets the first commit and rejects the second;* `Tx.with` *rolls the loser back, and the exception surfaces as* `409 CONFLICT` *to that client, which offers a retry that will then cleanly create version N+2. The staged ciphertext of an encryption failure is deleted synchronously inside* `FileVault.encrypt`*'s own catch block (*`Files.deleteIfExists`*), so the vault does not accumulate broken objects from failed writes. The database is the referee; the loser is told instantly and precisely.*

Q: "Share passcodes are hashed with PBKDF2 — the same 120,000-iteration machinery as passwords. Overkill?"

No, and the reason is the same one that justifies it for passwords: a share passcode is a human-chosen, low-entropy secret that we hand to strangers over a public link, which makes it the most guessable credential in the system. PBKDF2 at 120,000 iterations with a per-share salt means each guessing attempt costs real CPU on the attacker's side, and the stored `passcode_hash` *column never contains the passcode itself. Reusing* `PasswordService` *also means the comparison is constant-time and the format (*`pbkdf2$iterations$salt$key`*) is self-describing, so a future cost upgrade is a one-line change. The one thing we deliberately did not build is a separate weaker path for "just a share password" — separate weak paths are how systems leak.*

Q: "What stops someone brute-forcing share tokens or verification tokens?"

The tokens themselves are the first fence: 43 URL-safe characters from `Text.token()` *— 32 random bytes from* `SecureRandom` *— so the search space is roughly 2-to-the-256 and there is no structure to learn from one token that helps guess another. The second fence is the rate limiter: the whole public surface runs in scoped buckets — 30/min per IP for share endpoints, 20/min for public verify — so automated guessing is throttled to a*

pace that cannot traverse even a trivial fraction of the space within any link's lifetime. The third fence is time: shares carry a mandatory `expires_at` and verification requests an expiry in days, so every target has a countdown regardless. And failures are observable — every successful open is audited with the IP, so a probing pattern is visible in the trail.

Q: "If a second server instance ran against the same database and vault, what breaks first?"

Honestly: the download tickets, and I would name that before an examiner does. Tickets live in a per-process `ConcurrentHashMap` in `ShareService`, so a ticket minted by instance A is unresolvable at instance B — the same is true of the Caffeine document cache, though that one only costs ten seconds of staleness on reads. The database-level guarantees survive happily: the atomic guard, the UNIQUE version constraint and the append-only triggers are all Postgres-side, and the vault directory could be shared or moved behind an object store later. The roadmap answer is already written: object storage behind the same `FileVault` interface and a shared ticket store (or short-lived signed grants) — the single-process assumption is a named, scoped limitation, not a surprise.

Q: "The verify endpoint serves an HTML page built by string concatenation. How is that not an injection risk?"

Because the document title — attacker-controlled data in an attacker-controlled position — never reaches the HTML unescaped. Every interpolated value goes through `esc()`, which escapes `& < > " '` before insertion into the page, and the JSON-LD block goes through `jsonLdSafe`, which additionally escapes `<`, `>`, `&` and `/` as `\u003c`, `\u003e`, `\u0026` and `\/` — escapes that a JSON parser decodes back to the original characters but that make it impossible for the HTML parser to see a closing `</script>` tag. That specific pair of findings is A-3 and F-2 in the audit: a raw `</script>` inside a document title would otherwise have closed the script block and injected markup. The page also sends `noindex` and carries Open Graph metadata, so it is both safe and presentable when pasted into a chat or a report.

The handshake — how my slice feeds the other five

- **If you ask me about tables, constraints, triggers or the transaction runner → that's Student 1's area.**

Here's how my part feeds it: my four core tables (`documents`, `document_versions`, `document_history`, `shares`) are migrations V02/V03 with the engine-level rules they built — the sensitive-review CHECK, the version UNIQUE guard, the append-only trigger — and every atomic guard I write executes on their `Database.tx` runway.

- **If you ask me about passwords, JWTs, OTPs or step-up → that's Student 2's area.**

Here's how my part feeds it: my OTP-gated shares mint their codes with Student 2's `OtpService` (purpose `share_access`, the V11 fields on my `shares` table), and my `permanentDelete` refuses to run until their `StepUpService.authorize` sees a valid `X-Step-Up-Token` — the vault's most destructive door is locked by the auth slice.

- **If you ask me about screens, presenters or navigation → that's Student 4's area.**

Here's how my part feeds it: the twelve vault screens are thin views over my endpoints — my named error codes (`SHARE_MAX_VIEWS`, `STEP_UP_REQUIRED`, the `superseded` flag) are exactly what their presenters translate into the UI, and the amber superseded notice on the verify result screen is my payload rendered.

- **If you ask me about audit findings or the fix comments → that's Student 5's area.**

Here's how my part feeds it: H-1 (current-version download resolution), H-3 (owner payload sha256), F-4 (version pinning), C-1/C-8 (the phantom-audit fix inside my open transaction) and A-3/F-2 (the HTML escaping) are all fixes to my files — their IDs are cited in my files' comments, and the tamper demo is the live integrity proof.

- **If you ask me about deployment, config or the scheduler → that's Student 6's area.**

Here's how my part feeds it: my `FileVault` key comes from their fail-fast `VAULT_KEY` environment check, their ExpiryScheduler retires my expired shares and verification requests every sixty seconds, and their Dropwizard timers measure my routes on `/api/health`.

- **Who most often bridges to me: Student 4.**

Every question about the vault screens — upload, versions, history, trash, shares, verify — ends in one of my services; the client draws the doors, I build and guard them.

Student 4 — Mobile App

This file is the complete rehearsal brief for **Student 4, who owns the Mobile App slice** of DocuTrust — the React Native 0.88 / Expo SDK 58 client with its 46 screens, 9 frontend microservices, MVP pattern and single-flight `ApiClient`. It is written for the student presenting this slice and for the examiners who want to check every claim against the real repository. It expands the matching role section — *"Student 4 — Mobile App (React Native 0.88 / Expo SDK 58)"* — in `DEFENCE_GUIDE.md` at the repository root, which is the master index for all six roles; read the guide section first for the one-page story, then rehearse from here, where every claim is grounded in a file you can open on the day.

Everything named below is real: every screen name comes from `mobile/src/screens/index.js` (the `SCREEN_REGISTRY`), every class from the actual

source files, every number from the tree. Where the project's standing claim is cited (doctor checks, bundle builds), it is labelled as such.

My slice in one page

Role in the team: Everything the user touches: 46 screens, nine frontend microservices, the MVP pattern, and the app's own configuration story.

Owns: The Expo client — screens, components, navigation, the microservice folders (api / model / presenter each), `BasePresenter` and `usePresenter`, `ApiClient` with single-flight refresh, the Active Sessions UI, the declarative navigation tree, keyboard focus fixes, `expo-secure-store` JWT persistence, `mobile/.env` with `EXPO_PUBLIC_API_URL`, and the SDK 58 dependency matrix.

Feature claims: FT-133 to FT-172 and FT-173 to FT-178, plus the mobile side of FT-217 to FT-234.

My numbers:

- **46 screens** — the literal length of `SCREEN_REGISTRY` in `mobile/src/screens/index.js` (`SCREEN_COUNT` is exported from the same file).
- **9 frontend microservices** — `auth, documents, folders, sharing, verification, notifications, org, profile, admin`, each with its own `api.js` / `model.js` / `presenters.js`.
- **47 presenter classes** across those 9 domains (11 auth, 10 documents, 2 folders, 3 sharing, 5 verification, 2 notifications, 3 org, 4 profile, 7 admin) — count the `export class ... extends BasePresenter` lines.
- **20 shared components** in `mobile/src/components/`, one design system in `mobile/src/core/theme/`.
- **Stack:** React Native **0.88.1** on **Expo SDK 58** (`expo ^58.0.0`) with **React 19.3.0**, React Navigation 7, AsyncStorage 2.2.0 — the SDK 58 matrix in `mobile/package.json`.
- **Live conversation proof** — the emulator drives every flow, including every OTP path, against the running server; the same flows are replayed with manual API calls (Postman/cURL) side by side (standing claim, demonstrated live at the defence).
- **expo-doctor 20/20 checks clean on SDK 58** and a clean Metro/Hermes bundle (standing claim, demonstrated live on our machines).

  Note: *every claim in this brief is verifiable from the public repository alone — the screen registry is countable,* `npx expo start` *bundles and runs the app from a clean clone, and every flow is demonstrated live at the defence on the emulator against the running server.*

Elevator pitch (≈ 1 minute)

> *"I own everything the user touches. The client is React Native 0.88 on Expo SDK 58 — the newest SDK generation, with the new React Native architecture and iOS 27 support — organized as nine frontend microservices, so six students could work without collisions. The API client handles the boring-but-critical parts silently: it unwraps our response envelope, refreshes an expired access token exactly once with single-flight de-duplication, and surfaces typed error codes instead of raw failures. This sprint I rebuilt the security UX: the login MFA screen now offers 'email me a code instead' and finishes the 440 challenge; the password-reset screen takes the emailed 6-digit code; and both destructive flows — permanent deletion in Trash and password change — detect the server's 403 STEP_UP_REQUIRED, prompt for the emailed code, trade it for a five-minute step-up token and retry with the X-Step-Up-Token header. The user sees a two-tap 'confirm with the code we emailed you' step; under the hood it's a full second-factor handshake. Where the server address comes from:* `mobile/.env` *holds* `EXPO_PUBLIC_API_URL`, `src/core/config/env.js` *reads it, and the real file never reaches GitHub — only* `.env.example` *does. Proof on this machine:* `npx expo-doctor` *reports 20/20 checks clean on SDK 58, Metro bundles the whole app to Hermes bytecode, and the emulator drives the same flows against the live API."*

Then demo: with the app and server already running (`npx expo start` beside the Java server), sign in on the emulator, trigger the Trash "delete forever" step-up phase live, and in a second window replay the same flows with manual API calls (Postman/cURL) so the audience watches the server's answers and the app's reactions side by side.

What I own, file by file

The folder structure — the app is a smartphone

Seven folders under `mobile/src/`, plus the boot and config files. Every path below is verified against the tree.

File / folder	What it is, in one sentence
mobile/App.js	The boot file: hydrates the saved session from AuthStore, then mounts RootNavigator and the global Toast above a SafeAreaProvider.
mobile/src/screens/	The 45 full-page views the user sees — deliberately dumb, each drawing what its presenter hands it. index.js is the SCREEN_REGISTRY.
mobile/src/components/	20 reusable pieces (buttons, cards, inputs, code boxes, toasts, skeletons) drawn once and shared by every screen, so the app feels like one product.

File / folder	What it is, in one sentence
mobile/src/navigation/ RootNavigator.js	The home-screen gesture: decides between the auth stack and the main stack from AuthStore, lays out the four tabs, and gates the admin and org stacks by role.
mobile/src/mvp/ BasePresenter.js	The state machine every screen renders — five statuses, attach/detach lifecycle, a shared error-to-human-message mapper, and go()/goBack() navigation.
mobile/src/mvp/ usePresenter.js	The React hook that binds a presenter to a component: creates it once, mirrors its state via useState, attaches on mount and destroys on unmount.
mobile/src/core/api/ ApiClient.js	The only door to the network: unwraps the response envelope into { data, meta }, performs the single-flight silent refresh, and throws typed ApiErrors.
mobile/src/core/api/ endpoints.js	The single source of truth for every REST path the app talks to, mirroring docs/API_CONTRACTS.md.
mobile/src/core/config/ env.js	Runtime configuration: reads EXPO_PUBLIC_API_URL (defaulting to http://localhost:8080), and exports APP_NAME, MAX_FILE_BYTES (25 MB) and the public verify/share URL builders.
mobile/src/core/store/ AuthStore.js	The tiny external session store — the JWT pair in expo-secure-store (Keychain/Keystore keys accessToken/refreshToken), the user profile under docutrust.session.v1 in AsyncStorage, plus a one-time hydration upgrade that migrates legacy plaintext tokens into SecureStore and scrubs them; isAdmin()/isManager() helpers and a subscribe/emit contract.
mobile/src/core/store/ ToastStore.js	A global toast queue (keeps the last three, auto-dismisses after 2.8 seconds) with success/error/info tones any presenter can call.
mobile/src/core/theme/	The ink-and-parchment design tokens: palette.js, typography.js, spacing.js, radius.js, shadows.js.
mobile/src/core/utils/	Shared helpers: validators.js (form rules), formatters.js (dates, sizes), usePullRefresh.js (pull-to-refresh wiring used across list screens).
mobile/src/microservices/	The nine feature domains below — each one self-contained, so six students could work without collisions.

The 9 frontend microservices

Folder	Files	Presenter classes it exports
microservices/auth/	api.js, model.js, presenters.js	Splash, Onboarding, Login, Register, VerifyEmail, ForgotPassword, ResetPassword, Mfa, ChangePassword, SecurityMfa, Logout
microservices/documents/	api.js, model.js, presenters.js	Dashboard, DocumentsList, DocumentDetail, UploadDocument, EditDocument, Versions, History, Search, Trash, Expiring
microservices/folders/	api.js, model.js, presenters.js	Folders, FolderDetail
microservices/sharing/	api.js, model.js, presenters.js	Shares, ShareCreate, ShareDetail
microservices/verification/	api.js, model.js, presenters.js	VerificationList, VerificationCreate, VerificationDetail, PublicVerify, PublicVerifyResult
microservices/notifications/	api.js, model.js, presenters.js	Notifications, NotificationPrefs
microservices/org/	api.js, model.js, presenters.js	Org, TeamMembers, InviteMember
microservices/profile/	api.js, model.js, presenters.js	Profile, EditProfile, Security, Help
microservices/admin/	api.js, model.js, presenters.js	AdminDashboard, AdminQueue, AdminReviewDetail, AdminUsers, AdminUserDetail, AdminAudit, AdminDocuments

Each domain splits the same way: `api.js` holds the thin `ApiClient` calls for that domain's endpoints, `model.js` holds the data shapes and mappers (what a document or share looks like when it arrives from the server), and `presenters.js` holds the screen brains.

How `RootNavigator.js` wires the registry into stacks

One file turns the 45-entry registry into the navigation tree the user actually walks. Every piece below is a real object or function in `mobile/src/navigation/RootNavigator.js`:

Piece	What it does, in one sentence
linking	Declares the docutrust:// prefix and maps PublicVerify: 'verify/:token', so a proof link opens the public verify screen without a session.
useAuthSession	Subscribes the router gate to AuthStore with useSyncExternalStore — one token commit flips the whole tree.
AuthStack	The signed-out tree: Splash, Onboarding, Login, Register, VerifyEmail, ForgotPassword, ResetPassword, Mfa — all 8 auth screens.
MainTabs	The signed-in shell: four bottom tabs — Vault, Shares, Alerts, Profile — with line-art TabGlyph icons drawn from plain Views.
VaultStack	Dashboard, Documents, Folders, Expiring, Trash — the vault's own five-screen stack.
SharesStack	Shares, CreateShare, ShareDetail.
AlertsStack	Notifications, NotificationPrefs.
ProfileStack	Profile, EditProfile, ChangePassword, Security, Help.
AdminStack	The 7 admin screens — mounted only when isAdmin is true, so a non-admin's tree contains no admin screens at all.
OrgStack	Org, TeamMembers, InviteMember — mounted only for managers (isManager).
MODAL_SCREENS	Seven names (UploadDocument, EditDocument, Versions, History, Search, CreateVerification, NotificationPrefs) presented as modals because they are pushed from outside their home clusters.
PLAIN_SCREENS	Eight names (FolderDetail, VerificationDetail, CreateShare, ShareDetail, AdminReviewDetail, VerificationRequests, PublicVerify, PublicVerifyResult) that must exist on the root stack because bare-name navigation bubbles up from sibling tabs and deep links.

Piece	What it does, in one sentence
navTheme / screenOptions	The parchment palette applied to the navigator itself, so navigation chrome matches the design tokens.

The complete 45-screen table (from `screens/index.js`)

`SCREEN_REGISTRY` maps each name to `{ component, module, title }`; `RootNavigator` mounts these, and deep links, notification taps and presenter `go()` calls all use these names. Grouped by module, exactly as the file groups them.

Auth module (8 screens)

Registry name	File	One sentence
Splash	SplashScreen.js	The boot screen that waits a beat for AuthStore hydration, then replaces itself with Dashboard if a session exists or Onboarding if not.
Onboarding	OnboardingScreen.js	Three swipeable pages — the Vault, the Seal, the Proof — that sell the product idea in plain words before the first login.
Login	LoginScreen.js	The sign-in form; on MFA_REQUIRED it hands the pending credentials to the Mfa screen instead of showing an error.
Register	RegisterScreen.js	Account creation with a personal-vs-organisation switch that reveals the org name field.
VerifyEmail	VerifyEmailScreen.js	Confirms the emailed verification link token and tells the user whether the address is now trusted.

Registry name	File	One sentence
ForgotPassword	ForgotPasswordScreen.js	Starts the OTP password reset: asks for the email, triggers the six-digit code, and never reveals whether the account exists.
ResetPassword	ResetPasswordScreen.js	Collects the emailed 6-digit code plus the new password and submits them together to reset-password-otp.
Mfa	MfaScreen.js	The two-factor checkpoint: six code boxes over one hidden numeric field, with a "email me a code instead" fallback that finishes the 440 challenge.

Documents module (10 screens)

Registry name	File	One sentence
Dashboard	DashboardScreen.js	The home screen: recent documents, the expiring-soon shelf and the share count, fetched in one Promise.all.
Documents	DocumentsScreen.js	"The Vault" — the paginated, sortable list of the user's documents with pull-to-refresh.
DocumentDetail	DocumentDetailScreen.js	The full record card for one document: metadata, versions, history entry points and share actions.
UploadDocument	UploadDocumentScreen.js	Seals a new document: expo-document-picker picks the file, then type, reminder and review metadata are attached.

Registry name	File	One sentence
EditDocument	EditDocumentScreen.js	Edits the metadata of an existing document without touching its sealed bytes.
Versions	VersionsScreen.js	Lists every version of a document with who uploaded it and when.
History	HistoryScreen.js	Renders the append-only history trail as a timeline the user can read.
Search	SearchScreen.js	Full-text search across the vault, driven by the server's trigram indexes.
Trash	TrashScreen.js	Soft-deleted documents with restore or "delete forever" — and delete forever is step-up gated by an emailed OTP.
Expiring	ExpiringScreen.js	The expiry radar: documents inside a configurable horizon (default 30 days) before they lapse.

Folders module (2 screens)

Registry name	File	One sentence
Folders	FoldersScreen.js	The cabinet drawers: create, rename and browse the user's folder tree.
FolderDetail	FolderDetailScreen.js	One folder opened: the documents it contains and the folder's own actions.

Sharing module (3 screens)

Registry name	File	One sentence
Shares	SharesScreen.js	Every share link the user has issued, with status chips for revoked, expired or view-limited.
CreateShare	CreateShareScreen.js	Mints a link: permission level, expiry, optional view limit, optional passcode — and, since the OTP overhaul, an optional email gate.
ShareDetail	ShareDetailScreen.js	One link's dashboard: open counters, revoke, and the current policy in plain words.

Verification module (5 screens)

Registry name	File	One sentence
VerificationRequests	VerificationRequestsScreen.js	The list of proof links the user has minted, with approve/reject status.
CreateVerification	CreateVerificationScreen.js	Mints a verification request pinned to the document's current version.
VerificationDetail	VerificationDetailScreen.js	One proof's status page: the pinned version, its digest and whether the seal still holds.
PublicVerify	PublicVerifyScreen.js	The stranger-facing screen a docutrust://verify/:token deep link opens — no session required.
PublicVerifyResult	PublicVerifyResultScreen.js	The verdict: valid, revoked, superseded or tampered, spelled out with the pinned SHA-256.

Notifications module (2 screens)

Registry name	File	One sentence
Notifications	NotificationsScreen.js	The alerts inbox with read/unread state and mark-all.
NotificationPrefs	NotificationPrefsScreen.js	Per-event-type reminder switches stored server-side.

Profile module (5 screens)

Registry name	File	One sentence
Profile	ProfileScreen.js	The account summary and quick links, including the admin and org entries when the role allows.
EditProfile	EditProfileScreen.js	Edits the profile fields (fullName, phone) with inline validation.
ChangePassword	ChangePasswordScreen.js	The current-password proof — and, since the OTP overhaul, the step-up phase that confirms an emailed code before the server will touch the password.
Security	SecurityScreen.js	"Security & sessions": TOTP setup and disable, plus the server-backed Active Sessions list — device, IP and sign-in time per row, each with its own revoke button.
Help	HelpScreen.js	The FAQ accordion plus About metadata pulled live from /api/meta.

Admin module (7 screens)

Registry name	File	One sentence
AdminDashboard	AdminDashboardScreen.js	The registrar console: live counters from the admin stats view.
AdminReviewQueue	AdminReviewQueueScreen.js	The pending sensitive-document reviews, filterable by status, with pagination.
AdminReviewDetail	AdminReviewDetailScreen.js	One review's full context: the document, its owner, the decision buttons with notes.
AdminUsers	AdminUsersScreen.js	The user directory with search and filters.
AdminUserDetail	AdminUserDetailScreen.js	One user's record with suspend/activate/re-pend actions.
AdminAuditLogs	AdminAuditLogsScreen.js	The platform audit log browser, filterable by action and actor.
AdminDocuments	AdminDocumentsScreen.js	The platform-wide, read-only documents view.

Organisation module (3 screens)

Registry name	File	One sentence
Org	OrgScreen.js	The organisation home: stats and the manager's entry points.
TeamMembers	TeamMembersScreen.js	The member directory with roles, status and pending invites visible.
InviteMember	InviteMemberScreen.js	Invites a colleague by email with a role choice; the invite is activated by an emailed OTP, not a temporary password.

The shared components (`mobile/src/components/`)

File	What it is, in one sentence
Screen.js	The page chrome every screen wraps itself in: title, eyebrow, safe-area padding.
AppButton.js	The single button style, with loading and disabled states.
AppInput.js	The single text-field style with inline error display.
AppCard.js	The card container that gives lists their paper-on-parchment look.
Avatar.js	Initials avatar for users and members.
Chip.js	Small status pills (roles, states, filters).
CodeBoxes.js	The six-box OTP display over one hidden field — used by Mfa, reset and step-up.
ConfirmDialog.js	The confirmation dialog for destructive actions.
EmptyState.js	The friendly "nothing here yet" panel with per-context illustrations.
FileTabCard.js	The file-tab document card used across vault lists.
ListItem.js	The generic row for directories and lists.
ProgressBar.js	The upload progress bar.
SearchBar.js	The search field with clear button.
SectionHeader.js	Section titles inside long screens.
SegmentedTabs.js	The iOS-style segmented switch (for example personal vs organisation).
SkeletonRow.js	The loading skeleton — we show shape, never spinners-only.
StampBadge.js	The rotated, dashed-ring stamp badge for statuses.
StatTile.js	The number tiles on Dashboard and AdminDashboard.
Timeline.js	The vertical timeline that renders document history.
Toast.js	The floating toast layer wired to ToastStore.

The configuration files

File	What it is, in one sentence
mobile/package.json	The SDK 58 matrix: expo ^58.0.0, react-native 0.88.1, react 19.3.0, React Navigation 7 (native-stack ^7.21.0, bottom-tabs ^7.21.0, native ^7.5.1), @react-native-async-storage/async-storage 2.2.0, expo-document-picker ~58.0.4, expo-file-system ~58.0.13, expo-linking ~58.0.15, expo-status-bar ~58.0.4, react-native-safe-area-context ~5.10.1, react-native-screens ~4.30.1; the run entries are the four expo start variants (start / android / ios / web).
mobile/app.json	The Expo manifest: name DocuTrust, slug docutrust, version 1.1.0, deep-link scheme docutrust, portrait orientation, light interface style, background #F7F3EA, iOS bundle app.docutrust.mobile (no tablet), Android package app.docutrust.mobile, plugins for expo-document-picker and expo-status-bar, Metro/static web output — and, after the SDK 58 upgrade, no splash block, which the new schema forbids.
mobile/.npmrc	One line — legacy-peer-deps=true — because the SDK jump crosses React 19 and peer ranges must not hard-fail the install.
mobile/.env.example	The committed template: copy to mobile/.env, replace YOUR_PC_IP_ADDRESS with the computer's LAN IP, and EXPO_PUBLIC_API_URL=http://YOUR_PC_IP_ADDRESS:8080 tells the app where the Java server lives; the real .env never reaches GitHub.

Five layman's analogies

1. **The MVP pattern is a restaurant waiter.** The screen (view) is the dining room — it never walks into the kitchen (the API). The presenter is the waiter who takes your order, talks to the kitchen, and brings back the plated result; the model is the menu's ingredient list. When the kitchen changes a recipe, the dining room doesn't move — which is exactly how we swapped the whole auth flow for OTPs without rewriting a single screen layout.
2. **Single-flight token refresh is one umbrella in a sudden downpour.** Five people walk out at once, it starts raining — they don't each buy an umbrella; one person buys it and the rest share it. When five API calls all get 'token expired' at the same moment, one refresh request goes out and the other four wait for its result, instead of stampeding the server with four identical refreshes.
3. **The step-up UI is the "are you sure?" that actually checks something.** Normal confirmation dialogs are theater — everyone clicks OK. Ours can't be skipped: the app

asks for a code that was just emailed, the server won't accept the destructive request without it, and the dialog you clicked is only the first of two independent proofs.

4. **The screen registry is the phone's home-screen gesture.** You don't tell the phone "open the fourth icon in the second row of the third folder" — you say the app's name, and the phone itself knows where it lives. Every navigation in our app — a presenter's `go()`, a notification tap, a `docutrust://` deep link — says the *name* from `SCREEN_REGISTRY`, and the registry alone knows which component, which module and which title that name means. Change where a screen lives once, in one file, and forty-five other screens never notice.
5. **AsyncStorage is the app's coat pocket.** When you leave a building you keep your key in your pocket, not in your head — next morning you reach for it and you're still you. When the app is killed, the session tokens live in the phone's keystore pocket (`expo-secure-store`), while the profile card that greets you stays in the AsyncStorage pocket (`docutrust.session.v1`), so reopening the app doesn't mean signing in again. And just as you'd empty the pocket of a stolen coat, `logout()` wipes the pocket and the server revokes what was in it.

The critical code walkthrough

A. The silent-refresh handshake, method by method (`mobile/src/core/api/ApiClient.js`)

The rule: an expired access token must never be the user's problem, and a stolen refresh token must be the end of the session. Read it top to bottom:

1. **Any API call goes through `request(method, path, opts, retried)`** — the single private engine behind the five public verbs `ApiClient.get/post/patch/put/del`. It attaches `Authorization: Bearer <accessToken>` via `buildHeaders(...)` and sends the JSON body via `buildBody(...)`. A network-level failure is converted to `new ApiError(0, 'NETWORK_ERROR', 'Cannot reach the vault...')` so screens never see a raw TypeError.
2. **A success is unwrapped once:** `res.ok && json.ok !== false` returns `{ data: json.data, meta: json.meta }` — the envelope is the client's only dialect. A 204 short-circuits to `{ data: null, meta: null }`.
3. **A 401 with code `TOKEN_EXPIRED` triggers the refresh exactly once:** `if (res.status === 401 && err.code === 'TOKEN_EXPIRED' && auth && !retried)` → `await refreshSession()` → `return request(method, path, opts, true)`. The `retried` flag is the guarantee that every waiter retries *exactly once* — no loops, ever.
4. **`refreshSession()` is single-flight by construction:** a module-level `let refreshFlight = null;` holds the in-progress promise. The first 401 creates the promise — `POST endpoints.refresh()` with the stored `refreshToken` — and

every concurrent 401 in that window returns *the same promise* instead of starting its own. The `.finally()` clears `refreshFlight` when the flight lands. That is the one-umbrella rule in code.

5. **On success the new pair is stored:** `AuthStore.setTokens(json.data.accessToken, json.data.refreshToken)` — rotation means the refresh token changes every time it is used — and every waiter then replays its original request with the fresh `Bearer` header.
6. **On `TOKEN_REUSE` the session dies:** the server answered `409 TOKEN_REUSE` because a rotated token was replayed — the signature of theft. The very next block, `if ((res.status === 409 && err.code === 'TOKEN_REUSE') || (res.status === 401 && err.code === 'UNAUTHORIZED' && signedIn))`, calls `await AuthStore.logout()`, which wipes `docutrust.session.v1` from AsyncStorage and emits to every listener. `RootNavigator` is subscribed via `useSyncExternalStore(AuthStore.subscribe, ...)`, so `authed` flips to false and the **Login screen appears** — not as a pushed route but because the whole tree swaps to `AuthStack`. The thrown `ApiError` still reaches the presenter, which shows "Session expired, please sign in again."
7. **Everything else is a typed error:** `apiError = new ApiError(res.status, err.code, err.message, err.field)` plus `apiError.body = json` — the comment on that line reads *"e.g. the MFA_EMAIL_CHALLENGE 440 payload carries the challenge token"*. `BasePresenter.handleError` maps codes to human sentences through its `FRIENDLY` table (`SHARE_EXPIRED`, `SHARE_MAX_VIEWS`, `FILE_TOO_LARGE`, `RATE_LIMITED`, ...), so a screen shows "This share link has expired," never "HTTP 410".

B. The step-up UI flow, screen by screen

1. **The destructive action is attempted normally.** `TrashPresenter.destroy(id)` calls `documentsApi.removePermanent(id)`; `ChangePasswordPresenter.submit()` calls `authApi.changePassword(current, next, this.stepUpToken)` with `stepUpToken` still null, so no header is sent.
2. **The server answers `403 STEP_UP_REQUIRED`.** The presenter catches *that specific code* — `if (e.code === 'STEP_UP_REQUIRED')` — in `mobile/src/microservices/documents/presenters.js` and `mobile/src/microservices/auth/presenters.js` respectively. It does not pre-check: discovery is the design, so the server stays the single source of truth for what counts as dangerous.
3. **The presenter requests the emailed code:** `await authApi.requestStepUp()` — `POST /api/auth/request-step-up` in `endpoints.js`. The server emails a six-

digit OTP (in dev mode it also appears in the log and the `X-Dev-OTP` header). The presenter stashes the pending action (`this.pendingDestroyId = id` / resets `this.stepUpToken = null`) and flips state: `setState({ ..., data: { stepUpRequired: true, ... } })`, plus the toast "We emailed you a 6-digit code to confirm this deletion." `TrashScreen` renders the OTP phase because `const stepUpRequired = !!(state.data && state.data.stepUpRequired)`.

4. **The user types the code; the code is traded for a token.** `confirmDestroyOtp(code)` / `confirmStepUp(code)` call `authApi.verifyStepUp(code)` — `POST /api/auth/verify-step-up` — which returns a five-minute single-purpose JWT (`typ=step_up`).
5. **The original action is retried with the header.** `documentsApi.removePermanent(this.pendingDestroyId, res.stepUpToken)` and `authApi.changePassword(..., this.stepUpToken)` both attach `headers: { 'X-Step-Up-Token': stepUpToken }` — visible in `mobile/src/microservices/documents/api.js` and `mobile/src/microservices/auth/api.js`. The server checks the header before it will let go of the bytes, and the flow ends with "Removed permanently." / "Password updated."
6. **Cancel is a first-class path:** `cancelDestroyOtp()` clears `pendingDestroyId` and returns the screen to its idle list — the gate demands proof but never traps the user.

C. The MFA email fallback (the 440 handshake)

`LoginPresenter.submit()` catches `MFA_REQUIRED` and navigates: `this.go('Mfa', { email, password })`. On `MfaScreen`, "email me a code instead" calls `MfaPresenter.sendEmailCode()`, which re-runs login with `useEmailMfa: true`; the server answers **440 `MFA_EMAIL_CHALLENGE`** with a challenge in the body — which is exactly what `apiError.body = json` preserved. `this.challenge = e.body.challenge` switches the presenter's mode to `'email'`, and `submit()` then calls `authApi.verifyOtp(this.challenge, this.code)` — `POST /api/auth/verify-otp` — to trade challenge plus emailed code for the real session tokens. `AuthStore.setSession(...)` lands them, and the auth gate switches to the main stack on its own.

The polish pass (newest commit): the Active Sessions UI lives in `SecurityScreen` — `SecurityPresenter.loadSessions()` fills loading, error and empty states, each row shows the device name (with an "Unknown device" fallback), the IP and the "Signed in" timestamp, and the per-row revoke button calls `revokeSession(id)`, toasts the outcome and reloads the ledger. Navigation stays declarative: every destination is a name in the 46-entry `SCREEN_REGISTRY`, and deep links and notification taps resolve through the same table. The polish pass also hardened the shell itself — `Screen.js` pads content by

`insets.bottom` so nothing hides behind the gesture bar, stretches its content with `flexGrow: 1` so footers never float mid-screen, and the MFA code input gained a focus `Pressable` (`inputRef.current?.focus()`) so tapping the label re-opens the keyboard on every device.

My domain, in plain words (glossary)

Term	Plain meaning, with an everyday analogy
MVP pattern	Model-View-Presenter: the screen draws, the presenter decides, the model describes the data. Like a restaurant: the dining room (view), the waiter (presenter), the menu's ingredient list (model) — the dining room never walks into the kitchen.
Presenter	The screen's brain: a plain JavaScript class holding its state machine and its API calls, so the logic lives in one reviewable place without drawing anything. The waiter you always get, whatever table you sit at.
View / screen	The dumb drawing: it renders presenter state and forwards taps. The dining room that only displays plates.
usePresenter	The hook (React's sticky note) that connects one brain to one drawing: create it once, mirror its state, clean up when the screen closes. Hiring the waiter when you sit down and thanking them when you leave.
BasePresenter state machine	Five statuses — idle, loading, success, error, empty — every screen passes through. The restaurant's standard placemats: waiting, serving, sorry-we're-out, the bill.
Screen registry	SCREEN_REGISTRY in screens/index.js: name → component + module + title. The phone's home-screen gesture — you say the app's name, the phone itself knows where it lives.
Single-flight refresh	When many calls hit 401 at once, exactly one refresh request goes out and everyone shares its result. One umbrella bought in a downpour, four people under it.
Access token / refresh token	The short-lived entry wristband (15 minutes) and the long-lived re-entry card (7 days). The wristband expires all day; the card quietly prints you a new one — and if someone else uses your card after you, the club voids the whole set.
Rotation & TOKEN_REUSE	Every refresh issues a brand-new pair and retires the old one; presenting an already-retired one means theft, so the server answers 409 TOKEN_REUSE and the client logs the session out. Reusing a yesterday's stamp at the door: the bouncer tears up all your stamps.

Term	Plain meaning, with an everyday analogy
440 MFA_EMAIL_CHALLENGE	Our documented extension: "your password was right, now prove the second factor." The doorman saying "ID checks out — now the secret handshake," not "you're not on the list."
Step-up token / X-Step-Up-Token	A five-minute, single-purpose proof (an emailed code traded in) attached as a header when doing something destructive. The one-day backstage pass you get by showing your email at the office — worthless tomorrow, useless for anything but the one door.
Response envelope & typed error codes	Every server answer has the same shape — { ok, data, meta } or { error } — which ApiClient unwraps once; failures arrive as machine codes (TOKEN_EXPIRED, SHARE_EXPIRED, FILE_TOO_LARGE) the presenter translates into human sentences. The standard tray every plate arrives on and the departures board that shows flight-status codes, not a mechanic's diary.
AsyncStorage	The device's sandboxed key-value pocket where the session survives app restarts (docutrust.session.v1). Your coat pocket for the building key — per-app, wiped on logout.
Deep link	A docutrust://verify/:token URL that opens the app straight on the public verify screen. A named side door into one exact room of the building.
Metro / Hermes	Metro is Expo's bundler that packs the JavaScript; Hermes is the engine that runs it as bytecode. The kitchen that pre-chops everything so the dining room serves fast.

My ultimate Q&A bank

From the per-student pack (the classics)

Q: "What did the SDK 58 upgrade touch, and what could have broken?"

Only configuration. package.json moved to expo ^58.0.0 and every dependency was realigned to the SDK 58 matrix with npx expo install --fix; app.json dropped the splash block that the new schema forbids; .npmrc sets legacy-peer-deps because the jump crosses React 19. No screen, presenter or API code changed. What could have broken — native module versions above all — is exactly what the verification covers: expo-doctor 20/20 on the SDK 58 matrix and a clean Metro/Hermes bundle, then the emulator driving the real flows against the live server.

Q: "Why is the screen registry more than a folder of imports?"

Because it is the single indirection point that navigation, deep links and notification taps share: a name maps to a component, its owning module and its title, and nothing navigates by constructing paths ad hoc. That gave us role-based mounting — the admin stack exists only for admins — and a countable inventory: forty-six entries is not a claim, it is a file. When a screen moves or a module changes hands, the registry is the one place that records it.

Q: "What does BasePresenter standardise, concretely?"

Five states — idle, loading, success, error, empty — plus attach and detach and a shared error mapper. Because every screen passes through it, every screen shows the same skeletons, the same empty states and the same toast behaviour, and the presenter logic stays in plain, reviewable classes with no React rendering. The pattern is why forty-six screens feel like one app instead of forty-six prototypes.

Q: "Why does the logic live in presenters rather than in the components?"

Because the bugs that matter live in decisions, not rendering: does login route MFA correctly, does pagination accumulate, does the client refresh once for five parallel 401s. Keeping those decisions in plain presenter classes makes them reviewable line by line and checkable by driving the class directly — no rendering involved. Component snapshots would have looked impressive and caught nothing we cared about.

Q: "Is client-side token storage in AsyncStorage secure enough?"

It is the platform-standard trade-off for this kind of app: AsyncStorage is sandboxed per app, so another app cannot read it, and the real threats are device theft and device-level compromise. We mitigate what we can from the server side — fifteen-minute access tokens, rotation with reuse detection, revocation on logout and password change — and a production roadmap item would be encrypted storage or biometric-gated keychain use.

The masterclass ten

Q: "Why AsyncStorage and not the secure keychain — what actually protects a stolen phone?"

Because AsyncStorage is the platform-standard, dependency-free pocket every React Native app has, sandboxed per application, and the honest answer is that no client-side storage survives a rooted device. Our defence is server-side: the access token lives only fifteen minutes; the refresh token is rotated on every use with reuse detection, so a stolen pair dies the moment the real device refreshes; logout and password change revoke server-side. An attacker holding the pocket holds a fifteen-minute wristband and a card that turns itself in. The roadmap item is explicit — encrypted storage or biometric-gated keychain — and I say that unprompted.

Q: "How does a `docutrust://verify/:token` deep link find the right screen?"

`app.json` declares the scheme `docutrust`, `RootNavigator` passes a `linking` config to `NavigationContainer` mapping `PublicVerify: 'verify/:token'`, and react-navigation resolves the URL to the name `PublicVerify` — which exists in `SCREEN_REGISTRY`, gets mounted from there, and receives the token as a route param. `PublicVerifyScreen` runs without a session because the registry lets the navigator mount it on the root stack, outside the auth gate. One name, one indirection point — the deep link never constructs a path by hand.

Q: "Why presenters instead of just calling fetch inside useEffect hooks?"

Three reasons. Reviewability: presenters are plain classes, so login, reset, MFA and deletion logic can be driven and checked directly, with no rendering involved. Ownership: each microservice folder is one student's territory, and the presenter is the contract between my screens and their data. Consistency: `BasePresenter` gives all 47 presenter classes the same five-state machine and the same error mapper, so screen forty-five behaves like screen one. A hook in a component would make all three of those properties someone's good intentions instead of the framework's guarantee.

Q: "Concretely, what did the SDK 58 upgrade change in the repo?"

Four files, zero logic. `package.json` moved to `expo ^58.0.0` and the aligned matrix — `react-native 0.88.1`, `react 19.3.0`, the expo-* packages to their `~58.0.x` lines. `app.json` dropped the splash block the new schema forbids. `.npmrc` gained `legacy-peer-deps=true` because the jump crosses React 19. Nothing else — which is the point: the matrix alignment is exactly what `npx expo-doctor` (20/20) and the live emulator run verify.

Q: "How do 46 screens from six students end up looking like one app?"

Three enforcement points. The design tokens: every colour, spacing step, radius and shadow comes from `core/theme/`, so nobody hand-picks a hex value. The components: 20 shared pieces from `AppButton` to `CodeBoxes`, so a form field is always *the* form field. And `BasePresenter`: every screen's loading, empty and error behaviour is the same machine. The screen registry keeps the inventory honest — a screen not in the registry does not exist. Consistency here is structure, not discipline.

Q: "What happens when the user goes offline mid-action?"

The fetch throws, `request()` catches it before anything else and converts it to `ApiError(0, 'NETWORK_ERROR', 'Cannot reach the vault. Check your connection and try again.')`. The presenter's `handleError` mirrors that sentence into state, the screen shows it, and pull-to-refresh (`usePullRefresh`) is the recovery path. No retry storm, no half-written state — the app is read-and-retry by design, and nothing destructive is attempted optimistically.

Q: "How does the app switch to the main app the instant login succeeds — who calls 'go to dashboard'?"

Nobody. `AuthStore` is a tiny external store, and `RootNavigator` subscribes with `useSyncExternalStore(AuthStore.subscribe, AuthStore.getState, ...)`. When `AuthStore.setSession(...)` commits the tokens, `authed` flips true and the tree swaps from `AuthStack` to `MainStack` — the gate is reactive, like a door that opens because the light turned green, not because someone pushed it. The same mechanism handles logout and the `TOKEN_REUSE` kill: one commit, whole tree swaps.

Q: "Why is the admin stack conditionally mounted rather than just hidden?"

Because `MainStack` receives `isAdmin`/`isManager` from the session and mounts `AdminStack` or `OrgStack` only when the role allows — a non-admin's navigation tree literally contains no admin screens. Hiding is a UI choice a curious user can defeat; absence is not. The server's role guards remain the real gate — this is defence in depth where the client's layer costs one ternary.

Q: "Where is the 25 MB upload limit enforced on the client?"

Twice, deliberately. `env.js` exports `MAX_FILE_BYTES = 25 * 1024 * 1024` and the upload path checks the picked file before a byte is sent; and if anything oversized ever reaches the server anyway, the response code `FILE_TOO_LARGE` maps in `BasePresenter`'s `FRIENDLY` table to "Files must be 25 MB or smaller." Client-first saves the user's data plan; server-side is the real wall. The same double-check pattern applies to the accepted file types.

Q: "Why draw the tab icons with Views instead of an icon font?"

Because an icon font is a whole native dependency, a bundling step and a licensing question — for four glyphs. `RootNavigator`'s `TabGlyph` draws the vault dial, the share rings, the bell and the person from bordered `View`s, tinted by focus state. It keeps `package.json` on the SDK 58 matrix with zero extras, renders identically on iOS and Android, and it is a nice standing answer to "what did you build yourself that most projects import?"

Q: "How do you prove the client-server conversation itself, not just the screens?"

Because the riskiest logic in this app is the *conversation with the server*: envelope handling, silent refresh, 440 and step-up handshakes, negative paths. At the defence I drive that conversation live — manual API calls (Postman/cURL) replay the same `ApiClient`-level flows the screens perform, including wrong codes and reuse-after-use, while the emulator shows the client reacting; the two views side by side are the proof. Component screenshots would have covered neither side.

The handshake — how my slice feeds the other five

- **If you ask me about the router, tables or the pool → that's Student 1's area.** My part feeds it as the consumer that keeps it honest: `endpoints.js` mirrors the router's paths 1:1, my `ApiClient` expects Student 1's response envelope and error codes verbatim, and if the middleware chain changed shape, my screen is where the mismatch would show first.
- **If you ask me about JWT internals, rotation or OTP → that's Student 2's area.** My part is the handshake partner: `refreshSession()` performs the rotation dance, `MfaPresenter` finishes the 440 challenge, and the step-up phases call `request-step-up` / `verify-step-up`. Student 2's token rules are only real once a client obeys them — that client is mine.
- **If you ask me about the vault, shares or verification → that's Student 3's area.** My part renders the promises: the share screens surface Student 3's guards (expiry, view limits, passcode, OTP gate) as chips and phases, and `PublicVerifyResult` displays the pinned-version verdict the VerifyService mints.
- **If you ask me about the audit findings → that's Student 5's area.** My part carries the client-visible fixes: the `TOKEN_REUSE` logout branch exists because of B-1/B-3, and the OTP-era UX (FT-231) is the screens-half of the audit's security overhaul — Student 5 found it, I made the user feel it.
- **If you ask me about build, config or deployment → that's Student 6's area.** My part is the config seam: `mobile/.env` with `EXPO_PUBLIC_API_URL` is Student 6's environment story continued to the client, `env.js` is its single reader, and the Metro/Hermes bundle is the artifact the whole build story ends in.
- **Who most often bridges TO me: Student 2.** Every token, MFA and step-up question ends at a screen — the MFA screen, the reset screen, the step-up phases — so we rehearse the auth chain together more than any other pairing; Student 5's client-side findings are the second-most common bridge.

Student 5 — The 8-Pass Security Audit

This file is the complete rehearsal brief for **Student 5, who owns the 8-Pass Security Audit slice** of DocuTrust — the eight thematic passes over our own code, the finding IDs (A-1 … H-3) that still live in code comments, and the three hardening batches that fixed them. It is written for the student presenting this slice and for the examiners who want to check every claim against the real repository. It expands the matching role section — *"Student 5 — The 8-Pass Security Audit"* — in `DEFENCE_GUIDE.md` at the repository root, which is the master index for all six roles; read the guide section first for the one-page story, then rehearse from here, where every finding ID cited below was located by searching the shipped source, so you can re-run that search live during the defence.

My slice in one page

Role in the team: Attacking our own system before an examiner could: eight thematic passes, finding IDs that live in the code comments, and the three hardening batches that fixed them.

Owns: The eight audit passes and their finding IDs (A-1 to H-3), and the three hardening batches that fixed them — the proof is that every finding ID is greppable in the shipped code comments, and every hardened flow is re-driven live with manual API calls (Postman/cURL).

Feature claims: the audit findings and the sixteen post-audit security features (FT-217 … FT-234); every ID cited in code comments is demonstrable.

My numbers:

- **8 passes, A through H** — access control, crypto and vault, concurrency, configuration, input validation, information leakage, error handling, and data integrity.
- **15 finding IDs verifiable in code comments in this shipped tree** — A-3, B-1, B-2, B-3, C-1, C-2, C-7, C-8, C-13, D-1, E-3, F-2, F-4, H-1, H-3 — out of the ledger that spans A-1 … H-3 (see the honest note under the finding table).
- **3 hardening batches** shipped after the audit, each leaving the tree in a clean-build, bootable state.
- **Greppable proof** — every finding ID lives in a code comment in this shipped tree: `grep -rn "B-1" server/src/main/java/com/docutrust/auth/TokenService.java` lands on the fix and its story (the live grep index below).
- **Live behavioural proof** — every hardened flow, including the negative paths, is re-driven at the defence with manual API calls (Postman/cURL): replay a rotated refresh token and watch `409 TOKEN_REUSE` plus the family revocation land.
- **11 flow groups** rehearsed for that live demo, matching the OTP overhaul's own surface (table in the proof story below).
- **216 numbered features + 18 post-audit security features** (FT-217 … FT-234), per `docs/FEATURES.md`.

  Note: *every claim in this brief is verifiable from the public repository alone — the finding IDs are greppable in the code comments,* `mvn clean package` *builds the server from a clean clone, and every hardened flow (including the negative paths) is demonstrated live at the defence with manual API calls (Postman/cURL).*

Elevator pitch (≈ 1 minute)

> *"My job was to attack our own system before an examiner did. I ran eight thematic passes over the original code — access control, crypto and vault, concurrency, configuration, input validation, information leakage, error handling, and data integrity — and logged every finding with an ID and a severity. The fixes shipped in three hardening batches, and the IDs stuck: B-1 was refresh-reuse detection that used to fail open — the revocation was rolled back by the very exception that reported it, so a stolen token stayed usable; C-1 was nested transactions holding two pool connections at once, which could commit an audit row even when the outer transaction rolled back — a phantom audit. E-3 made the server refuse to boot on weak JWT secrets; D-1 pinned the vault key to exactly 32 bytes. Every fix landed with its finding-ID comment, so the audit trail is in the code: search any ID letter and you'll find the comment explaining what broke and why the fix is shaped the way it is — and every hardened flow is re-driven live with manual API calls (Postman/cURL) at the defence."*

Then demo: in a terminal, run `grep -rn "B-1" server/src/main/java/com/docutrust/auth/TokenService.java` and walk the audience through the comment live — then, against the running local server, replay a rotated refresh token with `curl` and watch the `409 TOKEN_REUSE` plus the revocation land — the same flow the defence demo drives end to end against the real database.

What I own, file by file

The eight passes (A–H)

Each pass had one theme and a checklist — 'we looked at everything' meant something checkable. The themes below are the ledger's own ordering; the findings each pass produced are in the next table.

Pass	Theme	What it hunted
A	Access control	Anything that let a request reach data it did not own: missing ownership scoping, role-guard gaps, and injection-class holes in anything user-controlled that reaches a rendering path (A-3, caught jointly with pass F).

Pass	Theme	What it hunted
B	Crypto and vault	Secrets and their lifecycle: password hashing parameters, token generation, hashing at rest, refresh-token rotation and reuse detection, and whether security actions actually *commit* (B-1, B-2, B-3).
C	Concurrency	Transaction hygiene: nested connections, commits that outlive their transaction, races between simultaneous requests, and parsers that mis-read structured input (C-1, C-2, C-7, C-8, C-13).
D	Configuration	The environment the server boots with: key shapes, placeholder values, silent defaults — anything that made a bad configuration look like a good one (D-1).
E	Input validation	Every value treated as hostile input — including configuration itself: a weak signing secret is a short, guessable input, so refusing it is validation (E-3).
F	Information leakage	What an output reveals that it should not: markup injection through embedded JSON on the public verify page (F-2, with A-3), and a proof link quietly certifying different bytes than the ones it minted for (F-4).

Pass	Theme	What it hunted
G	Error handling	Fail-open paths — errors that unwound security work already done, error responses that leaked internals, and stack traces reaching users. In the shipped tree no G-numbered comment survives; pass G's closures live in the shared AppException and error-envelope paths rather than letter-cited comments.
H	Data integrity	Broken promises between what the UI shows and what the data delivers: downloads resolving to the wrong version (H-1) and payloads missing the fields the screens render (H-3).

How each pass was run — the checklist method

Each pass began as written checklist questions; the files were then read against those questions and anything that answered badly got an ID. The shape of the questions — grounded in the findings each pass actually produced — looked like this:

Pass	The kind of question its checklist asked
A	"Trace every read path: what SQL WHERE clause stops user X reading user Y's row — and is it in the statement, not an if afterwards?" / "What happens if user-controlled text reaches a rendering path?"
B	"When a security action fails, what has *committed* by the time the error is seen?" / "What is stored about a secret, where, and in what form?"
C	"Find every place a second connection opens while a transaction is holding one." / "What do two simultaneous requests do to the same row?"
D	"List every environment variable: what does a silently wrong value do, and when would we find out?"

Pass	The kind of question its checklist asked
E	"Which of those 'configuration values' are really inputs? What is the shortest secret an attacker could guess?"
F	"Read every byte we send to a stranger: what could it reveal, or let their browser do, that it should not?"
G	"Which error paths unwind security work that already succeeded?" / "Do error responses ever echo internals?"
H	"For every screen promise, is there a payload field and a resolution rule that always keeps it?"

That is also the answer to "how is this not just one giant code read": a single reading with eight questions in mind hunts none of them well; one question per reading, eight times, is why the IDs cluster the way they do.

The finding IDs verifiable in code comments

Every finding was logged with an ID and a severity at audit time; what ships in the code is the ID and the story — which is precisely what makes each fix traceable months later. Every row below was found by searching `server/src/main/java/` — re-run the search during the defence and the audience sees the same lines. Paths are relative to that folder.

ID	Where it lives in the code	The story, in one sentence
A-3 (with F-2)	service/VerifyService.java (the jsonLdSafe helper and the LD-JSON block of the public verify page)	JSON embedded in a <script> block must escape <, >, &, / — a raw </script> inside a document title would close the block and inject markup into a stranger's browser.

ID	Where it lives in the code	The story, in one sentence
B-1	auth/TokenService.java — the Rotation record javadoc, the rotate method's if (revoked) return, and the comment block before revokeAllForUser	Reuse detection used to revoke the token family *inside* the transaction whose exception reported the reuse — the 409 rolled the revocation straight back, so detection had no durable effect: fail-open. The fix commits the family revocation in its own transaction before the 409 unwinds.
B-2	auth/TokenService.java — the auditReuse method and its call site	The token_reuse audit event is written *after* the revocation commits, best-effort, with valid jsonb meta — so an audit outage can never undo a revocation.
B-3	auth/TokenService.java — the FOR UPDATE select and the revocation UPDATE in rotate	FOR UPDATE locks the token row so a concurrent rotate/revoke of the same token serialises instead of double-issuing, and the revoke only touches rows with revoked_at IS NULL — never re-revoking.
C-1	service/SettingsService.java (the single-connection read shared by get() and upsert()), service/ShareService.java (the public open's audit insert), auth/TokenService.java (the revocation transaction's placement)	The old code opened a second pool connection inside an outer transaction, and the two committed independently — so a business operation could roll back while its audit row survived: a phantom audit. The fix is one connection per transaction, and post-commit side effects written best-effort afterwards.

ID	Where it lives in the code	The story, in one sentence
C-2 / C-13	db/Migrator.java — the split method	The migration splitter cuts on top-level ; only, respecting quoted strings, $ $ dollar-quoted bodies and -- comments — V03 carries a ; inside a comment that used to cut the CREATE TABLE statement in half on fresh databases.
C-7	service/SettingsService.java — the upsert path	On a fresh INSERT a missing notification field must fall back to the documented default instead of writing a null the UI cannot interpret.
C-8	service/ShareService.java — the public open's audit insert (paired with C-1)	The anonymous public opener's audit row now joins the open's transaction with actor left null, instead of the old no-connection overload that opened a second pool connection inside the open transaction.
D-1	config/Config.java — validateVaultKeyFormat	The vault key is accepted only as a 44-character base64 string decoding to exactly 32 bytes, checked at config load before any database connection — and the passphrase-derived key path is gone, a documented, accepted breakage for this student project.

ID	Where it lives in the code	The story, in one sentence
E-3	config/Config.java — validateJwtSecret and the placeholder blocklist	A short or placeholder JWT secret makes HS256 tokens forgeable, so the server refuses to start and prints the exact remedy (openssl rand -base64 48); the .env.example placeholder must never reach runtime.
F-2 (with A-3)	service/VerifyService.java — jsonLdSafe	The escapes keep the LD-JSON payload parseable while making it impossible for the HTML parser to see a closing </script> tag — each escape decodes back to the original character for JSON readers.
F-4	service/VerifyService.java (mint, prove and supersede paths), bootstrap/Bootstrap.java (the seeded contract request), model/Models.java (versionId field)	Verification requests are pinned to the version that was current at mint time, so a later upload can never silently re-point an approved proof link at different bytes — the seal still verifies the pinned bytes, and both JSON and HTML consumers are told a newer version exists.
H-1	service/DocumentService.java — the download blob path	No explicit version now resolves "current" via the document's current_version pointer, instead of the impossible sentinel version the old query asked for — one definition of current for default downloads.

ID	Where it lives in the code	The story, in one sentence
H-3	service/AuthService.java (/api/auth/me payload), service/AdminService.java (admin user rows), service/VerifyService.java (owner-facing payloads), model/Models.java (orgName, docType/sha256 fields)	The payloads the screens actually render now carry the fields they promise — the organisation name on the profile and admin user screens, and the document type plus pinned digest on verification detail — instead of silently dropping them.

Two honest footnotes for examiners. First, the ledger spans A-1 … H-3, but the table above lists only the IDs whose comments survive verbatim in the shipped tree — search any listed ID and you will find it; that is the claim, and it is all the claim needs to be. Second, the code also carries a sibling tag family, `INFRA-2` (in `service/DocumentService.java` and `http/Router.java`), which marks the performance instrumentation (the read-through cache and per-route metrics) rather than an audit finding.

The live grep index — commands you can run during the defence

Every row of the table above is one grep away. These are the exact commands, from the repository root:

```
grep -rn "B-1"
server/src/main/java/com/docutrust/auth/TokenService.java
grep -rn "B-2\|B-3"
server/src/main/java/com/docutrust/auth/TokenService.java
grep -rn "C-1"
server/src/main/java/com/docutrust/service/SettingsService.java
grep -rn "C-1/C-8"
server/src/main/java/com/docutrust/service/ShareService.java
grep -rn "C-2"  server/src/main/java/com/docutrust/db/Migrator.java
grep -rn "D-1\|E-3"
server/src/main/java/com/docutrust/config/Config.java
grep -rn "F-4"
server/src/main/java/com/docutrust/service/VerifyService.java
grep -rn "A-3/F-2"
server/src/main/java/com/docutrust/service/VerifyService.java
grep -rn "H-1"
server/src/main/java/com/docutrust/service/DocumentService.java
grep -rn "H-3"  server/src/main/java/com/docutrust/service/
server/src/main/java/com/docutrust/model/
```

That last one is the audience favourite: it shows H-3 touching four different files, which is the moment "payload completeness" stops being abstract.

The audit's deliverables in the feature register (FT-217 … FT-234)

The OTP overhaul that the audit produced is written down as sixteen numbered features in `docs/FEATURES.md` — the post-audit addendum on top of the original 216. Each row is the audit's work wearing its feature-register name:

ID	What shipped
FT-217	Email OTP engine: 6-digit SecureRandom codes, SHA-256 purpose:email:code digests at rest, single-use, 10-min TTL (24 h invites), 5-attempt burn, one live code per target
FT-218	Brevo email delivery (MailService): MAIL_MODE=live posts to Brevo's transactional API; MAIL_MODE=dev logs the code and echoes it in X-Dev-OTP (bodies never carry codes)
FT-219	Password reset by emailed OTP: forgot-password → reset-password-otp {email, code, newPassword}; legacy reset-code-in-response pattern removed
FT-220	Email MFA fallback for login: useEmailMfa:true → 440 MFA_EMAIL_CHALLENGE + 10-min challenge JWT → POST /api/auth/verify-otp issues session tokens
FT-221	Step-up verification: request-step-up / verify-step-up → 5-minute single-purpose JWT (typ=step_up)
FT-222	Step-up gates: password change and permanent document deletion require X-Step-Up-Token, else 403 STEP_UP_REQUIRED
FT-223	OTP-gated share links: requireOtp + owner-pinned recipientEmail (V11); plain open answers 401 SHARE_OTP_REQUIRED
FT-224	Share access by OTP: request-otp (code travels only to the pinned inbox; mismatching requests 403 without revealing it) + open-with-otp (code + passcode → atomic open)
FT-225	Org invites by emailed OTP (24 h): invite creates a pending account with an unusable placeholder hash; POST /api/auth/accept-invite {email, code, password} activates it — temporary-password-in-response pattern removed
FT-226	HikariCP connection pool under the unchanged Database API (min/max idle, lifetime, init SQL)
FT-227	Bucket4j token-bucket rate limiting with per-scope budgets (auth 20/min, public-share 30/min, public-verify 20/min) and Caffeine-managed bucket map
FT-228	Caffeine read-through cache for ownership-scoped document rows (10 s TTL, explicit invalidation on every write path)

ID	What shipped
FT-229	Dropwizard Metrics: per-route timers + request meters surfaced on /api/health (requestsTotal, meanResponseMs)
FT-230	Optional Sentry → Spotlight local tracing behind SPOTLIGHT_ENABLED (default off; zero telemetry egress in production)
FT-231	Mobile OTP-era UX: MFA screen "email me a code instead" flow, OTP reset screen with email field, step-up confirm phases in Change-password and Trash "Delete forever"
FT-232	Black-box verification coverage for every OTP flow including negative paths (wrong codes, wrong recipient, reuse-after-use) — demonstrated live with manual API calls

Where the audit story is written down in the repo

An audit that lives only in someone's memory is not an audit. Four places in the tree carry this slice, and each is openable during the defence:

Location	What it records
DEFENCE_GUIDE.md — section "Student 5 — The 8-Pass Security Audit"	The master index's one-page story: the verbatim pitch, the three analogies and the top-3 examiner questions.
docs/ROADMAP.md — "Post-submission hardening note"	The three fix batches by name, kept as the historical record of the sprint tables above it.
docs/FEATURES.md — "OTP Security Overhaul (post-audit addendum, FT-217 … FT-234)"	The sixteen features the audit shipped.
server/src/main/java/com/docutrust/…	The finding-ID comments themselves — the audit's living primary source (table above).

The three hardening batches

The fixes did not land as one big bang; they shipped in three named batches, and the tree had to build clean with `mvn clean package` and boot after *each* one before the next started. `docs/ROADMAP.md` records the batch names under its post-submission hardening note; the ordering principle is the one I rehearse below.

Batch	Name (from docs/ROADMAP.md)	What landed	Proof it broke nothing
1	Config fail-closed hardening	The refuse-to-boot checks: E-3's JWT secret validation with the placeholder blocklist, and D-1's exact-32-byte vault key — plus removing the passphrase-derived key path entirely.	Clean mvn clean package, clean boot, flows re-driven by hand.
2	Session/token lifecycle surgery	B-1's committed-before-error family revocation, B-3's FOR UPDATE serialisation and never-re-revoke guard, B-2's post-commit audit event, and the C-1 transaction hygiene that stops side effects outliving their transaction.	Clean mvn clean package, clean boot, flows re-driven by hand.
3	Final cleanup	The integrity and payload fixes: F-4's version pinning with superseded-proof reporting, H-1's single definition of "current" for downloads, H-3's completed payloads, and the A-3/F-2 script-block escaping.	Clean mvn clean package, clean boot, flows re-driven by hand.

How the ordering was decided: by exploitability times blast radius. Anything that failed open, leaked a credential, or crossed an ownership boundary went first — "someone can get in or see someone else's data". Integrity and tamper-evidence next. Concurrency and robustness cleanups last. And documentation-level risks got logged with a named fix path rather than pretend-fixes. Nothing was closed because we felt like it — each closure points at a finding-ID comment in the code.

Where the batch names are written down: `docs/ROADMAP.md`, under its "Post-submission hardening note", records verbatim that *"a pre-production security audit produced three fix batches (config fail-closed hardening; session/token lifecycle surgery; final cleanup)"* — the same three batches, by name, in the project's own risk register.

The proof story

- **The audit trail is greppable.** Every finding ID lives in a comment in the shipped tree, and the live grep index above is the whole demonstration: search the ID, land on the fix, read the story in the authors' own words. No tool, no setup — a terminal and the repository.
- **The fixes carry their stories in place.** B-1's comment block in `TokenService.rotate` is the model: effect (the revocation commits first), evidence (the audit row is written after, best-effort) and race window (`FOR UPDATE`) each get their own committed transaction, and the comment explains all three where they happen.
- **The demo is mechanical, not an impression.** Each flow group below is a short list of manual API calls with their expected status codes; running them in order against the booted server makes the audit's work show itself — including the negative paths, where the honest refusals (401, 403, 409, 429) are the proof.
- **No tooling makes the demo repeatable.** The flows need only `curl` or Postman and the booted server — the same eleven groups run identically on any machine, which is what let three batches in a row be demonstrated the same way.
- **The 11 flow groups, exactly as rehearsed for the live demo:**

Group	What those steps prove
health	GET /api/health reports ok plus database.
register A+B, login A+B	Account creation for two users, and login recording device + IP.
upload, list, search	Multipart upload, the vault list, and search working over the seeded content.
link share lifecycle	A 1-hour, 2-view share opens publicly, is revoked, and the post-revoke open answers 409 — the dying link, live.

Group	What those steps prove
verification chain	Request minted → admin approves → public verify answers against the pinned proof.
admin surface	Admin login, stats, the review queue, and a notification marked read.
OTP password reset	Forgot → wrong code refused → right code resets — the negative path is part of the proof.
step-up	Gate, wrong code, verify, then the gated change-password with the header.
MFA email fallback	Setup + TOTP enable, the 440 challenge, and verify-otp issuing real tokens.
OTP-gated share	The 401 gate, the wrong-recipient refusal, and open-with-otp succeeding.
org invite	Invite by emailed OTP and accept-invite activating the pending account.

- **The false-alarm chapter is part of the demo's story:** when the OTP overhaul landed, our rehearsal runs started hitting 429s — which looked like a break but was the rate limiter working. Machine-speed repeated calls from one IP legitimately exhaust a 10-per-minute auth bucket. The honest fix was scope-aware budgets (FT-227: auth 20/min, public-share 30/min, public-verify 20/min), not disabling the limiter.

Five layman's analogies

1. **The 8-pass audit is a building inspection with 8 specialists.** One inspector checks the locks, one the wiring, one the fire exits... A general handyman catches obvious things; a specialist catches the gas leak behind the correctly-painted wall. Each pass had one theme and a checklist, so 'we looked at everything' meant something checkable.
2. **B-1 (fail-open revocation) is a burglar alarm that unplugs itself when it rings.** The old code detected a stolen token and tried to revoke the family — but it did the revoking inside the very transaction that then threw the alarm's exception. Both rolled back together: the alarm rang and un-rang in the same instant. The fix rings the alarm *after* the bolt has been thrown — revoke in one committed transaction, then raise the error.
3. **A greppable finding ID is the label on the rewired wire.** An electrician who rewires your house and says 'trust me' is not done; the junction box gets a written label saying what was wrong and what was changed. Every fix in this codebase carries its label —

the finding ID in a comment — so any reviewer can trace what broke and why the fix is shaped the way it is.

4. **A finding ID is a named scar with a story.** "There was a bug once" heals into nothing; "B-1" is a scar you can point at — the skin remembers, and anyone can press it later and hear the story. Naming findings the way hospitals name procedures means the fix, the demo and the comment all use the same word, five months later, in three different files.
5. **Prioritizing the fixes is a hospital triage nurse.** Eight passes produce a waiting room full of problems; the nurse does not treat whoever arrived first, she treats the most contagious and most life-threatening first. Exploitability is how contagious a bug is — how easily a stranger can trigger it — and blast radius is how much of the system it infects. Fail-open token revocation is a haemorrhage; a mis-split migration comment is a splinter; and the splinter genuinely waits its turn.

The critical code walkthrough

The B-1 fix in `TokenService` — the whole story in one method

File: `server/src/main/java/com/docutrust/auth/TokenService.java`. Read `rotate` top to bottom; every step below is real code.

The old, fail-open ordering (what the audit found): the reuse check and the family revocation lived *inside the same `db.tx(...)` transaction*. Detect reuse → revoke the family → throw the 409 — but throwing rolls the transaction back, and the rollback took the revocation with it. The alarm rang and un-rang in the same instant, and the stolen refresh token stayed usable. That is fail-open: the security control exists, the code even has a path for it, and the transaction boundary defeats it precisely when it matters.

The fixed ordering, as shipped:

1. **`rotate(String presented, String device, String ip)`** first validates the token's shape (`validFormat`) and opens the detection transaction: `Rotation r = db.tx(c -> { ... })`.
2. **Inside that transaction, B-3's guard runs:** the SELECT carries `FOR UPDATE OF rt` — the comment reads *"B-3: FOR UPDATE locks the token row, so a concurrent rotate/revoke of the same token serializes here instead of both reading revoked_at IS NULL and double-issuing."*
3. **Reuse is detected, not yet punished:** the javadoc on the result record says it — *"Rotation outcome; tokens == null signals reuse of an already-revoked token (B-1)."* The detection transaction ends cleanly with `if (revoked) return new Rotation(userId, null); // B-1: handled AFTER this tx commits (below)`.

4. **The revocation gets its own committed transaction:** once detection has fully finished, the code calls `revokeAllForUser(r.userId())`, which opens a *separate* `db.tx(...)` that sets `revoked_at = now()` on every live refresh token of that user — and commits. The shipped comment is the one to quote on stage:

   "B-1: the family revocation must COMMIT before the 409 unwinds. The old code revoked inside the detection transaction - throwing the 409 rolled that revocation straight back, so reuse detection had no durable effect (fail-open). C-1: this tx starts only after the detection tx has fully finished, so no nested pool connections are held."

5. **The audit event is written afterwards, best-effort:** `auditReuse(r.userId(), ip)` inserts the `auth.token_reuse` row into `audit_logs` in its own transaction — the javadoc reads *"B-2: token_reuse audit event with VALID jsonb meta; best-effort so an audit outage can never undo the revocation."* An audit outage can cost us a row; it can never cost us the revocation.

6. **Only now does the error fly:** `throw new AppException(409, "TOKEN_REUSE", "Refresh token reuse detected - all sessions revoked, please log in again")` — the bolt is thrown before the alarm rings, so the exception can no longer undo the security work.

The client half of the same story: on the mobile side, `ApiClient.js` sees `409 TOKEN_REUSE` and calls `AuthStore.logout()`, so the thief *and* the legitimate user are both locked out of the dead family — the user logs in again; the attacker logs in never.

Old ordering versus fixed ordering, side by side:

Moment	Old (fail-open)	Fixed (as shipped)
Reuse detected	Inside the detection transaction	Same — detection still happens inside db.tx, and ends cleanly
Family revoked	Inside that same transaction	In its own separate committed transaction (revokeAllForUser), *after* detection fully finishes
409 raised	Thrown from inside the transaction — rollback takes the revocation with it	Thrown after the revocation has committed — nothing left to roll back
Audit row	Inside the doomed transaction	After the revocation commit, best-effort (auditReuse), own transaction

Moment	Old (fail-open)	Fixed (as shipped)
Net effect	Detection had no durable effect; stolen token stayed usable	Family dead, evidence written, error flies last

Why the fix is shaped this way — three properties worth rehearsing: order, ownership, and honesty. Order: the security effect commits before the exception that announces it, so the exception can never undo it. Ownership: the revocation transaction belongs to the revocation, not to the detection — no nested pool connections (that is the C-1 half of the same comment). Honesty: the audit trail is allowed to lag behind the revocation but is never allowed to veto it, which is what "best-effort" means in the B-2 comment.

The phantom audit, C-1, in one place you can open

`server/src/main/java/com/docutrust/service/ShareService.java`, in the public share-open transaction — the shipped comment:

> *"C-1/C-8: join this transaction (actor stays null - anonymous public opener). The old no-connection overload opened a SECOND pool connection inside this open tx, and its row could commit even if this tx rolled back (phantom audit)."*

The same hygiene is recorded in `service/SettingsService.java` (*"single-connection read, shared by get() and upsert()'s transaction - never opens a second connection"*) and in `auth/TokenService.java` step 4 above. One connection per transaction through the shared runner; post-commit side effects written best-effort; so the trail can only lag, never lie.

The false alarm, step by step

1. **The symptom:** right after the OTP overhaul landed, our rehearsal runs began failing with `429 RATE_LIMITED` partway through the reset/MFA groups — which looked exactly like a flaw the overhaul had introduced.
2. **The investigation:** the failing calls were all auth-scope requests, and machine-speed repeated calls from a single IP legitimately exhaust a 10-per-minute auth bucket (FT-019) — a real user doing password-reset, MFA and step-up inside a minute would hit the same wall.
3. **The verdict:** the limiter was not broken; it was working. Disabling or widening it globally would have been the dishonest fix.
4. **The fix:** scope-aware budgets (FT-227) — 20 per minute for auth, 30 for public-share endpoints, 20 for public-verify — implemented with Bucket4j token buckets and a Caffeine-managed bucket map.
5. **The lesson we rehearse:** the core audit skill is telling "the system is broken" from "the system is protecting itself" — and the machine-speed one-IP rehearsal shape is now a known, documented characteristic, not a surprise.

My domain, in plain words (glossary)

Term	Plain meaning, with an everyday analogy
Finding ID	A stable name for one discovered weakness (letter = pass, number = order in that pass). A named scar with a story — everyone on the team says "B-1" and means the same wound.
Fail-open	A control that lets everything through *when it fails*. A doorman who faints and props the door open. Fail-closed is the opposite: he faints and the door locks.
Transaction / commit / rollback	An all-or-nothing bundle of database changes — every item scans or the whole basket goes back. *Commit* is the till drawer closing (changes become permanent); *rollback* is the receipt torn up and the shelf restocked, which is what an exception triggers.
Race condition	Two operations interleaving badly because both "checked" before either "acted". Two people reading the last concert ticket on the board and both walking to the counter.
FOR UPDATE	A database row lock: "I'm being served about this row — everyone else waits." The numbered ticket dispenser at the deli counter.
Connection pool	A fixed set of database connections shared by all requests. Ten changing rooms for the whole shop: hold one too long (or hold two at once) and someone queues.
Nested transaction	Opening a second connection inside a transaction that is still open — holding two changing rooms at once, the root of C-1.
Phantom audit	An audit row that survived the rollback of the event it describes — a history book that records battles that never happened. Worse than a missing row, because the trail lies.
Best-effort write	A side effect we attempt but never let break the main result — if it fails, the important thing already committed. Spraying wax on the car after the engine is fixed: nice if it lands, irrelevant if the sprayer is empty.
Family revocation	Killing every refresh token belonging to one user at once, because one of them proved stolen. Changing all the locks, not just the lost key's door.
Rotation	Issuing a brand-new refresh token on every use and retiring the old one — so replaying an old one is itself the alarm. Tear-off ticket stubs: presenting yesterday's stub means someone kept your roll.

Term	Plain meaning, with an everyday analogy
Append-only audit trail	A history table rows can only be added to, never edited or deleted (enforced by database triggers). A carbon-copy receipt book — you can add a page, you cannot erase ink.
Exploitability × blast radius	Our triage formula: how easily a stranger can pull the trigger, times how much of the system is behind it. Contagiousness times severity.
Grep	The auditor's scalpel: search the codebase for a finding ID and every place that finding touched lights up. grep -rn "B-1" server/src/main/java is the whole demonstration in one line.
Fail-fast boot & hardening batch	Fail-fast: the server refusing to start on bad configuration, with the remedy printed — a pilot's pre-flight checklist that stops at the gate. A *hardening batch* is a named group of fixes shipped together, each batch leaving the tree building clean and booting — moving the wiring, then the plumbing, then the paint, checking after every crew.

My ultimate Q&A bank

From the per-student pack (the classics)

Q: "What was your most serious finding?"

B-1, the refresh-token reuse detection that rolled back its own revocation. It is the nastiest class of bug — the security control exists in the code, but the transaction boundary made it fail open exactly when it mattered. The fix separates 'revoke the family' into its own committed transaction before the 409 is raised, with the audit event written best-effort afterwards so an audit outage can never undo a revocation. You can read the whole story in the comments around TokenService.rotate.

Q: "Explain the phantom audit bug — why is it dangerous?"

C-1. The old code opened a second connection inside an outer transaction to write the audit row, and the two committed independently. So a business operation could roll back — the thing never happened — while its audit row survived: the trail then claims events that never occurred, which is worse than a missing row in a system whose promise is tamper-evident history. The fix is transaction hygiene: one connection per transaction through the shared runner, and post-commit side effects written best-effort, so the trail can only lag, never lie. The fix comments in `ShareService` and `SettingsService` record the rule exactly where the bug lived.

Q: "How did you prioritize eight passes' worth of findings?"

By exploitability times blast radius. Anything that failed open, leaked a credential, or crossed an ownership boundary was batch 1 — 'someone can get in or see someone else's data'.

Integrity and tamper-evidence was batch 2. Concurrency and robustness cleanups were batch 3. And documentation-level risks got logged with a named fix path rather than pretend-fixes. Nothing was closed because we felt like it — each closure points at a finding-ID comment in the code.

Q: "Did the audit ever produce a false alarm?"

Yes, and that is worth saying out loud. The OTP overhaul initially looked like the overhaul had broken something when our rehearsal API calls started hitting 429s — but the limiter was doing its job: machine-speed repeated calls from one IP legitimately exhaust a 10-per-minute auth bucket, and a real user doing password-reset, MFA and step-up in a minute would come close too. The honest fix was a scope-aware budget — 20 a minute for auth, 30 for public share endpoints — not disabling the limiter. Knowing the difference between 'the system is broken' and 'the system is protecting itself' is the core audit skill.

Q: "Give one finding that was not a classic security hole."

H-1: the default document download resolved to the wrong version when the current-version pointer and the mirror disagreed, so a user could fetch bytes other than the version they were shown. The fix pins download resolution to one definition of current. H-3 was an orgName join that silently dropped organisations with unusual casing, and F-4 pinned the client's document versions so a stale screen cannot overwrite a newer one. None of these leak data on their own — all of them break a promise the UI makes, which is why they were worth a pass of their own.

The masterclass ten

Q: "How were eight passes checklists and not just vibes?"

Each pass started as a written checklist of the ways its theme fails in real systems — for concurrency: "find every place a second connection opens inside a transaction"; for configuration: "list every environment variable and what a silent wrong value does". I then walked the relevant files against the checklist and wrote an ID for anything that answered badly. That is why the IDs cluster: B-1, B-2 and B-3 are all answers to the same written question about token lifecycle. "We looked at everything" is not evidence; "these eight checklists were run against these folders" is.

Q: "Why is B-1 the nastiest* class *of bug, not just one bug?"

Because it is fail-open: the control exists, the code path exists, even the intent exists — and the surrounding machinery (the transaction rollback) silently defeats it exactly when it fires. A missing control is loud in code review; a defeated control looks like a control. The only reliable way to catch the class is to trace what actually commits when the error flies, which is exactly what our review of `TokenService.rotate` now documents in its comments for the next reader.

Q: "What did the phantom audit (C-1) actually break?"

Trust in our own history. DocuTrust's promise is tamper-evident history — every security-relevant event leaves an append-only row. C-1 meant the history book could record events that never happened: a business operation rolls back, its audit row survives on the second connection. Anyone querying `audit_logs` would be reading fiction. The fix makes one connection the unit of truth; the shipped comments in `ShareService` and `SettingsService` record the rule where it happened, and the phantom can never return because the second-connection overload no longer exists in the tree.

Q: "You fixed the same reuse bug three ways — B-1, B-2, B-3. Why three?"

Because a security action has three independent parts: the effect (revoke the family), the evidence (the audit row), and the race window (concurrent rotations). B-1 makes the effect durable before the error unwinds; B-2 makes the evidence post-commit and best-effort so it can never veto the effect; B-3 closes the race so two simultaneous refreshes cannot both pass the "is it still live?" check. Fixing only B-1 would have left the audit row able to roll the revocation back indirectly, or two racing requests double-issuing tokens. The trio is one correct action.

Q: "How do you audit without commercial scanners — isn't manual review unreliable?"

Manual and systematic beats automated and shallow for a codebase our size: a scanner would have flagged dependencies, not found that our revocation rolled back with its own exception. The discipline that makes manual work is the checklist per pass, the ID per finding, and the comment per closure — those three turn an opinion into a record. We also had an advantage a scanner lacks: we could read every line, because the whole server is a few thousand lines of framework-free Java.

Q: "Why eight passes instead of one giant pass?"

Because attention is single-threaded: reading one file for eight kinds of danger means you hunt none of them well. One pass per theme means each reading of a file has exactly one question, so a concurrency bug cannot hide behind a prettier crypto question. The side effect is the ID system itself — the letter tells you which lens found it, which tells the fixer which kind of care the fix needs (a concurrency fix wants a lock-ordering story; a leakage fix wants a payload check).

Q: "D-1 and E-3 both refuse to boot. What is the difference, and why two checks?"

They guard two different secrets with two different failure modes. D-1 is about the vault key's *shape*: it must be a 44-character base64 string decoding to exactly 32 bytes, because AES-256 needs precisely that much key material — a wrong-shaped key would fail later, mysteriously, at decryption time. E-3 is about the JWT secret's *strength and provenance*: at least 256 bits for HS256 and never a placeholder, because a guessable signing secret means

forgeable identities. Both run at config load, before database connections, so a bad configuration dies at the gate with the remedy printed.

Q: "How does 'every closure points at a comment' actually work?"

Each fix landed together with the comment that carries its ID: `TokenService.rotate`'s B-1/B-2/B-3 block explains the committed revocation, the post-commit audit and the row lock; the C-1 comments in `ShareService` and `SettingsService` record the one-connection rule where the bug lived; the D-1 and E-3 comments sit exactly on the config checks they describe. If a closure has no greppable comment, it is not closed — it is a wish. That rule is why three batches in a row can be audited the same way: search the ID, read the story, watch the flow live.

Q: "Did the audit change how the team writes code, beyond the fixes?"

Three habits became house rules. Transactions hold exactly one connection, and anything that must survive an error gets its own committed transaction before the error flies. Side effects after a commit are best-effort by design — the trail may lag, never lie. And configuration fails closed: any value the server cannot afford to guess (signing secret, vault key) is validated at boot with the remedy printed. You can see all three habits in code written *after* the audit, because the comments cite the IDs that taught us.

Q: "How do you demonstrate an audit in two minutes on stage?"

Two moves. First, `grep -rn "B-1" server/src/main/java/com/docutrust/auth/TokenService.java` — the comment tells the whole story in the authors' own words, which is stronger than any slide. Second, live behaviour: replay a rotated refresh token against the running local server and show the `409 TOKEN_REUSE`, then point at the database or the log to show every session's refresh token died with it. An audit you cannot show in the product is a document; this one is both.

Q: "What is still on the audit's open list — what would you fix next?"

The named, honest items: notification transport is still in-app only (no email or push for expiry reminders), the mobile session list is client-side only, and the docs-level risks we logged carry fix paths rather than pretend-fixes — that is what the universal "weakest point" answer says, and I own it the same way. None of them weakens the security core: the fail-closed boot, the committed revocation, the single-connection transactions and the version pinning are all shipped and demonstrable. An audit that ends with "everything is perfect" is the one you should distrust.

The handshake — how my slice feeds the other five

- **If you ask me about the router, tables or triggers → that's Student 1's area.** My part feeds it as the stress pass: the C-1 transaction hygiene lives in the shared database

layer Student 1 owns, the C-2/C-13 splitter fix is in the migration runner, and every append-only audit guarantee I lean on is a trigger Student 1 built.

- **If you ask me about JWT internals, rotation or OTP → that's Student 2's area.** My part found where it hurt: B-1/B-2/B-3 are Student 2's token lifecycle made durable, and the 429 false alarm was the rate limiter's budget being tuned (20/min auth, 30/min public share) rather than switched off. I attack it; Student 2 explains how it works.
- **If you ask me about the vault, shares or verification → that's Student 3's area.** My part pinned its promises: D-1 gave the vault key its exact shape, F-4 stops a proof link from silently re-pointing at different bytes, H-1 pins downloads to one definition of "current", and H-3 completes the payloads its screens render.
- **If you ask me about screens, ApiClient or UX flows → that's Student 4's area.** My part is the reason several of its branches exist: the client's `409 TOKEN_REUSE` logout, the step-up phases and the OTP-era UX (FT-231) are the user-facing halves of audit findings — Student 4 made the fixes feel like product instead of patches.
- **If you ask me about build, config or deployment → that's Student 6's area.** My part wrote the checks it boots with: E-3 and D-1 are the fail-fast config validation that stops the server starting on weak or misshapen secrets, and the `.env.example` placeholder blocklist is E-3's rule applied to documentation.
- **Who most often bridges TO me: Student 2.** Every token-lifecycle question walks from "how does rotation work" straight into "and the audit found it once failed open — B-1" — so we rehearse the refresh-replay demo together; Student 3's version-pinning story (F-4) is the second-most common bridge.

Student 6 — Deployment & DevOps

This file is the complete rehearsal brief for the student presenting **Student 6 — Deployment & DevOps** in the DocuTrust oral defence (Group 3). It is written to be read aloud from, handed to examiners, and pointed at during the live demo. It expands the matching role section in `DEFENCE_GUIDE.md` — the master index at the repository root — which carries the one-page version of this role. Everything here is grounded in the real repository: every class, method, variable and number below can be opened and verified live during the defence.

My slice in one page

Role in one line: Getting it running — anywhere, in one command — and keeping it observable, repeatable and secret-clean.

Owns:

- the Maven build (`server/pom.xml`) and the pinned dependency set it declares

- `.env` configuration with fail-fast validation (`config/Config.java`) and both `.env.example` templates (server and mobile)
- the root `.gitignore` — the fence that keeps secrets, build output and local-only files out of GitHub
- the demo seeder (`bootstrap/Bootstrap.java`), the `ExpiryScheduler` background job, and the metrics/health endpoints (`infra/Metrics.java`, `controller/HealthController.java`)
- the optional Sentry Spotlight tracing switch (`infra/Spotlight.java`) and the graceful shutdown path (`Main.App.stop`)
- the `refresh_tokens` session schema already includes the device/IP metadata in V01, so the Active Sessions list is built on a schema that already supports the feature

Feature claims: Infrastructure libraries (HikariCP, Bucket4j, Caffeine, Dropwizard Metrics, Sentry) plus the operations surface: boot, health, metrics, scheduler, seed.

My numbers:

- **74 API routes** — registered in `Main.start`, printed at boot as `routes=74`
- **12 migrations** — applied by `Migrator.apply` on first boot; the boot log prints `migrations=12`
- **8 pinned runtime jars** — declared in `pom.xml` (PostgreSQL 42.7.14, HikariCP 5.1.0, bucket4j-core 8.10.1, caffeine 3.1.8, metrics-core 4.2.25, sentry 7.14.0, slf4j-api 2.0.13, slf4j-nop 2.0.13)
- **16 active configuration variables** documented in `server/.env.example` (17 entries counting the optional, commented `DB_SCHEMA`)
- **Scheduler:** first tick after 15 s, then every 60 s; notifications de-duplicated to at most one per document, per type, per calendar day
- **Seeded demo:** runs only on an empty `users` table of a localhost database — an org, six accounts, 2 folders, 5 documents, 1 share, 1 approved verification
- Server: **one Java 21 process**, default port **8080**, 32 request threads, pool min 2 / max 10
- **One-command build and boot** — `mvn clean package` then `mvn exec:java`, both working from a clean clone (demonstrated live at the defence)
- `/api/meta` advertises `screens: 45`, `features: 216`
- Pool knobs I configure: **connection timeout 10 s, max lifetime 30 min** ("retire connections before Postgres default idle timeouts")
- Mail timeouts: **10 s connect, 15 s request** (`MailService`'s HTTP client); provider failure surfaces as **502 MAIL_SEND_FAILED**

- Jar artifact: **`docutrust-server.jar`** (`<finalName>docutrust-server-${project.version}</finalName>` in the pom), Main-Class `com.docutrust.Main`

  Note: *every claim in this brief is verifiable from the public repository alone — the pom is the build of record,* `mvn clean package` *turns a clean clone into the runnable jar, and the boot, health and metrics endpoints are demonstrated live at the defence.*

Elevator pitch (≈ 1 minute)

> *"I own getting it running — anywhere, in one command. The server is a single Java 21 process: one command —* `mvn clean package` *— fetches the exact eight libraries from Maven Central, compiles and packs the runnable jar, and* `mvn exec:java` *boots it with a fail-fast config check: a weak JWT secret or a missing vault key stops startup with a human explanation instead of a runtime surprise. Configuration is sixteen environment variables documented in* `.env.example`*, including the Brevo mail credentials and the MAIL_MODE switch — dev mode routes OTP codes to the log and an X-Dev-OTP header for demos, live mode sends real email through Brevo's API. Operations got real telemetry this sprint: HikariCP pool stats and Dropwizard request metrics surface on /api/health, and Sentry Spotlight tracing is one optional flag. The whole project zips to a submission with no build artifacts or local paths, and the demo seed brings up six accounts and sample documents on an empty database. Proof: from a clean clone, Maven packages and boots the server with the same* `.env`*, the same migrations and the same boot log — demonstrated live at the defence."*

Then demo: the live local boot — console up, `.env` loaded, and the proof line appearing: `[main] DocuTrust ready on port 8080 (migrations=12, routes=74, db=up, boot=…ms)` — followed by `curl http://localhost:8080/api/health` showing `db: true`.

What I own, file by file

Every path below exists in the repository today — open any of them during the defence.

Build and dependencies

File	What it does, in one plain-English sentence
server/pom.xml	The Maven build of record: Java 21 (maven.compiler.release), eight pinned runtime dependencies, a jar built as docutrust-server with com.docutrust.Main as the Main-Class, an exec-maven-plugin configuration for mvn exec:java, and a deliberate comment noting there is no Surefire/JUnit — the build's job ends at the runnable jar.

File	What it does, in one plain-English sentence
Local builds	For local builds, use mvn clean package and java -jar on the resulting docutrust-server.jar — Maven fetches the eight pinned libraries from Maven Central by itself.

The eight pinned dependencies, one line each

Every library is a named decision — `mvn clean package` resolves exactly the eight libraries the pom declares:

Dependency (version)	Why it is on the classpath
org.postgresql:postgresql (42.7.14)	The JDBC driver — the translator that speaks PostgreSQL's wire protocol for every query.
com.zaxxer:HikariCP (5.1.0)	The connection pool — the professional restaurant host behind db/Database.java.
com.bucket4j:bucket4j-core (8.10.1)	Token-bucket rate limiting that is exact under concurrency — the bouncer's clicker behind auth/RateLimiter.java.
com.github.ben-manes.caffeine:caffeine (3.1.8)	Bounded in-memory caches — holds the limiter's bucket map and the document read cache with size/time ceilings.
io.dropwizard.metrics:metrics-core (4.2.25)	The meters and timers behind infra/Metrics.java and /api/health.
io.sentry:sentry (7.14.0)	Optional tracing — inert unless SPOTLIGHT_ENABLED=true points it at a local Spotlight sidecar.
org.slf4j:slf4j-api (2.0.13)	The logging facade HikariCP and Sentry speak; kept quiet by the binding below.
org.slf4j:slf4j-nop (2.0.13)	The silent binding — one deliberate operational log stream (ours), no framework chatter.

The pom's comment says it outright: `Deliberately no surefire/junit` — the build compiles and packs the runnable jar, and that is all it promises; correctness is reviewed in the code and demonstrated live at the defence. The comment itself is public pom text, so an examiner can read it in place.

Configuration and boot

File	What it does, in one plain-English sentence
server/src/main/java/com/docutrust/config/Config.java	Loads configuration once — system properties first, then environment variables, then an optional .env in the working directory — and runs the fail-fast gates in its private constructor: the weak/placeholder JWT-secret refusal (E-3), the vault-key rules (D-1 plus the refuse-when-vault-has-files guard), the localhost-only demo-seed rule, and the live-mail credential check; a failed gate prints a human explanation and exits.
server/.env.example	The documented template of every configuration variable, with placeholder values the server actively refuses (the JWT placeholder sits on the E-3 blocklist) — you copy it to .env and fill in real values that never leave the machine.
mobile/.env.example	The mobile twin: ships EXPO_PUBLIC_API_URL=http://YOUR_PC_IP_ADDRESS:8080 with instructions to substitute the developer's LAN IP, so the phone and the server find each other on the same Wi-Fi — and no real address is ever committed.
.gitignore (root)	The customs list for the whole tree: secrets, build output, dependencies, editor settings and OS noise never reach GitHub — so a stray git add . cannot leak any of them.

The operations surface — boot, telemetry, background work, seeding

File	What it does, in one plain-English sentence
server/src/main/java/com/docutrust/Main.java	The boot sequence and the shutdown story: Config.load(), migrate, build the pool, wire services, register the 73 routes, seed, start the scheduler, open the HTTP server, print the proof line — and a JVM shutdown hook whose App.stop() stops the scheduler, stops the server and closes the pool in order.
server/src/main/java/com/docutrust/infra/Metrics.java	One shared Dropwizard MetricRegistry: route(method, pattern) marks a meter and opens a timer per route pattern (bounded cardinality), while requestsTotal() and the call-count-weighted meanLatencyMs() summarize it for the health endpoint.

File	What it does, in one plain-English sentence
server/src/main/java/com/docutrust/controller/HealthController.java	The two public telemetry routes: /api/health returns db (a live SELECT 1 via Database.healthy()), uptimeSeconds, requestsTotal and meanResponseMs; /api/meta returns the app's self-description — name, tagline, roles, screens: 45, features: 216 and the infra block naming HikariCP, Bucket4j, Caffeine, Dropwizard Metrics and the mail mode.
server/src/main/java/com/docutrust/db/Database.java (pool knobs)	The operational face of the pool I configure at construction: min idle 2 / max 10, a 10-second connection timeout, a 30-minute max lifetime so connections retire before Postgres idle timeouts, and poolIdle()/poolActive() exposing live occupancy through the HikariPoolMXBean.
server/src/main/java/com/docutrust/service/ExpiryScheduler.java	The housekeeping heartbeat: a single daemon thread named expiry-scheduler, first tick after 15 seconds then every 60, running markExpiring() (active → expiring, with expiry_warning notifications), markExpired() (→ expired, with document_expired notifications) and expireVerifications() (stale approved proofs stop answering valid), each pass isolated so one failure never kills the tick.
server/src/main/java/com/docutrust/bootstrap/Bootstrap.java	The demo seeder: refuses non-local databases outright, runs only when users is empty, then in one transaction creates the demo org and accounts, two folders, five hand-built PDF documents stored through the real FileVault.encrypt path, a live 7-day share link, an approved (version-pinned) verification request and starter notifications.
server/src/main/java/com/docutrust/infra/Spotlight.java	The optional tracing flag: only when SPOTLIGHT_ENABLED=true does Spotlight.init point Sentry at a local Spotlight sidecar on localhost:8969; default OFF, and an init failure is logged and swallowed so telemetry can never take the server down.

The `.env.example` variables, one by one

Variable	What it controls	What happens if wrong or missing
PORT	The HTTP port the API listens on (default 8080)	Falls back to 8080
DB_URL	The PostgreSQL JDBC URL (local form, or the Supabase session-pooler form documented in the file)	Main exits with a pointer to .env.example

Variable	What it controls	What happens if wrong or missing
DB_USER / DB_PASSWORD	Database credentials	Passed to the pool; missing values fall through to the URL
JWT_SECRET	The HMAC signing key for access tokens	Unset → an ephemeral secret is generated (tokens die on restart); shorter than 32 chars or the placeholder → refused at boot (E-3)
VAULT_KEY	The AES-256 vault key: 44-char base64 = exactly 32 bytes	Unset with a non-empty vault → refused (prevents fabricating tamper evidence); wrong format → refused (D-1); unset with an empty vault → requires ALLOW_EPHEMERAL_VAULT=true
ALLOW_EPHEMERAL_VAULT	Local-dev escape hatch for a throwaway key on an empty vault	Default false — fail closed
VAULT_DIR	Where encrypted document objects live (default ./data/vault)	Created on demand
SEED_DEMO	Whether to seed demo data (default true)	true on a non-localhost database → refused at boot
CORS_ORIGIN	The Access-Control-Allow-Origin value the server stamps on every response	Defaults to *
MIGRATIONS_DIR	Where the V*.sql files live (default ../database/migrations)	Missing directory → boot stops with MIGRATIONS_DIR hint
DB_SCHEMA *(optional)*	Isolate everything in a named schema (useful for throwaway local experiments)	Unset → default public
MAIL_MODE	dev (default): OTP codes go to the log and the X-Dev-OTP header; live: real Brevo email	live without credentials → refused at boot
BREVO_API_KEY / BREVO_SENDER_EMAIL	The Brevo transactional-mail credentials required by MAIL_MODE=live	Missing in live mode → refusal; rejected by Brevo → 502 MAIL_SEND_FAILED

Variable	What it controls	What happens if wrong or missing
BREVO_SENDER_NAME	The display name on outgoing mail (default DocuTrust)	Falls back to the default
SPOTLIGHT_ENABLED	Turns on Sentry → Spotlight local tracing (localhost:8969)	Default false — no telemetry egress

What the boot refuses, and the exact message

Fail-fast is only credible if the refusals are real sentences. Every gate below was read from the code (`Config.java`, `Main.java`, `Migrator.java`) — an examiner can trigger any of them live in seconds:

Broken setup	What the server does and prints
DB_URL blank	DB_URL not configured - set .env (see .env.example) → exit 1
JWT_SECRET shorter than 32 chars or the placeholder	JWT_SECRET must be at least 32 characters and must not be a placeholder… Generate one with: openssl rand -base64 48 → exit 1 (E-3)
VAULT_KEY unset but the vault directory holds files	VAULT_KEY is unset but vault contains existing files. Refusing to start (this prevents fabricating tamper evidence)… → exit 1
VAULT_KEY unset, vault empty, no escape hatch	Points at VAULT_KEY or ALLOW_EPHEMERAL_VAULT=true → exit 1
VAULT_KEY not 44-char base64 / not 32 bytes	VAULT_KEY must be a 44-character base64 string encoding exactly 32 bytes… → exit 1 (D-1)
SEED_DEMO=true with a non-local DB_URL	SEED_DEMO=true is only permitted with a localhost database. Demo credentials are in source control. → exit 1
MAIL_MODE=live without Brevo credentials	MAIL_MODE=live requires BREVO_API_KEY and BREVO_SENDER_EMAIL… → exit 1
MIGRATIONS_DIR does not exist	Migrations directory not found: … (set MIGRATIONS_DIR) → boot stops
Port already bound	[main] fatal: … from main's catch block → exit 1
SEED_DEMO=false (the polite case)	[bootstrap] SEED_DEMO=false - skipping demo seed — boots on, unseeded

The operations endpoints, and who reads them

Endpoint	Registered as	Returns	Who consumes it
GET /api/health	publicRoute (no auth)	ok, db (live SELECT 1), uptimeSeconds, requestsTotal, meanResponseMs	Me (and any monitor): is the engine running, is the database up, how busy are we
GET /api/meta	publicRoute (no auth)	name, tagline, description, roles, screens: 45, features: 216, infra block (pool, rateLimiter, cache, metrics, mail mode)	The mobile About screen (FT-192, FT-207), and anyone auditing which libraries we run

The scheduler's three passes, per 60-second tick

Pass	SQL move	Side effect
markExpiring()	UPDATE documents SET status='expiring' WHERE status='active' AND expires_at <= CURRENT_DATE + reminder_days … RETURNING id, owner_id, title	expiry_warning notification per owner — only if notifyOncePerDay finds no row for this document + type today
markExpired()	Same shape, status IN ('active','expiring') AND expires_at < CURRENT_DATE → 'expired'	document_expired notification, same once-per-day guard
expireVerifications()	UPDATE verification_requests SET status='expired' WHERE status='approved' AND expires_at < now()	None needed — the public verify link stops answering valid because the status changed

Each pass is wrapped in its own try/catch inside `tick()`, so one failing pass logs `[scheduler] … pass failed` and the other two still run; each pass is its own `db.tx`, so partial moves roll back cleanly.

The seed inventory, item by item

What `Bootstrap.seed` creates — in one transaction — on an empty, localhost `users` table:

Item	Detail
1 organisation	DocuTrust Demo Org
6 accounts	admin@docutrust.app (admin), manager@docutrust.app (org_manager), member@docutrust.app (member), student1–3@docutrust.app (members) — PBKDF2-hashed passwords
2 folders	Identity, Work (owned by the member)
5 documents	International Passport (active, expires +30 d), Driver Licence (expiring, +3 d), Old Visa (expired, −10 d), Medical Report (pending_review, review submitted → queues an admin_reviews row), Employment Contract (active)
1 share	7-day view link on the passport, minted through Text.token()
1 verification	Approved, version-pinned request on the contract for Acme Employer (30 days)
Notifications	Starter expiry_warning, share_opened, verification_approved rows, plus an admin_review notice for every admin

Every PDF is generated by `Bootstrap.pdf()` — a minimal but valid single-page PDF 1.4 with a correct xref table — and stored through `FileVault.encrypt`, so the seeded demo documents are indistinguishable from real uploads: encrypted bytes, hashed, versioned.

Five layman's analogies

1. **Fail-fast configuration is the pre-flight checklist.** Pilots don't discover a flat tyre at cruise altitude. Our server refuses to *start* with a weak signing key or a missing vault key, and prints exactly what to run to fix it — a two-minute fix on the ground beats a mystery crash in the air.
2. **MAIL_MODE is the flight simulator switch.** In `dev` mode the OTP codes appear in the console and the response header — you can rehearse the whole two-factor flow without emailing anyone. Flip to `live` and the same code path goes out through Brevo's real email service. Same cockpit, zero risk of spamming strangers while you learn.
3. **The migration runner is the moving truck with a checklist.** It looks at which numbered boxes (V01–V12) are already in the new house, carries in exactly the missing ones, in order, once each. Fresh database or five-months-old database, one boot command brings both to the same final layout.
4. **The health and metrics endpoints are the car's dashboard.** While driving you glance at the fuel gauge and the engine light instead of stopping and opening the bonnet

every hour: `/api/health` shows the engine light (`db` up or down), the odometer (`requestsTotal`), and the average speed (`meanResponseMs`). The one-line-per-request log is the journey diary; the dashboard is how you notice a problem before the passenger does.

5. **`.gitignore` is the customs list at the border.** GitHub is a foreign country every file must legally enter, and the list says what may never cross: real keys and passports (`.env`, `*.key`), unpackaged luggage (build output and downloaded dependencies), and personal belongings (editor settings). Only the blank application forms are waved through (`.env.example`, `mobile/.env.example`) — so even a careless `git add .` smuggles nothing in.

The critical code walkthrough

The boot sequence in order, naming the exact methods — this is `Main.main` → `Main.start`, verifiable by opening `server/src/main/java/com/docutrust/Main.java`.

1. `Config.load()` — the fail-fast gates. The private `Config` constructor reads variables through `get()` (system properties > environment > the `.env` file parsed by `load()`), then validates in order:

- `validateJwtSecret` (E-3): fewer than 32 characters, or the placeholder `change-me-to-a-long-random-string`, → prints the fix (`openssl rand -base64 48`) and `System.exit(1)`. An unset secret generates an ephemeral one with a clear warning that tokens will not survive restarts.
- Vault rules: if `VAULT_KEY` is unset but the vault directory already holds files, startup is refused ("this prevents fabricating tamper evidence"); with an empty vault it still refuses unless `ALLOW_EPHEMERAL_VAULT=true`. A provided key must pass `validateVaultKeyFormat` (D-1): exactly 44 base64 characters decoding to exactly 32 bytes.
- `SEED_DEMO=true` on a non-local database → refused, using `Config.isLocalDbUrl`, which parses the host out of the JDBC URL and **fails closed** on anything unparseable.
- `MAIL_MODE=live` without `BREVO_API_KEY` and `BREVO_SENDER_EMAIL` → refused ("live mode with no credentials would strand every OTP request at send time").

Because the constructor is private and these checks run inside it, no `Config` instance can exist that has not passed every gate — `Main` cannot even receive an unvalidated config. Three properties make the gate wall strong:

- the values land in `final` fields, so nothing downstream can swap a checked secret for an unchecked one after boot;

- the gates run in a fixed order — JWT secret, vault rules, demo-seed locality, mail credentials — so the first broken thing is always the one reported;
- the locality check `isLocalDbUrl` **fails closed**: a URL it cannot parse counts as non-local, so an odd format never sneaks past the seeder guard.

2. The early exits in `Main.main`. A blank `DB_URL` prints `DB_URL not configured - set .env (see .env.example)` and exits 1. Then `Main.start(cfg)` is called, and a JVM shutdown hook is registered so `Ctrl+C` runs `App.stop()`.

3. `Spotlight.init(cfg)` — first line of `start`, a no-op unless `SPOTLIGHT_ENABLED=true` (optional tracing never blocks the boot; its own failure is caught and logged).

4. The optional schema. If `DB_SCHEMA` is set, it is sanitised and `CREATE SCHEMA IF NOT EXISTS` runs once, with `SET search_path TO <schema>, public` passed to the pool as connection init SQL — handy for throwaway local experiments.

5. Pool init. `new Database(cfg.dbUrl, cfg.dbUser, cfg.dbPassword, 2, 10, initSql)` constructs the HikariCP `HikariDataSource`: pool name `docutrust`, minimum idle 2, maximum 10, connection timeout 10 seconds, max lifetime 30 minutes ("retire connections before Postgres default idle timeouts", says the comment).

6. The migration runner. `Migrator.apply(db, cfg.migrationsDir)` lists every `V*.sql`, sorts by filename, and for each file — inside one transaction — creates `schema_migrations` if needed, skips already-recorded names, executes the statements via the comment/string/dollar-quote-aware `split`, and records the name. The log prints `[main] migrations applied: N` (11 on a fresh database).

7. Wiring. `FileVault` is constructed with the decoded 32-byte key; then `JwtService`, `RateLimiter`, `Middleware` and every service and controller are built and handed their dependencies — the composition-root moment where the whole object graph is assembled in one readable file.

8. Route registration. The 73 `router.publicRoute(...)` / `rateLimited(...)` / `auth(...)` / `role(...)` / `anyRole(...)` calls run in labeled sections (public, auth, documents, folders, shares, public share links, verification, notifications, admin, organisation).

9. The seeder. `Bootstrap.seed(db, vault, passwords, cfg.seedDemo, cfg.dbUrl)` re-checks the two guards itself — `Config.isLocalDbUrl` refusal, then the empty-`users` check — and on a clean local database prints `[bootstrap] demo seed complete: 2 folders, 5 documents, 1 share, 1 verification`. With `SEED_DEMO=true` set while pointing anywhere non-local, `Main` never reaches this line (the config gate already exited); the second check is defense in depth.

10. The scheduler. `new ExpiryScheduler(db, notifications).start()` spawns the daemon thread `expiry-scheduler` with `scheduleWithFixedDelay(this::tick, 15, 60, TimeUnit.SECONDS)` and prints `[scheduler] expiry scheduler started (60s tick)`.

11. The listener. `HttpServer.create(new InetSocketAddress(cfg.port), 0)` binds the port (8080 by default — a taken port throws, the exception surfaces through `main`'s catch as `[main] fatal: …` and exit 1), `server.createContext("/", router)` hands every path to the router, `server.setExecutor(Executors.newFixedThreadPool(32))` sizes the worker pool, `server.start()` opens the doors.

12. The proof line. One line summarises the whole boot, with each number checkable: `[main] DocuTrust ready on port 8080 (migrations=12, routes=74, db=up, boot=…ms)` — migrations from `Migrator.apply`, `routes=74` from `router.routeCount()`, `db=up` from `Database.healthy()`, boot time measured from the first line of `start`.

13. Graceful shutdown. `App.stop()` runs `scheduler.stop()` (which calls `executor.shutdownNow()`), then `server.stop(0)`, then `db.close()` (the Hikari pool drains). That is FT-208 in `docs/FEATURES.md`, and it is why a demo can be stopped with `Ctrl+C` without orphaned connections.

A fragment worth quoting aloud, from `ExpiryScheduler.start()`:

```
executor = Executors.newSingleThreadScheduledExecutor(r -> {
    Thread t = new Thread(r, "expiry-scheduler");
    t.setDaemon(true);
    return t;
});
running = true;
executor.scheduleWithFixedDelay(this::tick, 15, 60,
TimeUnit.SECONDS);
```

The boot log of a clean run, line by line

What an examiner sees in the console, with the sender of each line — this is the transcript we rehearse against:

Console line	Printed by	Meaning
[main] migrations applied: 12	Main.start	The moving truck delivered every missing box
[main] DocuTrust ready on port 8080 (migrations=12, routes=74, db=up, boot=…ms)	Main.start	The proof line: schema up, 74 routes registered, database reachable, boot time

Console line	Printed by	Meaning
[bootstrap] demo seed complete: 2 folders, 5 documents, 1 share, 1 verification	Bootstrap.seed	Only on an empty local database (order varies by seed outcome)
[scheduler] expiry scheduler started (60s tick)	ExpiryScheduler.start	The heartbeat is armed
[http] GET /api/health -> 200 (2 ms)	Router.handle's finally block	One line per request, forever, method + path + status + duration
[main] shutting down...	the shutdown hook in Main.main	Ctrl+C ran App.stop() cleanly

The public runbook: clean clone to listening server

Every step below uses only what the public repository contains:

1. Install a JDK 21 (any distribution; the pom's `maven.compiler.release` is 21).
2. Create a PostgreSQL database — local, or the Supabase session-pooler URL form documented inside `server/.env.example`.
3. `cp server/.env.example server/.env` and fill it in; generate secrets with `openssl rand -base64 48` (JWT) and `openssl rand -base64 32` (vault key). Keep `SEED_DEMO=true` only if `DB_URL` is localhost.
4. `mvn -f server/pom.xml clean package` — Maven resolves the eight pinned dependencies from Maven Central and packs the runnable `docutrust-server.jar`.
5. From `server/`, run `mvn exec:java` — `Config.load()` finds `.env` in the working directory, the fail-fast gates check it, migrations apply, routes register.
6. Read the proof line; `curl http://localhost:8080/api/health` and check `db: true`.
7. Point the phone at it: `mobile/.env` with `EXPO_PUBLIC_API_URL=http://<your LAN IP>:8080` (see `mobile/.env.example`).
8. The flows then run live against this server at the defence — manual API calls (Postman/cURL) driving every OTP path, as rehearsed in the per-student packs.

What ships in the public repository

The public repository is clean by construction: secrets, build output and local configuration are ignored by pattern — and every claim in this pack maps to a path a fresh clone contains.

Ships in the public repository
server/pom.xml and all server source under src/main/java
server/.env.example and mobile/.env.example (placeholders only)
database/migrations/ V01–V12
The root .gitignore itself, plus all docs

Any examiner can verify the fence live: run `git check-ignore -v` on any ignored path and it prints the exact `.gitignore` rule that keeps it out. The result is a repository that is clean by construction and a defence where every capability is demonstrated in the room.

My domain, in plain words (glossary)

Term	Plain meaning, with an everyday analogy
Environment variable / .env file	A note pinned to the process's noticeboard before it starts: the same program reads different notes on your laptop, my laptop and the examiners' — code unchanged. The .env file is the physical noticeboard, one KEY=value per line in the project folder, loaded at boot — the house where the real keys live, and which never travels.
.env.example	The blank application form: every variable documented with a placeholder, safe to publish, and — in our case — with placeholders the server actively refuses to run on.
Fail-fast	The pre-flight checklist: refuse to start with a known-broken setup and say exactly why, instead of taking off and crashing at altitude (E-3 and the other Config gates).
JDK / JRE	The kitchen versus the dining room: a JDK can *cook* (compile) Java, a JRE can only *serve* (run) it — which is why the pom declares maven.compiler.release 21: a JDK 21 builds the jar, any JRE 21 runs it.
Jar	A zip of compiled Java with a label saying where the front door (Main-Class) is — our server is one jar plus eight library jars.
Maven	The recipe-and-shopping service: pom.xml lists exactly which ingredients at which versions, and Maven fetches, compiles and packages — the same list every machine builds from.

Term	Plain meaning, with an everyday analogy
Pinned dependency	Ordering the exact same dish by recipe, not "whatever the chef feels like": every library has a fixed version (HikariCP 5.1.0, not "latest"), so every machine builds identical behaviour.
Seeder	The model-flat furniture: Bootstrap.seed furnishes an *empty* database with demo accounts and documents so the app is demonstrable — and refuses to move furniture into an occupied building (non-local or non-empty database).
Daemon thread	A night-shift worker whose shift automatically ends when the building closes: the expiry-scheduler thread is marked daemon, so it can never keep a dying server alive.
Scheduled executor	An alarm clock that re-arms itself: first ring after 15 seconds, then every 60 seconds — scheduleWithFixedDelay, one ring at a time, never overlapping.
Graceful shutdown	Closing the restaurant properly: stop taking orders (scheduler), finish and lock the door (server), then count the till and hand back the keys (pool) — our App.stop() order, exactly.
Telemetry / metrics	The car dashboard: meters and timers (requestsTotal, meanResponseMs) collected while the server drives, read through /api/health instead of opening the bonnet.
Tracing (Sentry Spotlight)	The CCTV timeline for developers: an optional sidecar on localhost:8969 that draws one request's journey across the system — off by default so production ships with no egress.
Reverse proxy / TLS	The front desk and the armored window: a small web server (nginx-style) in front of the API that owns the HTTPS padlock and forwards traffic inside — how this plain-HTTP Java process faces the internet.

My ultimate Q&A bank

From the per-student pack (the classics)

Q: "How would you deploy this to real users?"

The server is a single jar with no framework: any JVM 21 host or container works. I would run it behind a reverse proxy for TLS (the API already sets CORS per config), point DB_URL at managed PostgreSQL — we already support the Supabase pooler form — set MAIL_MODE=live with the Brevo key, VAULT_KEY from a secret store, SEED_DEMO=false (the server itself refuses demo seeds on non-localhost databases), and

let the migration runner bring the schema up on first boot. Health checks already exist at /api/health.

Q: "What stops a developer from leaking production secrets into the repo?"

Three fences. First, secrets only exist in .env files, and a comprehensive .gitignore sits at the project root blocking secrets, build output, dependencies, editor settings and OS files from ever being committed — you can verify any pattern live with git check-ignore. Second, .env.example documents every variable with placeholders that the server actively refuses — the JWT placeholder is on a blocklist (E-3) — so nobody can run with example secrets by accident. Third, the committed template files carry no real values: mobile/.env.example ships YOUR_PC_IP_ADDRESS as its placeholder.

Q: "If the server crashes at 3am, how do you know what happened?"

Three layers. Every request logs one line — method, path, status, duration. Every security-relevant event lands in the append-only audit_logs table with actor, IP and JSON detail. And /api/health plus the Dropwizard counters on /api/meta show whether the database and the request path are healthy. Optional Sentry Spotlight tracing adds the distributed-timeline view when it is switched on. The logs answer 'what happened'; the audit trail answers 'who did it'; the metrics answer 'is it still happening'.

Q: "What if the demo machine can't reach Maven Central?"

The build fails loudly at the resolve step — Maven names the artifact it could not fetch, which is the honest behaviour for a pinned build. The mitigation we rehearse is Maven's offline mode against a pre-populated local repository cache: the eight versions are fixed, so a machine that built once can rebuild offline. Same code, same jars, no network surprises because the versions are pinned in `pom.xml`.

Q: "How does the scheduler avoid notifying twice about the same expiry?"

One notification per document, per type, per calendar day: before inserting, the scheduler checks for today's existing row with the same entity and type. A document sitting in the expiring state for two weeks produces fourteen reminders, not hundreds. The same sixty-second tick retires expired verification requests so their public links stop answering valid.

The masterclass ten

Q: "Configuration is read at boot — how do you change config without downtime?"

Every value is captured once into `final` fields of `Config` during the validated constructor, so a change is applied by restarting the process — and the restart is cheap by design: the shutdown hook drains the scheduler, server and pool in order, the migrations are idempotent so the second boot re-checks and skips, and nothing session-shaped lives in the server's memory (sessions are database-backed JWTs), so a new process is indistinguishable from the old one. The one caveat I name before the examiner does: boot with an explicit `JWT_SECRET`,

because the ephemeral fallback would invalidate tokens across a restart. For genuinely zero-downtime changes I would run two instances behind the reverse proxy and restart them one at a time — the stateless design permits it today.

Q: "What do the metrics actually count?"

One Dropwizard `MetricRegistry`, fed from exactly one place: `Metrics.route(method, pattern)`, called by the router around every handler execution. It marks a **meter** and opens a **timer** keyed by the route *pattern* — `POST /api/auth/login` — never by raw URL, so the namespace is bounded by the 73 registered patterns and cannot explode with ids. `requestsTotal()` sums every meter's count; `meanLatencyMs()` is the call-count-weighted mean of the per-route timer means. Because the timer is a try-with-resources block, failed and slow requests are counted too, not just happy paths. `HealthController.health()` publishes the two aggregates alongside the live `SELECT 1` database ping and the uptime.

Q: "Why does the seeder refuse to run against a non-local database?"

Because the demo credentials are in source control — that is their whole point, and their whole danger. `Config.isLocalDbUrl` extracts the host from the JDBC URL and only accepts `localhost` or `127.0.0.1`, deliberately failing closed on unparseable URLs; `Bootstrap.seed` re-checks the same rule before touching anything. So no accidental `DB_URL` pointing at our shared Supabase project can ever populate it with accounts whose passwords everyone in the room knows. The gate exists twice — once in the `Config` constructor at boot, once inside the seeder — because a safety check you can bypass by skipping a step is not a safety check.

Q: "What happens when Brevo is down or rejects a mail?"

`MailService` posts to Brevo's transactional API with a 10-second connect timeout and a 15-second request timeout; a network interruption, a timeout or a non-success HTTP status from Brevo is converted into an `AppException` with status 502 and code `MAIL_SEND_FAILED` — the one place our API surfaces a provider failure, documented in `docs/API_CONTRACTS.md`. The user sees an honest "email delivery failed" rather than a hang or a fake success, and can retry. And our demos are immune by construction: `MAIL_MODE=dev` never touches the network at all — the code goes to the log and the `X-Dev-OTP` header — so the entire OTP feature chain is rehearsable with the postman on strike.

Q: "Why are all eight dependency versions pinned?"

Two audiences, one truth. `pom.xml` is the single source every machine builds from; pinning means "it works on my machine" stops being an excuse — every clone resolves byte-identical libraries (HikariCP 5.1.0, Bucket4j 8.10.1, Caffeine 3.1.8, Metrics 4.2.25, Sentry 7.14.0, PostgreSQL 42.7.14, SLF4J 2.0.13). It also makes the build auditable: an examiner can read the eight versions in one file and know exactly what runs.

Q: "What exactly does the shutdown hook do, and why does the order matter?"

`Main.main` registers a JVM shutdown hook that calls `App.stop()`, and the order is the closing checklist: `scheduler.stop()` first, so no tick fires against a draining system; then `server.stop(0)`, so no new request is accepted; then `db.close()` (the Hikari pool drains). Reverse the order and you invite the classic exit crashes — a scheduler tick borrowing from a closed pool, or a request borrowing from a dying pool. This is FT-208 ("graceful shutdown closing pool + scheduler"), and it is why `Ctrl+C` during a demo is a clean stop rather than a stack trace.

Q: "Why is slf4j-nop on the classpath — are you hiding logs?"

The opposite: it is about owning the log line. HikariCP and Sentry speak SLF4J, and a SLF4J-using application with no binding nags on every boot about missing implementations. We ship `slf4j-nop` — the silent binding — because our operational interface is deliberate: one `System.out` line per request from the router, one boot proof line, one line per scheduler pass. Those are the lines we actually read at 3am, they are greppable, and they never drown in framework chatter. Adding a real logging backend later is a one-dependency change precisely because the facade is already in place.

Q: "`SEED_DEMO` defaults to true in `.env.example` — isn't that dangerous?"

It is the demo-first default, and it is fenced on three sides. The default only ever does something when the database is empty, and the localhost rule means it can only ever act on a developer's own machine — a remote or production URL makes the server refuse to start outright. The seed writes only the documented demo accounts and sample documents, and flipping one variable to `SEED_DEMO=false` — the production runbook value — disables it completely; the server then prints `[bootstrap] SEED_DEMO=false - skipping demo seed`. A dangerous default would be one that acts on shared infrastructure; this one physically cannot.

Q: "What happens if port 8080 is already taken when the server boots?"

`HttpServer.create(new InetSocketAddress(cfg.port), 0)` throws a bind exception, which surfaces in `main`'s catch block as `[main] fatal: …` with a stack trace in the console and `System.exit(1)` — a hard, immediate, loud failure, not a server that limps up half-alive. The fix is one variable: set `PORT` in `.env` and boot again. I put this in the same family as the E-3 secret checks: every boot-blocking condition is discovered in the first seconds, with an actionable message, because a process that cannot bind its port has nothing useful to offer.

Q: "Why is Sentry tracing off by default, and what does it even do here?"

Because telemetry is a promise about data leaving the machine, and a document vault should make that promise rarely. `Spotlight.init` runs only when `SPOTLIGHT_ENABLED=true`,

and even then it points at a Spotlight sidecar on `localhost:8969` — a developer's own machine, no external account, no cloud destination — drawing the timeline of a request across layers while I profile locally. Default OFF means production builds ship with zero telemetry egress, and the code is defensive about it: an init failure is caught, logged as "continuing without telemetry", and the server boots anyway. Observability should be a switch you flip for a reason, not a faucet left open.

The handshake — how my slice feeds the other five

- **If you ask me about the router, the pool or the schema → that's Student 1's area.** My side of the border: I launch and configure what they built. Their numbers are printed by my boot sequence — `migrations=12, routes=74` — and their pool (min 2, max 10) is sized by my `Database` construction, but the matching logic and the schema invariants are theirs.
- **If you ask me about JWTs, OTP flows or Brevo → that's Student 2's area.** My side: their OTP engine's two delivery modes are my two `.env` worlds — `MAIL_MODE=dev` feeds their codes to the log and the `X-Dev-OTP` header, `MAIL_MODE=live` hands them to Brevo — and my fail-fast gate refuses a live mode that would strand their flows mid-send.
- **If you ask me about encryption, shares or verification → that's Student 3's area.** My side: their encrypted bytes live in the `VAULT_DIR` my config points at, under the `VAULT_KEY` my boot validates to exactly 32 bytes before anything mounts — and my seeder creates their demo documents through their real `FileVault.encrypt` path, never around it.
- **If you ask me about screens, presenters or the mobile build → that's Student 4's area.** My side: the phone finds my server through the two env files I own — `mobile/.env.example` with `EXPO_PUBLIC_API_URL` on the app side, `CORS_ORIGIN` and `PORT` on mine — and their About screen reads its facts (`screens: 45`, `features: 216`) live from my `/api/meta`.
- **If you ask me about audit findings or the 8-pass audit → that's Student 5's area.** My side: two of their named findings are implemented *in* my files — E-3, the weak-JWT-secret refusal in `Config.validateJwtSecret`, and D-1, the 32-byte vault-key format in `validateVaultKeyFormat` — plus the `.gitignore` fences that close their information-leakage pass. Their IDs are cited in my code to this day.
- **Who most often bridges TO me:** everyone's live demo, honestly — the server must be up, seeded and observable before any of the five other slices can show anything. In DEFENCE_GUIDE.md's bridge map, build, config, secrets, seeding, metrics and deployment all route to this desk, and Student 5's hardening work most often ends up as changes in my files.

Appendix — The Eight Core Concepts, Explained for Absolute Beginners

This chapter assumes the reader has never written a line of code. Each of the eight sections explains one core concept with an everyday analogy, assigns it to the student who should rehearse it hardest, and ends with a one-sentence version you can say when the examiner asks for the short answer. Every folder name, number and library mentioned here is real — open the folder during the defence and point at it.

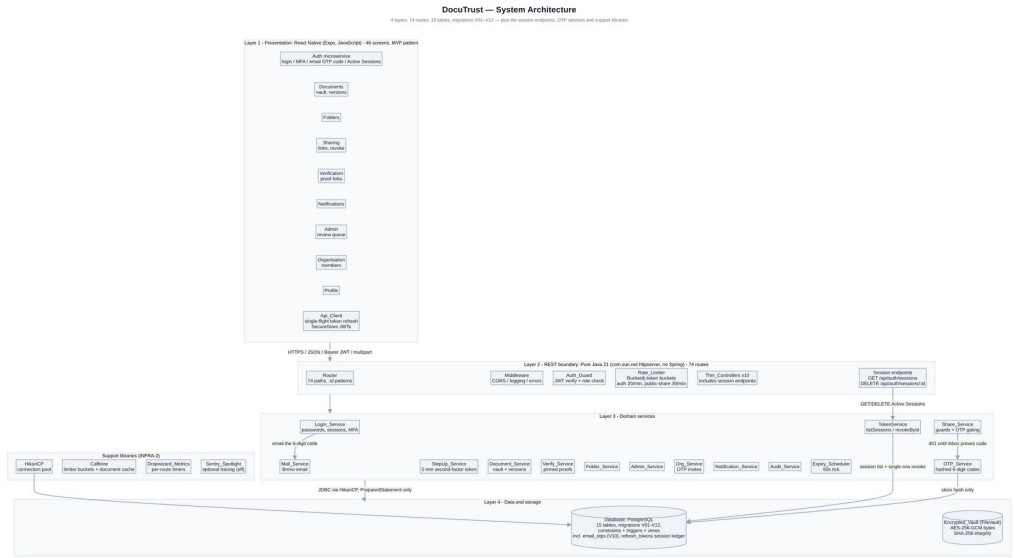


Figure A. The whole system in one picture: the React Native client, the pure-Java server with its 74 routes, and the PostgreSQL pantry beneath.

1. React Hooks — the sticky notes that remember things (rehearse: Student 4)

A phone app screen is redrawn hundreds of times a second — every tap, every new piece of data. But a drawing has no memory: wipe it and start again, and nothing is left over. A **hook** is a sticky note that React — the drawing library — keeps beside the drawing. The screen is wiped and redrawn all day long, but the sticky notes stay on the desk, remembering things between redraws. The word hook literally means that your screen component hooks into React's memory. Three sticky notes matter in our app.

useState — the sticky note with a magic pen. You write a value on the note, for example "unread notifications: 3", and the screen copies whatever is on the note. When a new notification arrives, you call the pen that came with the note; the note now reads 4, and React automatically redraws the screen. You never redraw anything by hand — changing the note is the whole job. Every counter, form field, loading flag and error message in our 46 screens lives on such a note.

useEffect — the "after you finish drawing, run this errand" slip. Drawing a screen should stay fast and dumb. The errands — fetching the document list from the server, starting a countdown, subscribing to notifications — go on the slip, and React runs each errand after the drawing is done. The slip also has a cleanup line: when the screen closes, the timer is stopped and the subscription cancelled, so nothing keeps running in the background like a tap left on.

usePresenter — our own custom sticky note, the one an examiner will ask about. DocuTrust does not let screens talk to the server directly; each screen gets a **presenter** — a plain-JavaScript "waiter" holding the screen's brain: what to fetch, what counts as a valid form, what to show while loading, on error, on empty. `usePresenter` is the three-line glue that hires the waiter when the screen opens, passes the waiter's status onto a sticky note so the screen redraws as it moves idle → loading → success → error, and fires the waiter when the screen closes so no presenter haunts a closed screen. One hook, and 46 screens all behave the same way.

One-sentence version: a hook is a sticky note that remembers things between redraws; useState remembers a value, useEffect remembers an errand to run after drawing, and our usePresenter remembers to hire — and later fire — the screen's waiter.

2. The Overall Architecture — the customer, the waiter, the kitchen (rehearse: Student 1)

DocuTrust is a restaurant with three rooms. The **customer** is the mobile app on your phone, built with React Native and Expo. It is good-looking and polite, but it has no kitchen: it cannot store your documents, check passwords or decide who may see what. It only orders.

The **waiter** is the backend server, a single Java program. Every wish from the customer — "show my documents", "share file X until Friday", "email me a login code" — is spoken to the waiter, never shouted into the kitchen. The waiter does four things, always in order: checks you are who you claim to be (a digital ID card called a JWT), checks you are allowed to order that (role and ownership rules), walks the order into the kitchen, and carries the plated result back on a standardized tray (a JSON envelope, explained in section 8). Our restaurant has exactly 74 dishes on the menu — the 74 registered API routes, a number printed when the server boots, so it is checkable rather than claimed.

The **kitchen and pantry** are the services and the PostgreSQL database. The chefs — the service code such as DocumentService, ShareService and OtpService — do the real cooking: encrypting files, minting share links, hashing one-time codes. The pantry is the database: fifteen filing drawers (tables) that remember every document, user, share link and audit entry, with a strict head clerk who refuses orders that break the house rules even when the chef made a mistake. Why three rooms? So each can be fixed alone: this sprint the entire OTP

flow was rebuilt inside the kitchen, and the customer barely noticed — only the ordering script changed.

One-sentence version: the app is the customer, the server is the waiter with 74 things on the menu, the database is the pantry — and no room ever walks into another room uninvited.

3. The Frontend Folders — it's like a smartphone (rehearse: Student 4)

Open mobile/src/ and you see seven folders. The easiest way to explain them: the app is a smartphone, and each folder is a part of the phone.

`screens/` are the different **apps you open** — forty-six of them: a Login app, a Documents app, a Trash app, a Share app. Each screen is one full-page view, and they are deliberately dumb: they draw what they are told and forward taps, like an app that only shows what its data says. `components/` are the **buttons on the screen** — small reusable pieces (a document card, a password input, a toast pop-up, a loading skeleton) drawn once and used on many screens, which is why the whole phone feels consistent.

`microservices/` are the **invisible background processes that fetch your data** — nine folders (auth, documents, folders, sharing, verification, notifications, org, profile, admin), one per feature area. Each holds that domain's api files (how to ask the waiter), model files (what the data looks like) and presenter files (the screen's brain). The user never sees these; like background processes they do the fetching and deciding. Why separated? So six students could work at once without stepping on each other — the sharing student rebuilt her whole OTP-gating flow and never touched the documents folder. The remaining folders are small: `navigation/` is the home-screen gesture (which app opens when you tap what), `mvp/` is the house rulebook every screen's waiter follows, and `core/` is the phone's settings — the server address and the design tokens.

One-sentence version: screens are the apps you open, components are the buttons on them, microservices are the invisible background processes fetching the data — and navigation is the home gesture that moves between apps.

4. The Backend Folders — a well-organized restaurant (rehearse: Student 1)

Open server/src/main/java/com/docutrust/ and you see the restaurant's staff chart. The `controller` folder is the **waiters taking orders** — ten of them (AuthController, DocumentController, ShareController and so on), one per section of the menu. A waiter's job is narrow on purpose: receive the order, check the ID, hand it to the kitchen, carry the plate back. Waiters never cook.

The `service` folder is the **chefs cooking** — the real logic: hashing one-time codes, encrypting files, enforcing share rules. Chefs never talk to the customer directly; that is the waiter's job. This split is why the team could swap the recipe for password resets — from "send a temporary password" to "send a one-time code" — without changing a single waiter. The `db` folder is the **pantry and the head clerk**: Database (the walk-in cooler connection), Migrator (the eleven numbered instruction sheets that build the pantry from scratch), Rows and Tx (the rules for reading a shelf and for all-or-nothing orders). The `vault` folder is the restaurant safe — files sit there encrypted with AES-256-GCM, a standard strong enough for banks.

The `auth` folder is the **ID checkers at the door**: PasswordService (checks your password without ever knowing it — it only compares scrambled fingerprints), TokenService (prints and re-prints your session wristband), MfaService (the second-proof checker) and RateLimiter (the bouncer who says "too many tries, wait a minute"). The `http` folder is the **uniforms and order pads** — Router (the 74-item menu index), Middleware (the security chain every order passes), Ctx (the standard order-slip format). Around them: config is the manager's opening checklist (it refuses to open with weak keys), infra is the office dashboard showing pool and traffic statistics, model holds the standard plate shapes, and util is the toolbox. Why separated? A new chef can be hired without retraining the waiters — and during the OTP overhaul the team rewrote chefs (OtpService, MailService) while the waiters and the pantry never changed.

One-sentence version: controllers are the waiters taking orders, services are the chefs cooking, db is the pantry, auth checks IDs at the door — and no waiter ever walks into the pantry himself.

5. The Database — a highly organized filing cabinet (rehearse: Student 1)

A **relational database** is a filing cabinet with strict rules, and PostgreSQL — the one we chose — is the strictest, most trusted cabinet on the market. The cabinet has fifteen drawers (**tables**): users, documents, document_versions, shares, email_otps, audit_logs, and so on. (Peek with a database tool and you will see a sixteenth — schema_migrations — but that is not one of ours: it is the moving truck’s own clipboard, where the migration runner writes down which numbered instruction sheets it already delivered.) Inside each drawer are **rows** — index cards, one per user, per document, per share link. Each card has the same printed **columns**: a user card always has name, email, role, created-at. You never find a hand-scribbled card with a different shape — the drawer's form is fixed when the drawer is built.

A **foreign key** is a cross-reference written on the card. A document card says "owner: see card 42 in the USERS drawer." The clerk refuses to file a document whose owner card does not exist, and — by our setting — refuses to destroy a user card while documents still point at

it. That is what "relational" means: the cabinet itself guarantees the references make sense, so a typo or a bug cannot create an orphan.

Why PostgreSQL, in plain words? Four reasons. First, it **enforces rules**: a CHECK constraint is a house rule the clerk applies even to the boss — a medical document physically cannot become "active" without review, no matter what buggy code asks for. Second, it **keeps honest history**: triggers make the audit cards append-only — they cannot be edited or torn out, only added. Third, it is **free and battle-tested for over twenty-five years**, running everywhere — including Supabase, a free cloud landlord for it, so our project costs nothing to host. Fourth, it speaks a standard language, SQL, that all of us could write. The cabinet was assembled from **eleven numbered instruction sheets (migrations)**: V01 builds the users drawer, V10 adds the email-OTP drawer, V11 adds three fields to shares — so any team member can rebuild the identical cabinet from nothing, in order, in minutes.

One-sentence version: tables are drawers, rows are index cards, foreign keys are cross-references the clerk refuses to break — and PostgreSQL is the strict, free, 25-year-old cabinet that enforces our house rules even against bugs.

6. The Server — a 24/7 receptionist answering phones (rehearse: Student 6)

What is a web server? It is a program that never sleeps, sitting by a phone that never stops ringing. Ours is one single Java process, and its day looks like this. The phone rings — a request arrives from the app. The receptionist picks up **within milliseconds, always**, whether it is one call or fifty at once. It asks "who is calling?" and checks the caller's ID card (the JWT): no card, or a fake card, gets a polite refusal — status 401. It looks up the requested department in its directory of **74 extensions** (the router) — and anything unknown gets a clean "no such extension" (404), never a crash. Then it walks the request to that department and carries the answer back, always on the same standardized tray (the JSON envelope), with a rubber stamp saying how it went.

Three more duties make ours a good receptionist. It **writes down every call** — one log line per request (what was asked, when, how long it took), and every security-relevant event into an append-only audit book that even the boss cannot tear pages from. It **limits repeated callers** — someone asking for a password reset eleven times in a minute is told to wait (rate limiting), which is how the system shrugs off people who try to guess codes. And it **refuses to open the office with broken locks** — at start-up it checks its own keys and configuration, and if the signing key is weak it refuses to start and prints exactly what to fix. That is called fail-fast: a two-minute fix on the ground beats a mystery crash in the air.

One-sentence version: a web server is a 24/7 receptionist: answers every call instantly, checks IDs, forwards to one of 74 departments, writes down everything, and never hangs up rudely.

7. The Tech Stack — what we chose and why (rehearse: Student 6)

Every technology in the project earns its place in one of three ways: the brief told us to, it is the boring trusted option, or it is the standard library for a problem too dangerous to hand-roll. The table below is the whole list — learn the "why" column and the examiner's question is already answered.

Java 21 is the language of the server — one recipe book that runs on any machine with a Java reader installed. The brief forbids frameworks, and Java is a twenty-five-year-old workhorse: strict (mistakes are caught before running), fast, and familiar to all of us; version 21 is a long-term-support release, the sturdy supported-for-years edition. **PostgreSQL, hosted free on Supabase**, is the filing cabinet of section 5 — strictest rule enforcement, free hosting, decades of trust. **React Native 0.88 with Expo SDK 58** is the phone-app builder: you write the app once in JavaScript and it runs on both iPhone and Android, like a translator fluent in both — and Expo is the training wheels: scan a QR code and the app runs on a real phone immediately, no app store needed for demos. The app even has its own mechanic: `npx expo-doctor` inspects the dependency list and currently reports 20/20 checks clean, and `npx expo install --fix` keeps every library on the version the SDK certified. Where the app finds the server is configuration, not code: `mobile/.env` holds EXPO_PUBLIC_API_URL and `src/core/config/env.js` reads it, while a root `.gitignore` keeps the real .env and local build output out of GitHub.

Brevo is the postman — an email-delivery service with a simple API. One-time codes must reach real inboxes; Brevo has a free tier and a two-line sending call, and our MAIL_MODE switch lets us rehearse the whole OTP flow without emailing anyone. **HikariCP** is the restaurant host for the database: it keeps a fixed set of tables (connections) ready and seats each query instantly — hand-rolling that seating is a classic source of 3am crashes. **Bucket4j** is the ration-book stamp that counts requests per minute per user, with budgets tuned per endpoint scope (20 a minute for auth, 30 for public share pages). **Caffeine** is the noticeboard by the kettle — the few hot facts everyone re-reads, kept in memory with size and time limits so the board can never go stale. **Dropwizard Metrics and Sentry Spotlight** are the office dashboard and the CCTV timeline: /api/health shows pool and request statistics live, and Spotlight (optional, one flag) draws the full timeline of a request across the system.

One-sentence version: every choice is either "the brief told us" (pure Java, no framework), "the boring, trusted option" (PostgreSQL, Java 21 LTS), or "the standard library for a problem too dangerous to hand-roll" (HikariCP, Bucket4j) — and we can name the danger for each.

8. The Data Consumption API — the standardized order form (rehearse: Student 3)

How does the app actually talk to the waiter? Two conventions make it work, and both are agreed paperwork. **HTTP methods are the verbs on the order pad**. Every request says what kind of wish it is. **GET** means "just let me look" — opening the documents list changes nothing. **POST** means "here is something new" — uploading a document, registering, requesting an OTP code. **PUT** means "replace this whole thing" — overwriting the notification settings. **PATCH** means "amend one field" — renaming a document. **DELETE** means "take this away" — moving a document to trash. The same address can behave differently by verb: GET /api/documents lists, POST /api/documents uploads, DELETE /api/documents/12 trashes document 12. Reading is safe; writing is checked twice — ID first, then permission.

JSON is the standardized order form — a way of writing data that both the app and the server read: labelled boxes in fixed positions, like {"title": "Passport", "status": "active"}. The phone speaks Android and the server speaks Java, but neither cares: they only swap forms. Every answer arrives in the **same envelope**: ok true with the data inside on success; ok false with an error code and a friendly message on failure — so the app opens every envelope the same way, and a screen that hits an error always finds a code it can translate into plain words.

The rubber stamp on the envelope is the **status code**, three numbers with fixed meanings worth memorising: 200 "here you go"; 400 "your form was filled wrong"; 401 "ID card missing or fake"; 403 "real ID, but not allowed" — or, with STEP_UP_REQUIRED, "allowed, but prove a second time first"; 404 "no such thing — and we will not say whether it exists for someone else"; 409 "conflict — the link is expired, revoked or out of views, or that token was reused"; 429 "too many requests, wait a minute"; 440 — our own stamp — "second factor needed, check your email"; 500 "our mistake, sorry — the details are in the log, not in your face."

One-sentence version: the frontend hands the backend standardized order forms — five verbs on the pad, JSON as the form's printed boxes, one envelope for every answer, and a rubber stamp (the status code) saying exactly how it went.

These eight explainers are also in the repository's *DEFENCE_GUIDE.md* (Part 2), next to the six per-student defence packs — the two documents share the same analogies on purpose, so the team tells one story.